Axion Control Plane Update Outline

Axion Control Plane Update Outline

1. Update Objective

Redesign the control planes so they remain the governing authority of the system while gaining AI-assisted support for analysis, recommendations, and operator efficiency.

Primary goal

- make the control planes smarter without making them non-deterministic

Core principle

- control planes keep authority
- AI provides bounded assistance
- governed actions remain deterministic, auditable, and enforceable

---

2. Update Intent

This update shifts the control planes from:

- partially defined operational shells

to:

- fully defined governance layers with AI support surfaces

The AI is not replacing the control planes.

The AI is being added as:

- advisor

- analyzer
- recommender
- classifier
- structured output generator

---

3. Architectural Direction

3.1 Deterministic core remains

The control planes still own:

- rules
- states
- transitions
- approvals
- permissions
- quotas
- audit
- release authority
- artifact integrity

3.2 AI support layer added

AI is introduced only inside bounded surfaces such as:

- recommendation points

- classification points
- summarization points
- anomaly detection points
- recovery suggestion points
- operator assistance points

3.3 Final architecture model

- **Artifacts define the roles**
- **Control plane services enforce the roles**
- **AI assists the roles**

---

4. Update Scope

4.1 In scope

- formalize the three control planes
- complete missing control plane roles
- add AI-assist operating boundaries
- define AI-safe action surfaces
- add auditability for AI-assisted decisions
- define human approval requirements
- preserve deterministic enforcement paths

4.2 Out of scope

- giving AI unrestricted authority
- replacing policy with model judgment
- replacing approvals with model decisions
- allowing AI to mutate protected state without validation
- removing audit or lineage requirements

---

5. Control Plane Structure

5.1 Execution Control Plane (ICP)

Owns:

- run lifecycle
- state transitions
- execution routing
- queue/scheduling
- environment selection

5.2 Governance Control Plane (GCP)

Owns:

- policy evaluation
- approvals

- overrides
- permissions
- quotas

5.3 Evidence Control Plane (ECP)

Owns:

- audit
- pinning
- release evidence
- artifact lineage
- notification evidence linkage

---

6. Control Plane Update Goals

6.1 Execution Control Plane goals

- stabilize run control behavior
- formalize state machine rules
- add queue and scheduler role
- add adapter/environment manager
- add AI-assisted failure analysis and recovery suggestions

6.2 Governance Control Plane goals

- complete approval routing
- complete override governance
- complete permissions/authz
- complete quota governance
- add AI-assisted policy interpretation and approval support

6.3 Evidence Control Plane goals

- strengthen audit role
- strengthen artifact lineage governance
- stabilize release governance
- formalize notification routing
- add AI-assisted evidence summarization and anomaly detection

---

7. AI Assistance Design

7.1 AI role in the updated control planes

AI will serve as:

- advisory engine
- reasoning assistant
- classification engine
- summarization engine
- anomaly detection layer

- structured recommendation producer

7.2 AI will not serve as

- final authority
- policy source of truth
- approval authority for protected actions
- permission grant authority
- state machine owner
- audit truth source

7.3 AI assistance modes

- advisory only
- propose and validate
- low-risk auto-action only where explicitly allowed
- human review required for protected actions

---

8. Proposed Model Strategy

8.1 Primary model

Qwen

- default AI assistant for control plane support
- best fit for code-aware, tool-using, structured operator workflows

8.2 Secondary reasoning model

DeepSeek

- used for heavier reasoning paths
- override analysis
- policy conflict explanation
- complex failure/postmortem synthesis

8.3 Optional coding sidecar

Devstral

- implementation help
- code issue workflows
- engineering support surfaces

---

9. AI Operating Boundaries

9.1 What AI may do

- classify failures
- summarize policy conflicts
- recommend retry paths
- recommend approver routing
- suggest override risk levels

- summarize audit trails
- detect suspicious patterns
- explain artifact drift
- produce structured recommendations

9.2 What AI may not do

- approve protected actions on its own
- bypass policies
- mutate audit records
- revoke or publish releases without governed checks
- change permissions directly
- alter quotas directly
- unpin protected artifacts without required approval
- force illegal state transitions

---

10. Role-by-Role Update Plan

10.1 ICP updates

- ICP-01 Run Controller
- ICP-02 Run State Store
- ICP-03 State Machine Governor

- ICP-04 Queue / Scheduler Manager
- ICP-05 Adapter / Environment Manager

AI additions

- failure classification
- retry suggestions
- execution path recommendations
- environment suitability scoring

10.2 GCP updates

- GCP-01 Policy Engine
- GCP-02 Approval Router
- GCP-03 Override Manager
- GCP-04 Permissions / AuthZ Manager
- GCP-05 Quota Manager

AI additions

- policy conflict summaries
- approval context drafting
- override recommendation support
- suspicious access pattern detection
- quota pressure forecasting

10.3 ECP updates

- ECP-01 Audit Logger
- ECP-02 Pin Manager
- ECP-03 Release Manager
- ECP-04 Artifact Registry / Lineage Governor
- ECP-05 Notification Router

AI additions

- audit summarization
- lineage anomaly explanation
- artifact drift interpretation
- release readiness summary
- alert prioritization support

---

11. New Artifact Group Required

11.1 Existing artifact groups remain required

- ICP artifacts
- GCP artifacts
- ECP artifacts

11.2 New AI governance artifact group added

AIG — AI Governance

Suggested artifacts:

- AIG-00 Purpose + Boundaries
- AIG-01 AI Decision Scope
- AIG-02 AI Authority Limits
- AIG-03 AI Input Contract
- AIG-04 AI Output Contract
- AIG-05 AI Action Gating
- AIG-06 AI Audit + Replay Rules
- AIG-07 Model Routing + Fallback Policy

---

12. Data and Contract Updates

12.1 New contracts needed

- AI request envelope
- AI response envelope
- recommendation schema
- confidence and evidence schema
- approval recommendation schema
- AI action eligibility schema
- model trace/audit schema

12.2 Existing contracts that need updates

- run manifest
- policy evaluation result
- approval request
- override request
- audit entry
- release record
- artifact registry entry
- notification event

---

13. Enforcement Rules

13.1 Deterministic checks stay mandatory

Every AI-assisted action must still pass:

- schema validation
- policy validation
- authorization checks
- state transition validation
- approval requirements
- evidence requirements
- audit recording

13.2 AI outputs must be treated as proposals unless explicitly whitelisted

Default rule:

- AI output is non-authoritative until validated

---

14. Safety and Trust Requirements

14.1 Required guarantees

- deterministic enforcement
- bounded AI authority
- full auditability
- replayable decision context
- protected action controls
- human oversight where required

14.2 Failure handling

- AI unavailable → control plane still functions
- AI low confidence → escalate to operator
- AI malformed output → reject and log
- AI/policy conflict → deterministic policy wins
- AI recommendation rejected → preserve full audit trail

---

15. Implementation Phases

Phase 1 — control plane stabilization

- formalize current role ownership
- complete missing control plane roles
- normalize contracts and schemas
- strengthen deterministic enforcement

Phase 2 — AI governance foundation

- create AIG artifact pack
- define AI input/output contracts
- define action gating rules
- define audit/replay rules

Phase 3 — bounded AI integration

- add AI support surfaces to each plane
- integrate Qwen as default assistant
- add DeepSeek for heavy reasoning paths
- keep AI outputs advisory-first

Phase 4 — operator workflow refinement

- approval assist flows
- failure triage assist flows

- audit summary views
- override review support
- release readiness summaries

---

16. Expected Outcome

After this update, Axion should have:

- three formal control planes
- deterministic system authority
- AI-assisted operational intelligence
- governed AI boundaries
- auditable AI usage
- higher operator efficiency without sacrificing control

---

17. Simple summary statement

This update turns Axion's control planes into deterministic governance systems with bounded AI assistance, not AI-governed systems.

Axion Control Plane Update Build Spec

Axion Control Plane Update Build Spec

1. Purpose

This update defines the next-generation Axion control plane architecture as a **deterministic governance system with bounded AI assistance**.

The goal is to improve intelligence, speed, and operator support across the system without allowing AI to become the governing authority.

This update establishes:

- the formal structure of the control planes
- the responsibilities of each control plane role
- the boundaries of AI assistance
- the contracts required for AI-assisted operation
- the enforcement rules that keep the system governed, auditable, and predictable

---

2. Objective

Build a control plane architecture that:

- remains deterministic at the authority layer
- supports AI-assisted analysis and recommendations
- preserves policy, approval, audit, and integrity enforcement
- scales into a complete operational system
- cleanly separates governance from assistance

The intended end state is:

Artifacts define authority
Control plane services enforce authority
AI assists authority within bounded surfaces

---

3. Problem Statement

The current control plane structure is partially defined and partially implemented. Core control-plane-adjacent surfaces exist, but the system does not yet operate as a fully formalized multi-plane governance architecture.

Current issues include:

- missing first-class roles for approvals, quotas, scheduling, permissions, and environment routing
- governance behavior that is only partially explicit
- evidence and lineage behavior that is incomplete
- no formal AI boundary model for safe advisory support
- risk that AI, if introduced without constraints, could blur authority and enforcement responsibilities

This update resolves those gaps by defining a complete control plane model before or alongside AI integration.

---

4. Design Principles

4.1 Deterministic authority

All governed actions must remain subject to deterministic validation and enforcement.

4.2 AI is assistive, not authoritative

AI may recommend, summarize, classify, predict, and draft structured outputs, but it does not become the source of truth for governed actions.

4.3 Explicit boundaries

Each control plane role must have explicit responsibilities, inputs, outputs, and dependencies.

4.4 Auditability

All control-plane-relevant actions, including AI-assisted actions, must be traceable and reviewable.

4.5 Replayability

Critical decisions must preserve enough context to understand and reconstruct why the action occurred.

4.6 Failure safety

The control plane must continue operating safely when AI is unavailable, low-confidence, or wrong.

---

5. Scope

5.1 In scope

This update includes:

- formalization of the Execution, Governance, and Evidence Control Planes
- formalization of their roles
- definition of role boundaries and contracts
- AI assistance integration model
- AI action boundaries
- AI audit and replay requirements
- new and updated schemas
- implementation phase structure

5.2 Out of scope

This update does not include:

- replacing deterministic policy with model judgment
- granting unrestricted authority to AI
- removing approvals for protected actions
- allowing AI to directly mutate protected state without governed checks
- removing audit, lineage, or integrity requirements

---

6. Target Architecture

The updated architecture consists of three control planes plus one AI governance layer.

6.1 Execution Control Plane

Owns execution flow and run movement.

6.2 Governance Control Plane

Owns policy, approvals, overrides, permissions, and quota governance.

6.3 Evidence Control Plane

Owns audit, integrity, release evidence, and artifact lineage governance.

6.4 AI Governance Layer

Defines how AI may assist the control planes without becoming them.

---

7. Control Plane Model

7.1 Execution Control Plane (ICP)

The Execution Control Plane governs run lifecycle and pipeline movement.

Its purpose is to ensure that runs move through the system legally, in order, and with explicit execution authority.

It owns:

- run creation
- run initialization
- stage advancement
- stage order enforcement
- state transitions
- queueing and scheduling
- environment selection
- execution routing

It does not own:

- policy approval
- override authority
- release authority
- audit truth
- operator permissions

Roles within ICP

- ICP-01 Run Controller
- ICP-02 Run State Store
- ICP-03 State Machine Governor
- ICP-04 Queue / Scheduler Manager

- ICP-05 Adapter / Environment Manager

7.2 Governance Control Plane (GCP)

The Governance Control Plane governs system authority rules.

Its purpose is to ensure that actions are allowed, approved, authorized, and resource-governed before they occur.

It owns:

- policy evaluation
- approval routing
- override governance
- permissions and authorization
- quota management

It does not own:

- low-level run execution
- artifact hashing
- release storage internals
- notification delivery plumbing

Roles within GCP

- GCP-01 Policy Engine
- GCP-02 Approval Router
- GCP-03 Override Manager

- GCP-04 Permissions / AuthZ Manager
- GCP-05 Quota Manager

7.3 Evidence Control Plane (ECP)

The Evidence Control Plane governs proof-bearing and integrity-bearing system records.

Its purpose is to ensure that the system preserves trustworthy operational evidence and artifact truth.

It owns:

- audit logging
- artifact pinning
- release records
- artifact registry
- lineage tracking
- delivery evidence for notifications

It does not own:

- policy decision logic
- operator authorization logic
- stage dispatch logic
- queue scheduling logic

Roles within ECP

- ECP-01 Audit Logger

- ECP-02 Pin Manager
- ECP-03 Release Manager
- ECP-04 Artifact Registry / Lineage Governor
- ECP-05 Notification Router

---

8. Role Specifications

8.1 ICP-01 Run Controller

Purpose

Own the lifecycle of a run.

Responsibilities

- create run records
- initialize runs
- enforce stage order
- advance stages
- record stage completion
- move runs into gated, failed, cancelled, or completed states
- coordinate with policy and evidence services before terminal actions

Key inputs

- pipeline definition
- run request

- stage result
- policy result
- state machine result

Key outputs

- run manifest updates
- stage status updates
- run status changes
- execution events

AI assist surfaces

- recommend retry path
- classify failure type
- suggest likely blocked dependency
- draft operator summary

AI restrictions

- may not advance a stage without deterministic validation
- may not force illegal run transitions
- may not bypass required approvals or policy checks

8.2 ICP-02 Run State Store

Purpose

Persist canonical run state.

Responsibilities

- create run record
- read run record
- update run record
- list runs
- maintain durable run state

Key inputs

- run controller actions
- stage result updates
- completion records

Key outputs

- canonical run manifest
- historical run records
- queryable run state

AI assist surfaces

- summarize stale or inconsistent state
- propose cleanup candidates
- explain state drift patterns

AI restrictions

- may not write canonical state directly
- may not synthesize missing truth without verification

8.3 ICP-03 State Machine Governor

Purpose

Enforce legal state transitions.

Responsibilities

- define legal run states
- define legal stage states
- validate transitions
- reject invalid transitions
- define retry, pause, resume, cancel, and recovery rules

Key inputs

- current run state
- proposed transition
- transition context

Key outputs

- pass/fail transition validation
- rejection reasons
- transition event record

AI assist surfaces

- explain why a transition was rejected
- recommend valid next actions
- cluster common failure patterns

AI restrictions

- may not redefine legal transitions
- may not override deterministic state rules

8.4 ICP-04 Queue / Scheduler Manager

Purpose

Govern queued and scheduled execution.

Responsibilities

- queue runs
- schedule runs
- prioritize work
- dispatch to available execution capacity
- throttle when needed
- manage retries at dispatch level

Key inputs

- run requests

- schedule requests
- quota state
- environment availability

Key outputs

- queue entries
- dispatch assignments
- schedule entries
- retry records

AI assist surfaces

- recommend dispatch priority adjustments
- forecast queue pressure
- suggest retry timing

AI restrictions

- may not violate quotas
- may not schedule protected actions outside allowed rules

8.5 ICP-05 Adapter / Environment Manager

Purpose

Determine execution target and environment suitability.

Responsibilities

- register adapters
- maintain environment profiles
- discover capabilities
- validate readiness
- route execution to correct target

Key inputs

- adapter registry
- environment profile
- run requirements
- quota and policy constraints

Key outputs

- environment suitability result
- adapter selection decision
- capability snapshot

AI assist surfaces

- recommend best-fit execution environment
- explain environment mismatch
- predict environment risk

AI restrictions

- may not override hard environment constraints

- may not force unsupported adapter routing

---

9. Governance Role Specifications

9.1 GCP-01 Policy Engine

Purpose

Evaluate whether actions are allowed under active policy.

Responsibilities

- load policy definitions
- evaluate policies
- resolve precedence
- return pass/fail results
- attach evidence requirements
- preserve policy snapshots where needed

Key inputs

- action request
- run state
- risk class
- artifact state
- release state

Key outputs

- policy result
- denial reasons
- required evidence set
- applicable policy snapshot

AI assist surfaces

- summarize policy conflicts
- explain denial reasons
- draft readable policy explanations
- recommend which policies likely apply

AI restrictions

- may not become the source of final policy truth
- may not approve policy violations on its own

9.2 GCP-02 Approval Router

Purpose

Route actions that require human or governed approval.

Responsibilities

- create approval requests
- route to approvers
- track pending approvals

- enforce expiry and escalation
- record decisions
- connect approval results to governed actions

Key inputs

- approval-triggering event
- approver map
- protected action request

Key outputs

- approval request
- approval decision
- approval status update
- escalation record

AI assist surfaces

- recommend likely approver path
- summarize context for approvers
- draft approval packet summaries

AI restrictions

- may not approve protected actions by itself
- may not alter approver authority

9.3 GCP-03 Override Manager

Purpose

Govern controlled exceptions to policy.

Responsibilities

- create override requests
- validate override eligibility
- require evidence and justification
- enforce expiry
- revoke invalid overrides
- ensure audit linkage

Key inputs

- override request
- justification
- evidence
- approval requirement

Key outputs

- override record
- override decision
- active override state
- override expiry event

AI assist surfaces

- score override risk
- summarize evidence quality
- draft override review context

AI restrictions

- may not grant override authority by itself
- may not suppress required evidence

9.4 GCP-04 Permissions / AuthZ Manager

Purpose

Determine whether an actor may perform an action.

Responsibilities

- resolve actor role
- evaluate permissions
- enforce separation of duties
- protect privileged operations
- deny unauthorized actions

Key inputs

- actor identity
- actor role

- requested action
- governed object

Key outputs

- authorization result
- denial reason
- role snapshot

AI assist surfaces

- flag suspicious access patterns
- summarize unusual permission activity
- explain denied actions in readable form

AI restrictions

- may not grant permissions directly
- may not rewrite role mappings
- may not bypass protected-action authz rules

9.5 GCP-05 Quota Manager

Purpose

Govern resource and usage limits.

Responsibilities

- track usage
- enforce quotas
- meter consumption
- reject or throttle overages
- maintain remaining quota state

Key inputs

- run activity
- environment usage
- project or workspace quota
- policy constraints

Key outputs

- usage ledger
- enforcement result
- quota state
- throttle record

AI assist surfaces

- forecast quota pressure
- suggest quota-aware scheduling
- explain resource spikes

AI restrictions

- may not increase quotas directly
- may not ignore hard limit rules

---

10. Evidence Role Specifications

10.1 ECP-01 Audit Logger

Purpose

Preserve append-only evidence of control-plane actions.

Responsibilities

- append audit entries
- chain entries with hashes
- preserve actor/action/target/time
- attach evidence references
- support later verification

Key inputs

- governed action events
- operator actions
- AI-assisted decision records

Key outputs

- audit entry

- hash chain
- audit verification record

AI assist surfaces

- summarize audit trails
- detect suspicious action clusters
- explain action sequences

AI restrictions

- may not alter audit truth
- may not remove or rewrite entries

10.2 ECP-02 Pin Manager

Purpose

Preserve artifact integrity through pinned hashes.

Responsibilities

- pin artifact state
- unpin artifact where allowed
- verify artifact against pin
- detect drift
- preserve pin evidence

Key inputs

- artifact reference
- hash material
- unpin request
- verification request

Key outputs

- pin record
- verification result
- drift record

AI assist surfaces

- explain likely drift causes
- summarize affected artifacts
- classify drift severity

AI restrictions

- may not alter pin truth
- may not unpin protected artifacts by itself

10.3 ECP-03 Release Manager

Purpose

Govern release lifecycle records.

Responsibilities

- create release
- sign release
- publish release
- revoke release
- preserve release evidence

Key inputs

- release request
- release artifact set
- signature input
- approval and policy state

Key outputs

- release record
- signature record
- publish/revoke event

AI assist surfaces

- summarize release readiness
- identify likely missing evidence
- draft release review packet

AI restrictions

- may not publish or revoke protected releases by itself

- may not forge release evidence

10.4 ECP-04 Artifact Registry / Lineage Governor

Purpose

Track artifact identity and provenance.

Responsibilities

- register artifacts
- track producer stage
- record consumer relationships
- maintain lineage graph
- preserve artifact references

Key inputs

- artifact creation events
- stage outputs
- release references
- pin references

Key outputs

- artifact registry entry
- lineage graph edge

- provenance record

AI assist surfaces

- explain lineage breaks
- summarize artifact dependency impact
- identify suspicious provenance patterns

AI restrictions

- may not rewrite lineage truth
- may not synthesize lineage without evidence

10.5 ECP-05 Notification Router

Purpose

Route and preserve delivery evidence for control-plane notifications.

Responsibilities

- emit events
- classify severity
- select delivery channel
- retry failed notifications
- preserve delivery status

Key inputs

- control-plane events
- approval events
- policy denial events
- release events

Key outputs

- notification event
- delivery record
- retry record
- severity class

AI assist surfaces

- prioritize alerts
- summarize noisy event groups
- recommend escalation handling

AI restrictions

- may not suppress mandatory notifications without governed rule support

---

11. AI Governance Model

AI is introduced as a governed support layer, not as a replacement for the control planes.

11.1 AI purpose

AI exists to improve:

- analysis
- recommendation quality
- operator speed
- summarization
- anomaly detection
- structured assistance

11.2 AI authority model

Default AI authority level is:

- advisory only

Optional allowed modes:

- propose and validate
- auto-act only for explicitly whitelisted low-risk cases

Protected actions must remain governed by deterministic rules and, where required, human approval.

11.3 AI non-authority zones

AI may not serve as:

- final policy engine
- final approval source
- final permission source
- final audit truth source
- final state transition authority

---

12. Proposed Model Strategy

12.1 Primary assistant model

Qwen
Use as the default model for:

- code-aware assistance
- repo and artifact analysis
- structured recommendation generation
- general control-plane support

12.2 Secondary reasoning model

DeepSeek
Use for:

- heavier multi-step reasoning
- complex exception analysis
- override review assistance
- conflict explanation
- postmortem synthesis

12.3 Optional implementation sidecar

Devstral
Use for:

- engineering support
- code issue workflows

- implementation assistance
- patch-oriented support

---

13. AI Operating Rules

13.1 What AI may do

AI may:

- classify failures
- summarize policy conflicts
- recommend retry paths
- explain denial reasons
- identify suspicious patterns
- summarize audit history
- draft approval packets
- summarize release readiness
- recommend environment selection
- produce structured proposed actions

13.2 What AI may not do

AI may not:

- approve protected actions alone
- bypass policy checks
- bypass authorization checks

- bypass state machine checks
- rewrite audit history
- publish or revoke controlled releases alone
- alter quotas directly
- directly change role mappings
- directly unpin protected artifacts

13.3 Validation rule

Any AI-assisted recommendation that may lead to action must still pass:

- schema validation
- policy validation
- authorization checks
- state transition validation
- approval requirements
- evidence checks
- audit logging requirements

---

14. Artifact Requirements

The update requires both control-plane artifacts and AI-governance artifacts.

14.1 Control-plane artifact groups

- ICP artifacts
- GCP artifacts

- ECP artifacts

14.2 AI governance artifact group

Suggested group:

- AIG-00 Purpose + Boundaries
- AIG-01 AI Decision Scope
- AIG-02 AI Authority Limits
- AIG-03 AI Input Contract
- AIG-04 AI Output Contract
- AIG-05 AI Action Gating
- AIG-06 AI Audit + Replay Rules
- AIG-07 Model Routing + Fallback Policy

---

15. Contract and Schema Updates

15.1 New contracts

This update should add:

- AI request envelope
- AI response envelope
- recommendation schema
- confidence schema
- evidence reference schema
- approval recommendation schema

- AI trace record
- model routing decision record

15.2 Existing contracts requiring updates

This update should modify:

- run manifest
- policy evaluation result
- approval request
- override request
- audit entry
- release record
- artifact registry entry
- notification event

---

16. Enforcement Requirements

16.1 Deterministic enforcement stays mandatory

No AI output may directly become a governed action unless the system validates it through the normal deterministic control path.

16.2 Protected action handling

Protected actions must require governed checks and, where applicable:

- approval
- authorization

- evidence completeness
- policy clearance
- release integrity checks
- audit record creation

16.3 AI failure handling

When AI fails, the system must fail safely.

Required behavior:

- AI unavailable → continue deterministic control-plane operation without AI assistance
- AI malformed output → reject output and log issue
- AI low confidence → escalate or keep advisory-only
- AI conflicts with deterministic rules → deterministic rules win
- AI recommendation rejected by operator → preserve rejection in audit trail

---

17. UI and Operator Workflow Implications

This update should support operator interfaces that make AI assistance visible but clearly bounded.

The UI should distinguish:

- deterministic system state
- AI recommendation
- human decision
- policy result

- approval result
- audit evidence

Operators should always be able to tell:

- what the AI suggested
- what the system validated
- what was actually executed
- who approved or rejected it

---

18. Build Phases

Phase 1 — control plane stabilization

Deliver:

- finalized ICP, GCP, and ECP role boundaries
- missing role definitions
- normalized state and action contracts
- stronger deterministic validation paths

Primary targets:

- ICP-01
- ICP-02
- ICP-03
- GCP-01

- ECP-01
- ECP-02
- ECP-03

Phase 2 — governance completion

Deliver:

- approval routing
- override governance
- authz model
- quota model

Primary targets:

- GCP-02
- GCP-03
- GCP-04
- GCP-05

Phase 3 — execution and evidence completion

Deliver:

- queue and scheduler
- adapter management

- artifact registry and lineage
- notification routing

Primary targets:

- ICP-04
- ICP-05
- ECP-04
- ECP-05

Phase 4 — AI governance foundation

Deliver:

- AIG artifacts
- AI contracts
- AI audit model
- action gating model
- fallback behavior

Phase 5 — bounded AI integration

Deliver:

- Qwen integration as primary assistant
- DeepSeek integration for heavier reasoning
- advisory-first workflows

- protected action validation path

Phase 6 — operator workflow refinement

Deliver:

- approval assist UX
- failure triage assist UX
- audit summary UX
- release readiness summaries
- override review support

---

19. Success Criteria

This update is successful when:

- the three control planes are explicitly defined
- each role has a clear purpose, contract, and boundary
- deterministic authority remains intact
- AI assistance is bounded and auditable
- protected actions remain governed
- AI failure does not compromise control-plane safety
- operators can see and trust the difference between suggestion and authority

---

20. Final Design Statement

Axion’s updated control plane architecture is a **deterministic multi-plane governance system with bounded AI assistance**.

The control planes remain the authoritative system roles.

AI is integrated as a governed support capability that improves analysis, routing, summarization, and operational efficiency without replacing authority, policy, validation, approval, or audit truth.

Axion Control Plane Role Spec

Axion Control Plane Role Spec

1. Execution Control Plane

This control plane governs **run creation, stage progression, execution state, and pipeline movement**.

ECP-01 Run Controller

Purpose
Own the lifecycle of a run from creation through release.

Responsibilities

- create run IDs
- initialize run directory structure
- seed stage list from pipeline definition
- enforce allowed run status transitions
- advance stages in order
- record stage outcomes
- complete or gate the run

Current repo match

- `Axion/src/core/controlPlane/api.ts`
- class: `RunController`

Current maturity

- present

- core role
- partially operational

What it already does

- `createRun`
- `advanceStage`
- `recordStageResult`
- `completeRun`
- `getRunStatus`

What it still needs

- pause/resume
- retry/rerun policy
- partial reruns
- concurrency controls
- cancellation handling

ECP-02 Run State Store

Purpose
Persist canonical run state.

Responsibilities

- store run manifest
- retrieve run manifest

- update manifest
- enumerate historical runs

Current repo match

- `Axion/src/core/controlPlane/store.ts`
- class: `JSONRunStore`

Current maturity

- present
- operational for local JSON storage

What it already does

- `createRun`
- `getRun`
- `updateRun`
- `listRuns`

What it still needs

- locking
- corruption recovery
- transaction safety
- multi-user or multi-process coordination
- pluggable backends

ECP-03 State Machine Governor

Purpose
Own legal state transitions for runs and stages.

Responsibilities

- define valid run transitions
- define valid stage progression rules
- block invalid transitions
- ensure ordering constraints

Current repo match

- embedded inside `api.ts`
- `VALID_RUN_TRANSITIONS`
- prior-stage checks in `advanceStage`

Current maturity

- present but implicit
- not first-class yet

What it still needs

- dedicated module
- retry state model
- paused/cancelled/degraded modes
- stage restart policy
- explicit recovery semantics

ECP-04 Queue / Scheduler Manager

Purpose
Govern queued execution and dispatch.

Responsibilities

- queued runs
- scheduled runs
- priority ordering
- dispatch policy
- execution slots

Current repo match

- not first-class

Current maturity

- missing

What it still needs

- durable queue
- scheduled execution
- dispatcher
- throttling
- worker assignment

ECP-05 Adapter / Environment Manager

Purpose
Control where and how runs execute.

Responsibilities

- local vs CI vs remote execution selection
- capability discovery
- environment validation
- adapter policy

Current repo match

- not first-class

Current maturity

- missing

What it still needs

- adapter registry
- environment profile model
- capability negotiation
- execution routing

---

2. Governance Control Plane

This control plane governs **policy, risk, overrides, permissions, and operator decisions**.

GCP-01 Policy Engine

Purpose
Evaluate whether a run or release is allowed under active policy.

Responsibilities

- load policy registry
- derive risk-class policy
- derive override policy
- evaluate single policy
- evaluate policy sets

Current repo match

- `Axion/src/core/controlPlane/policies.ts`

Current maturity

- present
- partial logic

What it already does

- `loadPolicies`
- `evaluatePolicy`
- `evaluateAllPolicies`

What it still needs

- richer condition language
- stage-aware enforcement
- compiled predicates
- policy snapshots persisted per run

- stronger evidence checks

GCP-02 Approval Router

Purpose
Send gated decisions to a human or authority role for approval.

Responsibilities

- route failed gates
- route release approvals
- capture signer decisions
- track pending approvals
- enforce separation of duties

Current repo match

- not first-class

Current maturity

- missing

What it still needs

- approval queue
- approver identity model
- approval records
- approve/reject/expire flow
- linkage to release and override actions

GCP-03 Override Manager

Purpose
Govern policy exceptions.

Responsibilities

- create override requests
- validate eligibility
- require evidence
- enforce expiry
- record justification

Current repo match

- concept partially present in `policies.ts`
- not operational as its own role

Current maturity

- missing as a real role

What it still needs

- override object schema
- lifecycle
- evidence requirements
- expiry enforcement
- audit integration

GCP-04 Permissions / AuthZ Manager

Purpose
Determine who is allowed to perform which actions.

Responsibilities

- operator identity
- action authorization
- role-based access
- protected operations

Current repo match

- not present as a system role

Current maturity

- missing

What it still needs

- actor model
- role model
- permission map
- protected command enforcement

GCP-05 Quota Manager

Purpose
Govern resource and execution limits.

Responsibilities

- runs/day
- token budgets
- storage budgets
- export limits
- policy-aware quota checks

Current repo match

- `quota_set` exists in run state
- no true manager role visible

Current maturity

- implied only

What it still needs

- quota definitions
- enforcement checks
- metering
- refusal/backoff behavior

---

3. Evidence Control Plane

This control plane governs **auditability, artifact integrity, releases, and proof-bearing state**.

EVCP-01 Audit Logger

Purpose
Create a tamper-linked record of control plane actions.

Responsibilities

- append action records
- include run ID and details
- chain entries with previous hash
- preserve operator evidence trail

Current repo match

- `Axion/src/core/controlPlane/audit.ts`
- class: `AuditLogger`

Current maturity

- present
- operational

What it already does

- append timestamped action records
- include `prev_hash`
- compute entry `hash`

What it still needs

- actor identity
- signed audit segments
- verification tooling

- rotation/retention policy

EVCP-02 Pin Manager

Purpose
Protect artifacts by anchoring expected content hashes.

Responsibilities

- pin artifact
- unpin artifact
- list pins
- verify one pin
- verify all pins

Current repo match

- `Axion/src/core/controlPlane/pins.ts`

Current maturity

- present
- usable

What it already does

- compute file hash
- persist pinset
- detect drift

What it still needs

- pin scopes
- immutable pin classes
- approval-required unpin
- artifact lineage integration

EVCP-03 Release Manager

Purpose
Control the lifecycle of release objects.

Responsibilities

- create release
- sign release
- publish release
- revoke release
- list/get release records

Current repo match

- `Axion/src/core/controlPlane/releases.ts`

Current maturity

- present
- shell is usable

What it already does

- release state transitions

- artifact list inclusion
- signature recording
- publish/revoke flow

What it still needs

- stronger signature model
- signer authorization
- multi-signer thresholds
- release policy checks before publish

EVCP-04 Artifact Registry / Lineage Governor

Purpose
Govern artifact identity, provenance, and relationships.

Responsibilities

- track artifact index
- map producer stage
- preserve lineage
- resolve artifact references

Current repo match

- partially implied through `artifact_index.json`
- seeded in `runControlPlane.ts`
- not a dedicated control-plane role yet

Current maturity

- partial / implicit

What it still needs

- central registry service
- lineage graph
- artifact dependency model
- mutation policy

EVCP-05 Notification Router

Purpose
Emit operational events to the operator layer.

Responsibilities

- gate failure notifications
- approval needed notices
- publish/revoke events
- policy denial alerts

Current repo match

- not present

Current maturity

- missing

What it still needs

- event bus
- subscription model
- notification sinks
- severity classes

Current repo role mapping

Already present in source

- **Run Controller**
- **Run State Store**
- **State Machine Governor** as embedded logic
- **Policy Engine**
- **Audit Logger**
- **Pin Manager**
- **Release Manager**
- **Artifact registry seeding** as partial behavior

Not yet first-class

- **Queue / Scheduler Manager**
- **Adapter / Environment Manager**
- **Approval Router**

- **Override Manager**
- **Permissions / AuthZ Manager**
- **Quota Manager**
- **Notification Router**
- **Full Artifact Lineage Governor**

Recommended official 3-plane model for Axion

ICP — Execution Control Plane

Owns:

- Run Controller
- Run State Store
- State Machine Governor
- Queue / Scheduler Manager
- Adapter / Environment Manager

GCP — Governance Control Plane

Owns:

- Policy Engine
- Approval Router
- Override Manager

- Permissions / AuthZ Manager
- Quota Manager

ECP — Evidence Control Plane

Owns:

- Audit Logger
- Pin Manager
- Release Manager
- Artifact Registry / Lineage Governor
- Notification Router

Current completion view

Strongest roles now

- Run Controller
- Run State Store
- Audit Logger
- Pin Manager

Mid-state roles now

- Policy Engine
- Release Manager
- State transition enforcement

Weak or missing roles now

- approvals
- permissions
- quotas
- notifications
- scheduling
- environment control
- full lineage governance

artifact-ready outline pack

Axion Control Plane Artifact-Ready Outline Pack

This is the clean pack version you can use as the official control plane artifact structure.

ICP — Execution Control Plane

ICP-00 Purpose + Boundaries

Defines the execution authority for runs, stages, transitions, dispatch, and environment routing.

Covers

- run lifecycle control
- run persistence
- state transitions
- queueing and scheduling
- execution environment routing

Does not cover

- policy approvals
- override governance
- audit proof governance
- release authority
- notification policy

ICP-01 Run Controller

Defines the role that owns run creation, advancement, completion, and execution flow.

Should define

- run creation contract
- run initialization requirements
- stage advancement rules
- completion rules
- gated/failure terminal states
- partial rerun model
- cancel/pause/resume model

Core responsibilities

- create run
- start run
- advance run
- record stage result
- mark run gated/failed/completed
- enforce pipeline order

Key outputs

- run manifest
- stage result updates
- run status updates
- run completion record

Primary dependencies

- pipeline definition
- run state store
- state machine governor
- policy engine
- artifact registry

ICP-02 Run State Store

Defines the canonical persistence layer for run state.

Should define

- run record schema
- write/update rules
- list/query keys
- versioning rules
- locking/consistency expectations
- backend abstraction rules

Core responsibilities

- create run record
- read run record
- update run record
- list runs

- persist stage state
- persist run metadata

Key outputs

- canonical run manifest
- state snapshots
- run query surfaces

Primary dependencies

- filesystem or database backend
- run controller
- audit logger

ICP-03 State Machine Governor

Defines legal transitions for runs and stages.

Should define

- run states
- stage states
- allowed transitions
- forbidden transitions
- retry transition policy
- pause/resume semantics
- recovery semantics

Core responsibilities

- validate transition legality
- block invalid transitions
- enforce ordering rules
- encode terminal state behavior
- enforce rerun/retry conditions

Key outputs

- transition validation result
- transition rejection reason
- state transition event

Primary dependencies

- run controller
- run state store
- policy engine

ICP-04 Queue / Scheduler Manager

Defines queued and scheduled execution behavior.

Should define

- queued run schema
- priority model
- dispatch rules

- scheduling rules
- worker slot model
- throttling rules
- starvation prevention rules

Core responsibilities

- queue runs
- schedule future runs
- dispatch runs
- throttle execution
- manage worker availability
- retry failed dispatches

Key outputs

- queue records
- dispatch assignments
- schedule records
- execution reservations

Primary dependencies

- run controller
- state store
- quota manager
- adapter manager

ICP-05 Adapter / Environment Manager

Defines how execution targets are selected and validated.

Should define

- adapter registry schema
- capability discovery rules
- environment profile schema
- environment validation gates
- execution selection policy

Core responsibilities

- register adapters
- detect environment capabilities
- select execution target
- validate environment readiness
- route run to correct adapter

Key outputs

- adapter registry
- environment capability snapshot
- execution routing decision

Primary dependencies

- queue manager

- run controller
- quota manager
- policy engine

GCP — Governance Control Plane

GCP-00 Purpose + Boundaries

Defines the governance authority for policy, approvals, overrides, permissions, and quotas.

Covers

- policy enforcement
- approval routing
- override governance
- operator permissions
- resource and quota governance

Does not cover

- low-level execution dispatch
- artifact hashing or pin storage
- release object persistence
- notification delivery plumbing

GCP-01 Policy Engine

Defines policy evaluation for runs, stages, artifacts, and releases.

Should define

- policy schema
- policy types
- predicate model
- evidence requirements
- evaluation result format
- policy precedence
- policy snapshot rules

Core responsibilities

- load policies
- evaluate policies
- return pass/fail results
- attach denial reasons
- require evidence where needed
- resolve policy conflicts

Key outputs

- policy evaluation result
- denial reason set
- applicable policy snapshot

Primary dependencies

- run controller
- state store
- approval router
- override manager
- release manager

GCP-02 Approval Router

Defines routing and handling of approval-required decisions.

Should define

- approval request schema
- approver identity requirements
- approval states
- escalation rules
- expiry rules
- separation-of-duties rules

Core responsibilities

- open approval requests
- route to approvers
- record decisions
- enforce expiry/escalation
- link decisions to governed actions

Key outputs

- approval request
- decision record
- approval status update

Primary dependencies

- policy engine
- override manager
- release manager
- notification router
- authz manager

GCP-03 Override Manager

Defines controlled exception handling to standard policy.

Should define

- override request schema
- eligibility rules
- evidence requirements
- approval requirements
- expiry and revocation rules
- audit requirements

Core responsibilities

- create override requests
- validate override eligibility
- require justification
- attach evidence
- enforce expiry
- revoke expired or invalid overrides

Key outputs

- override request
- override decision
- active override record

Primary dependencies

- policy engine
- approval router
- audit logger
- authz manager

GCP-04 Permissions / AuthZ Manager

Defines operator identity and action authorization.

Should define

- actor model
- role model

- permission map
- protected action list
- authz decision format
- delegation rules

Core responsibilities

- resolve actor role
- validate action permission
- protect privileged actions
- enforce separation of duties
- deny unauthorized requests

Key outputs

- authorization decision
- actor permission snapshot
- denial reason

Primary dependencies

- approval router
- override manager
- release manager
- audit logger

GCP-05 Quota Manager

Defines resource limits and consumption governance.

Should define

- quota schema
- quota scope
- metering rules
- enforcement actions
- burst rules
- cooldown/reset rules

Core responsibilities

- track usage
- enforce limits
- reject or throttle overages
- meter compute/storage/run counts
- expose remaining quota state

Key outputs

- quota state
- usage ledger
- enforcement decision

Primary dependencies

- queue manager
- adapter manager

- run controller
- policy engine

ECP — Evidence Control Plane

ECP-00 Purpose + Boundaries

Defines the evidence authority for audit, integrity, artifact lineage, release records, and event evidence.

Covers

- audit logging
- artifact pinning
- release lifecycle evidence
- artifact registry and lineage
- event/notification evidence linkage

Does not cover

- policy reasoning
- operator permissions
- stage execution dispatch
- scheduling logic

ECP-01 Audit Logger

Defines append-only action logging for governed control plane activity.

Should define

- audit entry schema
- hash chaining rules
- actor/action/target model
- evidence attachment rules
- retention/rotation rules
- verification rules

Core responsibilities

- append audit entries
- chain entries with hashes
- capture actor/action/time
- preserve evidence linkage
- support verification

Key outputs

- audit log
- audit entry hash chain
- audit verification result

Primary dependencies

- all control plane roles
- authz manager

- override manager
- release manager

ECP-02 Pin Manager

Defines artifact integrity anchoring via pinned hashes.

Should define

- pin record schema
- pin scope rules
- verification rules
- pin classes
- unpin authorization rules
- drift response rules

Core responsibilities

- pin artifact hash
- unpin artifact
- verify artifact against pin
- list pins
- detect drift/tamper

Key outputs

- pinset
- pin verification result

- drift record

Primary dependencies

- artifact registry
- release manager
- audit logger
- authz manager

ECP-03 Release Manager

Defines governed release lifecycle behavior.

Should define

- release object schema
- release states
- signing requirements
- publish conditions
- revoke rules
- release evidence requirements

Core responsibilities

- create release
- sign release
- publish release
- revoke release

- list/get releases

Key outputs

- release record
- signature record
- publish/revoke event

Primary dependencies

- policy engine
- approval router
- pin manager
- artifact registry
- audit logger

ECP-04 Artifact Registry / Lineage Governor

Defines artifact identity, provenance, and lineage relationships.

Should define

- artifact registry schema
- artifact identity rules
- lineage edge model
- producer/consumer references
- mutation/update policy
- resolution/query keys

Core responsibilities

- register artifacts
- track producer stage
- track artifact lineage
- resolve references
- preserve provenance

Key outputs

- artifact registry
- lineage graph
- provenance record

Primary dependencies

- run controller
- release manager
- pin manager
- audit logger

ECP-05 Notification Router

Defines event emission and delivery routing for governed actions.

Should define

- event schema
- notification classes

- severity model
- delivery channels
- retry policy
- delivery audit linkage

Core responsibilities

- emit control plane events
- route notifications
- retry failed deliveries
- map event severity
- preserve delivery evidence

Key outputs

- notification event
- delivery record
- delivery status

Primary dependencies

- approval router
- release manager
- policy engine
- audit logger

Recommended build order

Phase 1 — stabilize current real roles

- ICP-01 Run Controller
- ICP-02 Run State Store
- ICP-03 State Machine Governor
- ECP-01 Audit Logger
- ECP-02 Pin Manager
- ECP-03 Release Manager
- GCP-01 Policy Engine

Phase 2 — governance completion

- GCP-02 Approval Router
- GCP-03 Override Manager
- GCP-04 Permissions / AuthZ Manager
- GCP-05 Quota Manager

Phase 3 — operations completion

- ICP-04 Queue / Scheduler Manager
- ICP-05 Adapter / Environment Manager
- ECP-04 Artifact Registry / Lineage Governor
- ECP-05 Notification Router

Current repo mapping

Directly backed by current source

- ICP-01
- ICP-02
- ICP-03 partial
- GCP-01 partial
- ECP-01
- ECP-02
- ECP-03
- ECP-04 partial

Not yet first-class in source

- ICP-04
- ICP-05
- GCP-02
- GCP-03
- GCP-04
- GCP-05
- ECP-05

Suggested artifact filenames

- `ICP-00-purpose-boundaries.md`
- `ICP-01-run-controller.md`
- `ICP-02-run-state-store.md`
- `ICP-03-state-machine-governor.md`
- `ICP-04-queue-scheduler-manager.md`
- `ICP-05-adapter-environment-manager.md`
- `GCP-00-purpose-boundaries.md`
- `GCP-01-policy-engine.md`
- `GCP-02-approval-router.md`
- `GCP-03-override-manager.md`
- `GCP-04-permissions-authz-manager.md`
- `GCP-05-quota-manager.md`
- `ECP-00-purpose-boundaries.md`
- `ECP-01-audit-logger.md`
- `ECP-02-pin-manager.md`
- `ECP-03-release-manager.md`
- `ECP-04-artifact-registry-lineage-governor.md`
- `ECP-05-notification-router.md`

ICP Artifact Pack

ICP-00 — Purpose + Boundaries

1. Purpose

The Execution Control Plane (ICP) is the control plane responsible for **run movement, execution coordination, and legal pipeline progression** inside Axion.

Its purpose is to ensure that runs are created, advanced, scheduled, routed, retried, paused, resumed, cancelled, and completed through a **deterministic execution authority layer**.

The ICP exists to govern *how work moves through the system*.

It is the plane that turns a defined pipeline into actual controlled execution behavior.

---

2. Objective

The objective of the ICP is to provide a single execution authority that:

- owns run lifecycle movement
- enforces valid stage ordering
- validates execution transitions
- coordinates queueing and scheduling
- routes work into valid execution environments
- preserves stable execution state
- prevents illegal or out-of-order movement
- supports bounded AI assistance without surrendering execution authority

---

3. Problem It Solves

Without a formal execution control plane, execution behavior becomes fragmented across:

- stage logic
- orchestration scripts
- run records
- ad hoc transition checks
- environment-specific execution paths

That fragmentation creates risk such as:

- invalid run transitions
- skipped stages
- duplicate or conflicting execution
- inconsistent retry behavior
- weak pause/resume semantics
- unclear queue ownership
- ungoverned environment routing

The ICP solves this by making execution authority explicit.

---

4. What the ICP Governs

The ICP governs:

- run creation
- run initialization
- stage progression
- execution ordering
- run and stage state transitions
- queued execution
- scheduled execution
- retry dispatch behavior
- execution pause/resume/cancel behavior
- execution target selection
- environment capability matching

---

5. What the ICP Does Not Govern

The ICP does **not** govern:

- policy truth
- approval authority
- override authority
- permission authority
- quota policy definition
- audit truth
- artifact integrity truth
- release approval or release policy

- artifact lineage as evidence truth

Those belong to other planes.

The ICP may depend on those planes, but it does not replace them.

---

6. Core Principle

The ICP is the **execution authority**, not the reasoning authority.

That means:

- it decides whether execution movement is legal
- it does not invent policy
- it does not invent permission
- it does not invent approval
- it does not invent integrity truth

It enforces movement using explicit rules and validated inputs.

---

7. Governing Model

The ICP follows this model:

pipeline definition defines expected movement
run state defines current movement
state machine defines legal movement
queue/scheduler defines pending movement
adapter manager defines where movement may execute

The ICP is the plane that coordinates all of that into one governed execution path.

---

8. Primary Responsibilities

The ICP is responsible for:

8.1 Run lifecycle control

- create run
- initialize run
- start run
- advance run
- complete run
- fail run
- gate run
- cancel run
- pause run
- resume run

8.2 Stage progression control

- enforce ordered stage progression
- validate stage advancement preconditions
- record stage outcomes
- block illegal progression
- support rerun and retry semantics where allowed

8.3 State transition enforcement

- validate run transition legality
- validate stage transition legality
- reject illegal transitions
- preserve transition history

8.4 Scheduling and queue coordination

- enqueue runs
- schedule future runs
- prioritize execution
- dispatch runs
- retry dispatch when needed
- respect quotas and environment availability

8.5 Execution environment routing

- choose valid adapter/environment target
- validate environment capability
- reject unsupported execution routes
- preserve execution target decisions

---

9. ICP Role Set

The ICP is composed of five roles:

ICP-01 Run Controller

Owns run lifecycle operations.

ICP-02 Run State Store

Owns canonical persistence of run state.

ICP-03 State Machine Governor

Owns legality of run/stage transitions.

ICP-04 Queue / Scheduler Manager

Owns queued and scheduled execution flow.

ICP-05 Adapter / Environment Manager

Owns execution target and capability routing.

---

10. Inputs

The ICP consumes the following input classes:

10.1 Execution requests

- create run request
- start run request
- advance stage request
- pause/resume/cancel request
- retry/rerun request
- scheduling request

10.2 Pipeline inputs

- pipeline definition
- stage order
- stage metadata

- stage activation requirements

10.3 State inputs

- current run manifest
- current stage state
- prior transition history
- queue state

10.4 Governance-dependent inputs

- policy result
- approval result where required
- authorization result
- quota enforcement result

10.5 Environment inputs

- adapter registry
- capability snapshot
- environment readiness state
- dispatch availability

---

11. Outputs

The ICP produces:

11.1 Execution state outputs

- run manifest updates
- stage state updates
- transition records
- queue records
- schedule records
- dispatch assignments

11.2 Execution decisions

- transition allowed/denied
- dispatch allowed/denied
- environment selected/rejected
- retry allowed/denied

11.3 Execution events

- run created
- run started
- stage advanced
- run paused
- run resumed
- run cancelled
- run failed
- run completed
- dispatch queued
- dispatch retried

---

12. Dependencies

The ICP depends on other system layers but remains execution-authoritative within its own scope.

12.1 Governance dependencies

The ICP depends on the Governance Control Plane for:

- policy clearance
- protected action approval status
- permission validation
- quota enforcement decisions

12.2 Evidence dependencies

The ICP depends on the Evidence Control Plane for:

- audit recording
- evidence linkage for important execution actions
- artifact references where needed

12.3 Pipeline dependencies

The ICP depends on:

- pipeline definition store
- stage contracts
- run manifest contract

---

13. Authority Boundaries

The ICP has authority to:

- create runs
- move runs through legal transitions
- maintain execution state
- manage queueing and scheduling
- route execution to supported environments

The ICP does **not** have authority to:

- waive policy failures
- self-authorize protected actions outside valid authz
- approve overrides
- rewrite audit truth
- bypass integrity checks required by other planes

---

14. Protected Execution Actions

Certain execution actions must be treated as protected even inside the ICP.

Examples:

- rerunning protected stages
- resuming a gated run after approval-dependent hold
- dispatching execution into restricted environments
- performing administrative cancellation
- reordering queued execution in privileged contexts

These require deterministic checks and, where applicable:

- policy clearance
- authorization
- approval
- audit logging

---

15. AI Assistance Boundary

The ICP may be AI-assisted, but AI is not the ICP.

AI may help with:

- failure classification
- retry suggestions
- execution path recommendations
- environment suitability analysis
- bottleneck prediction
- readable operator summaries

AI may not:

- force a run transition
- bypass stage order
- dispatch unsupported environments
- resume protected execution illegally
- overrule deterministic execution rules

The ICP remains authoritative even when AI participates.

---

16. Operating Modes

The ICP should support these execution modes:

16.1 Deterministic-only mode

Execution occurs with no AI assistance.

16.2 AI-assisted advisory mode

AI produces recommendations, but deterministic systems and operators decide.

16.3 AI-assisted validate-first mode

AI proposes structured execution actions, but the ICP validates them before any state change.

16.4 Restricted low-risk automation mode

Only explicitly whitelisted low-risk AI-assisted execution actions may auto-apply, and only after deterministic validation.

---

17. Failure Handling Expectations

The ICP must fail safely.

17.1 Invalid transition

Reject transition and preserve reason.

17.2 Missing required governance signal

Block execution movement until required signal exists.

17.3 Environment mismatch

Reject routing and surface reason.

17.4 Queue conflict or dispatch failure

Retry or requeue according to deterministic rules.

17.5 AI failure

Ignore AI assistance and continue in deterministic control mode.

---

18. Required Guarantees

The ICP must guarantee:

- ordered stage progression unless explicit rerun policy allows otherwise
- legal state transitions only
- durable run state persistence
- stable queue and dispatch behavior
- explicit environment routing decisions
- auditable execution movement
- safe operation without AI

---

19. Success Criteria

The ICP is correctly established when:

- run lifecycle authority is centralized
- stage advancement is deterministic
- state transition rules are explicit
- queue and scheduling ownership are explicit
- environment routing ownership is explicit
- execution actions are auditable
- AI assistance is bounded and non-authoritative

---

20. Relationship to Other Planes

ICP vs GCP

- ICP governs **how work moves**
- GCP governs **whether certain actions are allowed**

ICP vs ECP

- ICP governs **execution movement**
- ECP governs **proof and integrity of evidence-bearing records**

ICP vs AIG

- ICP governs **execution authority**
- AIG governs **how AI may assist execution authority**

---

21. Summary Statement

The Execution Control Plane is the Axion execution authority layer.

It governs run lifecycle, stage progression, state legality, queueing, scheduling, and environment routing through deterministic services.

It may be assisted by AI, but it does not surrender execution authority to AI.

ICP-01 — Run Controller

ICP-01 — Run Controller

1. Purpose

The Run Controller is the role responsible for **owning run lifecycle operations** inside the Execution Control Plane.

Its purpose is to provide the single authoritative service that creates runs, initializes execution, advances stages, applies terminal outcomes, and coordinates legal run movement through the pipeline.

The Run Controller governs **run-level movement**.

It is the operational authority that turns execution requests into controlled state changes.

---

2. Objective

The objective of the Run Controller is to ensure that every run:

- is created through a controlled process
- starts from a valid initial state
- advances only when allowed
- records stage results consistently
- enters gated, failed, cancelled, or completed states legally
- preserves stable run history
- coordinates with governance and evidence services before protected actions
- remains deterministic even when AI assistance is present

---

3. Problem It Solves

Without a dedicated Run Controller, run lifecycle logic tends to become fragmented across:

- CLI commands
- stage executors
- ad hoc scripts
- direct state-store writes
- UI actions
- retry helpers

That fragmentation causes:

- inconsistent run creation
- skipped validation steps
- illegal advancement
- broken retry behavior
- mismatched terminal state handling
- weak audit linkage
- poor operator trust in run truth

The Run Controller solves this by centralizing lifecycle authority.

---

4. Scope

The Run Controller governs:

- run creation
- run initialization

- run start
- stage advancement requests
- stage result application
- run pause
- run resume
- run cancel
- run fail
- run complete
- run gated transitions
- rerun and retry entry points

The Run Controller does not own:

- run persistence implementation
- state legality rules definition
- policy evaluation logic
- approval routing logic
- audit storage implementation
- release logic
- artifact lineage logic

Those are handled by dependent roles.

---

5. Core Principle

The Run Controller is the **run lifecycle authority**, not the source of every decision rule.

It is responsible for coordinating a run action through the proper validators and dependent services before changing canonical run state.

This means:

- it issues lifecycle actions
- it does not invent legality rules
- it does not invent policy
- it does not invent permission
- it does not invent audit truth

It orchestrates run movement through deterministic control paths.

---

6. Primary Responsibilities

6.1 Create runs

- create a new run ID
- initialize run metadata
- attach pipeline definition reference
- create initial stage state layout
- seed initial run status
- prepare run manifest creation

6.2 Start runs

- validate that run start is legal

- move run from initial/queued state into active execution
- record run start event
- coordinate first executable stage entry

6.3 Advance stages

- receive stage advancement request
- validate current stage/run state
- consult state machine
- consult governance requirements where applicable
- move run to next legal stage when permitted

6.4 Record stage results

- apply stage result to canonical run state
- mark stage outcome
- update execution history
- determine whether next state is advance, gate, fail, or complete

6.5 Apply terminal run outcomes

- mark run completed
- mark run failed
- mark run gated
- mark run cancelled
- preserve terminal state metadata

6.6 Support controlled interruption and recovery

- pause run
- resume run
- retry allowed execution paths
- trigger rerun entry where policy allows

6.7 Coordinate with other planes

- require policy clearance where needed
- require authz where needed
- require approval where needed
- ensure audit recording for governed actions

---

7. Authority

The Run Controller has authority to:

- create run records through the proper state-store path
- request state transitions
- apply lifecycle operations after validation
- coordinate execution movement
- trigger deterministic run status updates
- reject invalid lifecycle requests

The Run Controller does not have authority to:

- bypass state machine validation
- override denied policy results
- bypass approval requirements
- bypass authorization requirements
- rewrite audit history
- directly define queue policy
- directly define environment capability truth

---

8. Inputs

The Run Controller consumes:

8.1 Lifecycle requests

- create run request
- start run request
- advance stage request
- pause request
- resume request
- cancel request
- retry request
- rerun request
- complete request

8.2 Run context

- current run manifest
- current stage states
- transition history
- pipeline reference
- execution metadata

8.3 Dependent validation inputs

- state machine validation result
- policy evaluation result
- approval result
- authorization result
- quota result where relevant
- environment eligibility result where relevant

8.4 Stage outcome inputs

- stage status
- stage evidence refs
- stage outputs
- gate result
- failure metadata

---

9. Outputs

The Run Controller produces:

9.1 Run state outputs

- run manifest updates
- run status updates
- stage status updates
- transition history entries
- terminal state records

9.2 Operational decisions

- lifecycle request accepted/rejected
- next stage selected/rejected
- retry path accepted/rejected
- run terminalized
- gated hold applied

9.3 Events

- run created
- run started
- stage advanced
- stage result recorded
- run paused
- run resumed
- run cancelled
- run failed

- run completed
- run gated

---

10. Dependencies

The Run Controller depends on the following roles:

10.1 ICP-02 Run State Store

For canonical persistence of run state.

10.2 ICP-03 State Machine Governor

For transition legality.

10.3 ICP-04 Queue / Scheduler Manager

For queued or scheduled movement where applicable.

10.4 ICP-05 Adapter / Environment Manager

For environment-aware dispatch or target validation where applicable.

10.5 GCP roles

- Policy Engine
- Approval Router
- Permissions / AuthZ Manager
- Quota Manager

These are used when execution actions require governance checks.

10.6 ECP roles

- Audit Logger

- Artifact Registry / Lineage Governor
- Notification Router

These are used where run actions require evidence capture or event routing.

---

11. Lifecycle Operations

The Run Controller must support the following operations.

11.1 `createRun`

Creates a new run in canonical initial state.

Must

- assign stable run ID
- create initial manifest structure
- attach pipeline reference
- initialize stage map
- set initial status

11.2 `startRun`

Begins execution of a valid run.

Must

- validate current status
- validate start eligibility
- record start transition
- set active execution state

11.3 `advanceStage`

Moves run forward to the next legal stage.

Must

- validate current stage
- validate prior state
- consult state machine
- ensure required governance checks are satisfied
- persist resulting state

11.4 `recordStageResult`

Applies a stage outcome to the run.

Must

- persist outcome
- record timestamps and result metadata
- determine next lifecycle action
- preserve failure or gate information where present

11.5 `pauseRun`

Applies paused state where allowed.

11.6 `resumeRun`

Resumes paused state where allowed and properly authorized.

11.7 `cancelRun`

Cancels run according to allowed cancellation rules.

11.8 `completeRun`

Moves run into terminal completion state.

11.9 `failRun`

Moves run into terminal failure state.

11.10 `gateRun`

Moves run into gated/hold state pending approval, policy resolution, or operator action.

11.11 `retryRun`

Re-enters execution through a legal retry path.

11.12 `rerunStage` or `rerunRun`

Optional controlled rerun behavior where explicitly allowed by policy and state rules.

---

12. Required Validation Path

Before any lifecycle-changing action is applied, the Run Controller must enforce the correct validation path.

12.1 Transition validation

Every lifecycle transition must be checked against the State Machine Governor.

12.2 Policy validation

Any action governed by policy must receive policy evaluation.

12.3 Authorization validation

Any actor-initiated protected action must pass authorization.

12.4 Approval validation

Any action requiring approval must not proceed until approval is satisfied.

12.5 Quota validation

Any resource-sensitive execution action must respect quota decisions.

12.6 Persistence validation

Canonical run state must remain consistent before and after update.

---

13. State Handling Rules

The Run Controller must not directly invent state truth.

It must:

- read canonical current state
- determine requested lifecycle action
- call validators
- apply allowed state update
- persist through the Run State Store
- emit supporting events and evidence records

The Run Controller must reject:

- stale lifecycle requests
- illegal transitions
- duplicate terminal actions
- advancement from inconsistent stage positions
- transitions missing required governance signals

---

14. Terminal State Rules

The Run Controller must define and enforce terminal outcomes clearly.

Completed

Used when the run has successfully passed all required execution and gating conditions.

Failed

Used when the run cannot proceed due to unrecoverable error or governed failure.

Gated

Used when the run is paused on a required decision, approval, policy hold, or similar governed checkpoint.

Cancelled

Used when the run is intentionally stopped under allowed cancellation rules.

Once terminalized, a run must not be reopened except through explicitly defined rerun/retry semantics.

---

15. Retry and Rerun Rules

The Run Controller must distinguish between:

Retry

Reattempt of a failed or blocked execution path within the same governed run context.

Rerun

A broader re-execution path that may restart a stage, stage set, or the entire run according to explicit rules.

The Run Controller must not allow retry/rerun unless:

- the state machine allows it

- policy allows it
- authorization requirements are met
- approval is satisfied where needed
- audit capture is performed

---

16. Audit Requirements

The Run Controller must ensure lifecycle actions are audit-linked.

At minimum, auditable lifecycle actions include:

- create run
- start run
- advance stage
- pause run
- resume run
- cancel run
- fail run
- complete run
- gated hold entry
- retry
- rerun

The audit trail must preserve:

- actor

- action
- target run
- time
- resulting status
- relevant evidence refs
- AI participation metadata where applicable

---

17. Notification Expectations

The Run Controller should emit or trigger events for important lifecycle changes, including:

- run started
- run gated
- run failed
- run cancelled
- run completed
- retry requested
- rerun requested
- protected transition denied

Notification delivery itself belongs to the Notification Router, but the Run Controller must generate the underlying event intent.

---

18. AI Assistance Boundary

The Run Controller may request AI assistance for:

- failure classification
- suggested retry path
- readable stage failure explanation
- operator summary drafting
- likely blocked dependency identification
- low-risk execution recommendation drafting

The AI is not the Run Controller.

AI may not:

- directly mutate run state
- force lifecycle transitions
- complete protected actions
- decide terminal truth
- overrule validation results

Any AI-proposed action must still pass deterministic validation before execution.

---

19. Failure Handling

The Run Controller must fail safely.

19.1 Invalid current state

Reject lifecycle request and preserve rejection reason.

19.2 Conflicting concurrent update

Reject or retry through deterministic concurrency rules.

19.3 Missing governance dependency

Block action until dependency result exists.

19.4 Store write failure

Do not partially commit lifecycle update.

19.5 AI output failure

Discard AI output and continue through deterministic operation.

---

20. Success Criteria

The Run Controller is correctly implemented when:

- all run lifecycle operations flow through one authoritative role
- stage advancement is centrally validated
- terminal states are applied consistently
- retries and reruns are governed explicitly
- lifecycle actions are auditable
- policy, authz, and approval dependencies are respected
- AI assistance remains bounded and non-authoritative

---

21. Summary Statement

The Run Controller is the authoritative lifecycle role within the Execution Control Plane.

It governs run creation, advancement, interruption, recovery, and terminal outcomes by coordinating deterministic validation and canonical state persistence.

It may use AI for assistance, but it does not delegate lifecycle authority to AI.

ICP-02 — Run State Store

ICP-02 — Run State Store

1. Purpose

The Run State Store is the role responsible for **persisting canonical run state** inside the Execution Control Plane.

Its purpose is to provide the single authoritative persistence layer for run records, stage state, transition history, lifecycle metadata, and related execution state.

The Run State Store governs **run truth at rest**.

It is the role that preserves what the system officially knows about a run.

---

2. Objective

The objective of the Run State Store is to ensure that:

- every run has a canonical stored record
- run state is durable and queryable
- stage state is preserved consistently
- transitions are reflected accurately in persistence
- lifecycle updates do not create conflicting truth
- execution services read and write through a stable contract
- historical run state remains inspectable
- persisted state remains usable with or without AI

---

3. Problem It Solves

Without a dedicated Run State Store, run truth becomes scattered across:

- transient runtime memory
- CLI execution state
- local files with inconsistent structure
- stage-level output folders
- duplicated status fields
- ad hoc JSON updates
- UI-side assumptions

That fragmentation causes:

- inconsistent run truth
- missing transition history
- race conditions in updates
- unrecoverable or ambiguous run state
- poor retry/recovery behavior
- weak operator trust in what the system considers canonical

The Run State Store solves this by making persistence authority explicit.

---

4. Scope

The Run State Store governs:

- run record creation
- canonical run manifest persistence

- run status persistence
- stage state persistence
- transition history persistence
- execution metadata persistence
- run listing and retrieval
- run state updates under controlled write rules
- stored run versioning expectations
- storage consistency expectations

The Run State Store does not govern:

- whether a transition is legal
- whether a policy passes
- whether an actor is authorized
- whether a release is publishable
- whether an artifact pin is valid
- queue scheduling policy
- environment routing policy

Those decisions are produced elsewhere and persisted here only after validation.

---

5. Core Principle

The Run State Store is the **canonical persistence authority**, not the decision-maker.

It stores the official execution state after governed roles determine what that state should be.

This means:

- it preserves truth
- it does not invent truth
- it does not reinterpret legality
- it does not resolve policy
- it does not replace audit
- it does not replace execution control

It is the stable persistence substrate for run lifecycle truth.

---

6. Primary Responsibilities

6.1 Create canonical run records

- create initial run record
- seed initial run metadata
- seed stage state structure
- persist initial status

6.2 Read canonical run records

- retrieve a run by ID
- retrieve current state
- retrieve state history where defined
- expose queryable run state for dependent services

6.3 Update canonical run records

- persist governed lifecycle changes
- persist stage state changes
- persist transition entries
- persist execution metadata updates
- maintain structural validity

6.4 List and query runs

- enumerate runs
- support run discovery
- expose query surfaces for status and filtering

6.5 Preserve consistency

- prevent malformed write application
- preserve required fields
- support controlled version evolution
- ensure persisted state remains structurally valid

---

7. Authority

The Run State Store has authority to:

- create canonical run records
- persist canonical state changes
- reject malformed persistence operations

- enforce storage contract requirements
- expose canonical run truth to readers

The Run State Store does not have authority to:

- approve transitions
- define state machine legality
- waive policy failures
- grant permissions
- advance runs by itself
- decide terminal outcomes by itself
- rewrite audit history
- fabricate missing stage results

It only stores what authorized and validated roles instruct it to store.

---

8. Data Governed by the Store

The Run State Store should govern these persisted data classes:

8.1 Run identity data

- run_id
- pipeline reference
- run creation metadata
- workspace/project reference if applicable

8.2 Run status data

- current run status
- terminal state markers
- active stage pointer where applicable
- lifecycle timestamps

8.3 Stage state data

- stage IDs
- stage statuses
- stage start/end timestamps
- stage outputs references
- stage result summaries
- stage failure/gate metadata

8.4 Transition data

- transition history entries
- prior status
- resulting status
- transition reason
- actor or system source where applicable

8.5 Execution metadata

- queue/schedule linkage
- environment routing references

- retry/rerun metadata
- control-plane linkage refs
- AI participation refs where applicable

---

9. Inputs

The Run State Store consumes:

9.1 Create/update requests

- create run record request
- update run record request
- append transition request
- patch stage state request
- terminalization update request

9.2 Structured run data

- canonical run manifest
- validated state changes
- transition payloads
- stage result payloads

9.3 Contract definitions

- run manifest schema
- stage state schema

- transition entry schema
- storage version rules

---

10. Outputs

The Run State Store produces:

10.1 Canonical persisted records

- run record
- stage state map
- transition history
- metadata snapshots

10.2 Query results

- run by ID
- run lists
- status-filtered run sets
- historical run snapshots where supported

10.3 Storage outcomes

- create success/failure
- update success/failure
- validation rejection
- conflict rejection where applicable

---

11. Primary Interfaces

The Run State Store should expose, at minimum, these operations.

11.1 `createRunRecord`

Create a new canonical run record.

Must

- require a stable run ID
- validate minimum schema
- reject duplicate canonical creation
- initialize required fields

11.2 `getRunRecord`

Return the canonical run record for a run ID.

Must

- return current stored truth
- reject unknown run IDs consistently

11.3 `updateRunRecord`

Persist a governed state update.

Must

- validate structural integrity
- preserve required fields
- avoid partial malformed updates

- maintain canonical shape

11.4 `listRuns`

Return available run records or summaries.

11.5 `queryRuns`

Return runs matching allowed query keys.

11.6 `appendTransition`

Persist a transition history entry tied to the run.

11.7 `patchStageState`

Persist stage-specific controlled updates without breaking overall run integrity.

---

12. Consistency Rules

The Run State Store must preserve canonical consistency.

12.1 Single canonical record rule

Each run must have one canonical current-state record.

12.2 Schema validity rule

Every persisted record must satisfy the current valid contract for that record version.

12.3 Required field preservation rule

Required fields may not disappear during updates.

12.4 Controlled update rule

Updates must occur through explicit store operations, not by uncontrolled direct mutation.

12.5 No invented state rule

The store must not synthesize missing execution truth on its own.

12.6 Stable identity rule

A run’s identity must not change after canonical creation.

---

13. Concurrency and Conflict Rules

The Run State Store must define behavior for conflicting or overlapping writes.

At minimum, it must handle:

13.1 Duplicate create attempt

Reject duplicate creation for the same run identity.

13.2 Stale update attempt

Reject or reconcile stale writes according to explicit version/concurrency rules.

13.3 Partial write failure

Do not leave canonical state in an invalid partial form.

13.4 Concurrent lifecycle updates

Require deterministic conflict handling so canonical truth does not fork.

The specific mechanism may vary by backend, but the role must define the rule.

---

14. Storage Model

The implementation may be JSON-backed, database-backed, or adapter-backed, but the role contract remains the same.

The storage model should support:

- durable canonical records

- predictable retrieval
- explicit version handling
- recoverable validation errors
- query surfaces for operators and dependent roles
- backend replacement without changing execution authority logic

This means persistence backend choice is an implementation detail, not the role definition.

---

15. Versioning Rules

The Run State Store must support storage version awareness.

It should define:

- record version field
- backward compatibility expectations where needed
- migration expectations where needed
- validation behavior for unknown versions

State persistence cannot rely on undocumented shape drift.

---

16. Relationship to Run Controller

The Run Controller decides when lifecycle changes should occur.

The Run State Store persists the resulting canonical truth.

In simple terms:

- Run Controller = movement authority

- Run State Store = persistence authority

The Run Controller must not bypass the Run State Store when updating canonical run truth.

---

17. Relationship to State Machine Governor

The State Machine Governor determines whether a transition is legal.

The Run State Store records the resulting state only after legality has been determined.

In simple terms:

- State Machine Governor = legality
- Run State Store = persisted truth

---

18. Relationship to Audit Logger

The Run State Store and Audit Logger are not the same thing.

The Run State Store preserves **current and structured execution truth**.
The Audit Logger preserves **append-only action evidence**.

Both may reference similar lifecycle events, but they serve different purposes.

In simple terms:

- Run State Store = canonical run state
- Audit Logger = historical proof of actions

---

19. AI Assistance Boundary

The Run State Store may be AI-assisted only in limited non-authoritative ways.

AI may help with:

- summarizing inconsistent state symptoms
- identifying likely stale or orphaned records
- explaining structural validation failures
- proposing repair candidates for operator review
- summarizing storage drift patterns

AI may not:

- directly write canonical state
- fabricate missing values as truth
- rewrite records without governed authorization
- resolve conflicting truth by itself
- decide final repair actions without deterministic validation and operator/governed approval where required

The Run State Store remains non-AI authoritative persistence.

---

20. Failure Handling

The Run State Store must fail safely.

20.1 Schema-invalid write

Reject the write and preserve reason.

20.2 Duplicate identity collision

Reject duplicate canonical creation.

20.3 Unknown run lookup

Return consistent not-found behavior.

20.4 Partial persistence failure

Avoid leaving invalid canonical records.

20.5 Backend unavailable

Return explicit storage failure and do not fabricate successful persistence.

20.6 AI analysis failure

Ignore AI assistance and preserve normal deterministic store behavior.

---

21. Required Guarantees

The Run State Store must guarantee:

- canonical run truth exists for created runs
- persisted state remains structurally valid
- canonical state is retrievable by run ID
- required fields remain preserved across updates
- lifecycle changes are durably recorded
- canonical truth does not fork through uncontrolled writes
- persistence behavior remains safe without AI

---

22. Success Criteria

The Run State Store is correctly established when:

- one canonical run record exists per run

- run state can be created, read, updated, and listed through explicit contracts
- lifecycle state remains structurally valid after updates
- transition history can be preserved consistently
- concurrent or stale writes do not silently corrupt truth
- backend choice does not change the role contract
- AI assistance remains advisory and non-writing

---

23. Summary Statement

The Run State Store is the canonical persistence role within the Execution Control Plane.

It preserves run identity, lifecycle state, stage state, transition history, and related execution metadata through controlled, durable, and structurally valid storage operations.

It may be AI-assisted for analysis, but it does not delegate canonical persistence authority to AI.

ICP-03 — State Machine Governor

ICP-03 — State Machine Governor

1. Purpose

The State Machine Governor is the role responsible for **defining and enforcing legal run and stage transitions** inside the Execution Control Plane.

Its purpose is to ensure that execution state changes occur only through explicit, valid, deterministic transition rules.

The State Machine Governor governs **state legality**.

It is the role that decides whether a proposed movement is allowed.

---

2. Objective

The objective of the State Machine Governor is to ensure that:

- run states are explicitly defined
- stage states are explicitly defined
- allowed transitions are known
- forbidden transitions are blocked
- retries, reruns, pauses, resumes, cancellations, gates, failures, and completions all follow formal rules
- execution movement cannot drift into undefined or contradictory state
- transition rejection reasons are consistent and inspectable
- AI may assist with interpretation but may not define legality

---

3. Problem It Solves

Without a dedicated state machine role, transition legality tends to become scattered across:

- controller logic
- stage handlers
- CLI commands
- UI assumptions
- retry helpers
- error handlers
- ad hoc conditional branches

That fragmentation causes:

- illegal stage skips
- inconsistent pause/resume behavior
- weak retry semantics
- contradictory terminal state handling
- duplicate transition logic
- poor operator visibility into why a transition was denied

The State Machine Governor solves this by making transition law explicit.

---

4. Scope

The State Machine Governor governs:

- run state definitions
- stage state definitions

- allowed run transitions
- allowed stage transitions
- transition preconditions
- terminal state legality
- retry legality
- rerun legality
- pause/resume legality
- cancel legality
- gated/hold legality
- transition rejection reasons

The State Machine Governor does not govern:

- persistence implementation
- policy evaluation
- approval routing
- permission evaluation
- release governance
- audit storage
- artifact integrity truth

It governs legality only.

---

5. Core Principle

The State Machine Governor is the **source of execution transition law**.

It does not execute transitions by itself.
It evaluates whether a requested transition is legal according to defined rules.

This means:

- it defines legal movement
- it rejects illegal movement
- it does not persist state itself
- it does not invent policy exceptions
- it does not approve protected actions
- it does not replace the Run Controller

It is the deterministic legality layer for execution state.

---

6. State Domains

The State Machine Governor must govern at least two state domains.

6.1 Run state domain

Examples may include:

- created
- queued
- scheduled
- running
- paused
- gated

- failed
- cancelled
- completed

6.2 Stage state domain

Examples may include:

- pending
- ready
- running
- passed
- failed
- gated
- skipped
- cancelled

The final exact enums may vary by implementation, but the role must formalize them.

---

7. Primary Responsibilities

7.1 Define state sets

- define valid run states
- define valid stage states
- define terminal states
- define recoverable states

7.2 Define allowed transitions

- enumerate legal source → target transitions
- define transition-specific preconditions
- distinguish normal progression from recovery movement

7.3 Validate proposed transitions

- validate run transition requests
- validate stage transition requests
- validate retry/rerun requests
- validate pause/resume/cancel requests

7.4 Reject invalid transitions

- reject undefined transitions
- reject contradictory transitions
- reject transitions from terminal states unless explicitly allowed

7.5 Provide explainable outcomes

- return pass/fail transition result
- return rejection reason
- expose allowed next moves where useful

---

8. Authority

The State Machine Governor has authority to:

- define legal run state transitions
- define legal stage state transitions
- approve or reject proposed transitions
- identify valid next states
- classify terminal versus recoverable movement

The State Machine Governor does not have authority to:

- apply state updates directly
- override policy failures
- authorize actors
- approve protected actions
- rewrite canonical run truth
- fabricate evidence
- bypass the Run Controller

It decides legality, not execution or persistence.

---

9. Inputs

The State Machine Governor consumes:

9.1 Current state inputs

- current run state
- current stage state
- active stage pointer where relevant

- transition history where relevant

9.2 Proposed action inputs

- requested lifecycle action
- requested source → target transition
- retry request
- rerun request
- pause/resume/cancel request

9.3 Context inputs

- pipeline ordering information
- stage position metadata
- terminal state markers
- recovery policy metadata
- current run mode or dispatch context where relevant

---

10. Outputs

The State Machine Governor produces:

10.1 Transition validation result

- allowed
- denied

10.2 Rejection data

- rejection reason code
- readable rejection explanation
- blocking condition summary

10.3 Optional next-step guidance

- allowed next states
- valid recovery paths
- legal retry/rerun options

The role may provide explanation support, but the authoritative output is the legality result.

---

11. Primary Interfaces

The State Machine Governor should expose, at minimum:

11.1 `validateRunTransition`

Validate a proposed run-state transition.

11.2 `validateStageTransition`

Validate a proposed stage-state transition.

11.3 `getAllowedRunTransitions`

Return legal target states from a given run state.

11.4 `getAllowedStageTransitions`

Return legal target states from a given stage state.

11.5 `validateRetry`

Validate whether retry is legal under the current run/stage context.

11.6 `validateRerun`

Validate whether rerun is legal under the current governed context.

11.7 `explainTransitionRejection`

Return structured rejection explanation.

---

12. Run Transition Rules

The State Machine Governor must define run transition law explicitly.

Examples of rule categories:

12.1 Initial movement

- created → queued
- created → scheduled
- created → running

12.2 Active execution movement

- queued → running
- scheduled → queued
- running → paused
- paused → running

12.3 Hold and governance movement

- running → gated
- gated → running only when required conditions are satisfied

12.4 Terminal movement

- running → completed
- running → failed
- running → cancelled
- paused → cancelled
- gated → cancelled

12.5 Forbidden examples

- completed → running
- cancelled → running
- failed → completed without governed recovery path
- completed → paused

The exact table belongs in implementation or companion contract, but the artifact must require explicit definition.

---

13. Stage Transition Rules

The State Machine Governor must also define stage transition law.

Examples of rule categories:

13.1 Normal progression

- pending → ready
- ready → running
- running → passed
- running → failed

- running → gated

13.2 Recovery movement

- failed → ready only if retry is allowed
- gated → ready only if hold is cleared
- running → cancelled only if cancellation is legal

13.3 Forbidden examples

- pending → passed without execution
- passed → running without rerun rules
- failed → passed without legal retry/rerun path

---

14. Terminal State Rules

The State Machine Governor must explicitly define which states are terminal.

14.1 Run terminal states

Expected examples:

- completed
- failed
- cancelled

14.2 Stage terminal states

Expected examples may include:

- passed

- failed
- cancelled
- skipped, depending on contract

14.3 Terminal recovery rules

Terminal states must not reopen unless:

- an explicit retry rule exists, or
- an explicit rerun rule exists, or
- a controlled replacement-run policy exists

The State Machine Governor must define the difference.

---

15. Retry Rules

Retry must be governed separately from ordinary progression.

The State Machine Governor must define:

- when retry is allowed
- which states are retry-eligible
- whether retry applies at run level, stage level, or both
- whether failure class affects retry legality
- whether governance preconditions must be rechecked before retry

Retry is not implied just because a stage failed.

---

16. Rerun Rules

Rerun must also be governed explicitly.

The State Machine Governor must define:

- whether rerun means stage rerun, partial rerun, or full-run rerun
- when rerun is allowed
- whether rerun creates a new run context or reopens an existing one
- what prior states must be preserved
- what governance requirements apply to rerun entry

Rerun is not equivalent to retry and must not be treated as informal re-execution.

---

17. Pause / Resume / Cancel Rules

These interruption paths must be formalized.

Pause

Allowed only from explicitly eligible active states.

Resume

Allowed only from paused or equivalent recoverable hold states and only when prerequisite conditions are satisfied.

Cancel

Allowed only from explicitly cancellable states.

The State Machine Governor must block:

- pause from terminal states
- resume from terminal states
- resume from gated state unless rules define it

- cancel reversal unless explicit recovery path exists

---

18. Gated / Hold Rules

The State Machine Governor must define how governed holds affect execution movement.

This includes:

- what states may enter gated/hold
- whether gated is run-level, stage-level, or both
- what conditions allow exit from gated
- whether gated may transition to cancelled, failed, or resumed execution
- whether approval or policy clearance is required for exit

This role does not perform approvals, but it must encode the movement law.

---

19. Relationship to Run Controller

The Run Controller requests lifecycle movement.

The State Machine Governor decides whether that movement is legal.

In simple terms:

- Run Controller = asks to move
- State Machine Governor = decides whether move is legal

The Run Controller must not apply lifecycle changes without State Machine validation.

---

20. Relationship to Run State Store

The Run State Store preserves the canonical result after a legal transition is applied.

The State Machine Governor does not persist the transition.

In simple terms:

- State Machine Governor = legality
- Run State Store = stored truth

---

21. Relationship to Governance Control Plane

The State Machine Governor defines execution legality, but some legal transitions may still require governance signals before execution.

For example:

- a transition may be structurally legal
- but blocked until approval exists
- or blocked until authz passes
- or blocked until policy clears

So the relationship is:

- State Machine Governor = structurally legal?
- Governance Control Plane = allowed under governance conditions?

Both may be required before the Run Controller applies movement.

---

22. AI Assistance Boundary

The State Machine Governor may be AI-assisted only in non-authoritative ways.

AI may help with:

- explaining why a transition was denied
- summarizing allowed next moves
- classifying common transition failure patterns
- drafting operator-readable transition explanations

AI may not:

- define legal transitions
- modify transition tables by itself
- invent exceptions
- authorize forbidden movement
- reinterpret illegal transitions as legal

The State Machine Governor remains deterministic and non-AI authoritative.

---

23. Failure Handling

The State Machine Governor must fail safely.

23.1 Unknown state

Reject transition and return explicit unknown-state error.

23.2 Undefined transition

Reject transition and preserve rejection reason.

23.3 Ambiguous transition rule

Fail closed until explicit rule exists.

23.4 Missing context needed for validation

Reject or defer with explicit blocking condition.

23.5 AI explanation failure

Ignore AI assistance and preserve normal deterministic legality output.

---

24. Required Guarantees

The State Machine Governor must guarantee:

- all governed run states are explicitly defined
- all governed stage states are explicitly defined
- legal transitions are deterministic
- illegal transitions are consistently rejected
- terminal-state rules are explicit
- retry/rerun rules are explicit
- pause/resume/cancel rules are explicit
- legality remains independent of AI assistance

---

25. Success Criteria

The State Machine Governor is correctly established when:

- no execution transition depends on ad hoc scattered logic
- run and stage state sets are explicit
- legal transitions are centrally defined
- invalid transitions are blocked consistently

- retry and rerun semantics are distinguished clearly
- hold and interruption behavior are governed explicitly
- operators can inspect why a transition was denied
- AI only assists with explanation, not legality

---

26. Summary Statement

The State Machine Governor is the transition-law role within the Execution Control Plane.

It defines and enforces legal run and stage movement, including progression, interruption, recovery, terminal outcomes, and governed holds.

It may use AI for explanation support, but it does not delegate transition legality to AI.

ICP-04 — Queue / Scheduler Manager

ICP-04 — Queue / Scheduler Manager

1. Purpose

The Queue / Scheduler Manager is the role responsible for **governing queued, deferred, prioritized, and dispatchable execution movement** inside the Execution Control Plane.

Its purpose is to ensure that runs do not move into execution arbitrarily. Instead, they enter and leave pending execution through explicit queueing, scheduling, prioritization, throttling, and dispatch rules.

The Queue / Scheduler Manager governs **pending execution flow**.

It is the role that decides when a run is eligible to be dispatched for execution movement.

---

2. Objective

The objective of the Queue / Scheduler Manager is to ensure that:

- runs can be queued explicitly
- future runs can be scheduled explicitly
- pending execution is ordered by deterministic rules
- dispatch happens only when prerequisites are satisfied
- execution capacity is respected
- privileged reordering is governed
- retries are coordinated through formal dispatch logic
- queue pressure and starvation are controlled
- AI may assist prioritization analysis without becoming dispatch authority

---

3. Problem It Solves

Without a dedicated queue/scheduler role, pending execution control becomes fragmented across:

- ad hoc CLI starts
- direct run controller calls
- operator timing decisions
- environment-specific scripts
- manual retry patterns
- implicit ordering assumptions

That fragmentation causes:

- unfair run ordering
- priority conflicts
- unbounded retries
- accidental concurrent overload
- no durable scheduled execution model
- weak control over deferred work
- poor operator visibility into why one run is executing before another

The Queue / Scheduler Manager solves this by making pending execution authority explicit.

---

4. Scope

The Queue / Scheduler Manager governs:

- enqueueing runs
- removing runs from queue
- scheduled execution registration
- dispatch eligibility
- dispatch ordering
- priority handling
- retry scheduling
- backoff behavior
- worker slot or capacity reservation logic
- starvation prevention rules
- queue visibility and status

The Queue / Scheduler Manager does not govern:

- whether a run transition is structurally legal
- whether a policy passes
- whether an actor is authorized
- whether a target environment is supported
- canonical run persistence
- artifact evidence truth
- release behavior

Those decisions are produced elsewhere and used as inputs to dispatch decisions.

---

5. Core Principle

The Queue / Scheduler Manager is the **pending execution authority**, not the execution engine itself.

It determines when work becomes eligible to move from waiting state into dispatch-ready state according to explicit rules.

This means:

- it governs order
- it governs timing
- it governs dispatch readiness
- it does not invent policy
- it does not invent legality
- it does not perform stage execution by itself

It is the deterministic control layer for pending execution flow.

---

6. Primary Responsibilities

6.1 Enqueue runs

- place runs into a durable queue
- assign queue metadata
- preserve enqueue time and priority
- prevent invalid duplicate queue placement where prohibited

6.2 Schedule future execution

- register a run for future dispatch

- preserve execution window or target time
- move scheduled work into queue or dispatch-ready state at the correct time

6.3 Prioritize queued work

- order queued runs by deterministic rules
- apply priority classes
- support governed administrative reordering
- prevent starvation of long-waiting work

6.4 Dispatch eligible work

- determine dispatch eligibility
- reserve available execution capacity
- hand off dispatchable run to the Run Controller / execution path
- preserve dispatch records

6.5 Coordinate retries and backoff

- place retryable work back into pending flow
- apply retry timing and backoff rules
- prevent unbounded thrash

6.6 Manage queue visibility

- expose queue state
- expose scheduled state
- expose blocked dispatch reasons

- expose capacity pressure state

---

7. Authority

The Queue / Scheduler Manager has authority to:

- enqueue a run
- schedule a run for later execution
- determine pending ordering
- determine dispatch readiness
- allocate dispatch opportunity according to deterministic rules
- reject invalid queue or schedule requests
- throttle dispatch when limits apply

The Queue / Scheduler Manager does not have authority to:

- override state machine legality
- override policy denials
- bypass approvals
- bypass authorization
- invent environment capability
- directly rewrite canonical run truth
- directly change quotas
- directly execute protected actions without downstream validation

It governs when pending execution may proceed, not whether all other governed conditions are satisfied in general.

---

8. Inputs

The Queue / Scheduler Manager consumes:

8.1 Queueing inputs

- enqueue request
- priority class
- queue policy metadata
- actor identity where needed

8.2 Scheduling inputs

- target time
- schedule window
- retry-after timestamp
- recurrence metadata if supported

8.3 Dispatch-readiness inputs

- current run state
- state machine eligibility
- policy clearance status
- authz result where needed
- quota status

- environment availability
- worker/capacity state

8.4 Operational context

- existing queue contents
- active dispatch count
- backlog pressure
- retry metadata
- failure history where relevant

---

9. Outputs

The Queue / Scheduler Manager produces:

9.1 Queue records

- queue entry
- updated queue position
- queue removal record

9.2 Schedule records

- schedule entry
- activation record
- schedule cancellation or expiry record

9.3 Dispatch records

- dispatch assignment
- dispatch denial reason
- retry scheduling decision
- throttle/defer record

9.4 Visibility outputs

- queue state snapshot
- schedule state snapshot
- capacity summary
- blocked-dispatch reasons

---

10. Primary Interfaces

The Queue / Scheduler Manager should expose, at minimum:

10.1 `enqueueRun`

Place a run into pending queue state.

10.2 `removeFromQueue`

Remove a run from the queue under valid conditions.

10.3 `scheduleRun`

Register a run for deferred future execution.

10.4 `cancelSchedule`

Cancel a pending schedule under allowed conditions.

10.5 `getQueueState`

Return current queue contents, positions, and queue metadata.

10.6 `getScheduleState`

Return current scheduled work and its activation metadata.

10.7 `dispatchNext`

Select and dispatch the next eligible queued/scheduled run.

10.8 `retryDispatch`

Place dispatch-failed or retry-eligible work back into pending flow according to policy.

10.9 `reprioritizeRun`

Apply governed queue priority changes where allowed.

---

11. Queue Rules

The Queue / Scheduler Manager must define explicit queue rules.

11.1 Queue identity rule

A queue entry must have stable identity tied to a governed run.

11.2 Queue state rule

Queue entry states must be explicit, such as:

- queued
- scheduled
- ready_to_dispatch
- dispatching
- deferred

- blocked
- cancelled
- expired

11.3 Duplicate queue rule

The system must define whether a run may have multiple active queue entries. Default expectation should be no uncontrolled duplication.

11.4 Position/order rule

Queue order must be based on explicit rules, not incidental data structure ordering.

11.5 Queue removal rule

Removal must be explicit and auditable where applicable.

---

12. Scheduling Rules

The Queue / Scheduler Manager must define scheduling behavior explicitly.

12.1 Deferred execution rule

A scheduled run must not dispatch before its eligible time/window.

12.2 Activation rule

When a scheduled run becomes eligible, it must transition into the proper ready/queued/dispatchable state according to explicit rules.

12.3 Expiry rule

If a scheduled dispatch window expires, the system must define whether the run is cancelled, deferred, or re-evaluated.

12.4 Retry scheduling rule

Retries must use explicit timing behavior, not ad hoc immediate reentry.

---

13. Priority Rules

The Queue / Scheduler Manager must define:

- priority classes
- tie-break behavior
- fairness rules
- starvation prevention behavior
- administrative override conditions
- priority escalation or decay behavior where applicable

Priority cannot be left implicit.

Examples may include:

- normal
- high
- critical
- low-priority backfill

But the role requires only that priority be explicit and deterministic.

---

14. Dispatch Eligibility Rules

A run may only dispatch when required conditions are satisfied.

The Queue / Scheduler Manager must define how dispatch eligibility depends on:

- queue/schedule state

- run state eligibility
- quota availability
- capacity/slot availability
- environment readiness
- required governance prerequisites
- retry timing conditions

A run may be:

- queued but not eligible
- scheduled but not active
- eligible but blocked by capacity
- eligible but blocked by governance dependency
- eligible and dispatch-ready

These distinctions must be explicit.

---

15. Capacity and Throttling Rules

The Queue / Scheduler Manager must govern execution pressure.

It must define:

- max concurrent dispatch limits
- capacity reservation rules
- per-project/workspace throttling if applicable
- dispatch denial when limits are hit

- backpressure behavior
- queue growth handling expectations

The role does not define quota policy itself, but it must consume quota/capacity signals in dispatch control.

---

16. Retry and Backoff Rules

Dispatch-level retry must be formalized.

The Queue / Scheduler Manager must define:

- what failures are retryable at dispatch level
- when immediate retry is forbidden
- backoff progression rules
- max retry count or retry exhaustion behavior
- reset conditions
- operator intervention triggers where applicable

Retry cannot be open-ended or implicit.

---

17. Relationship to Run Controller

The Run Controller governs run lifecycle movement.

The Queue / Scheduler Manager governs when pending work becomes eligible to enter lifecycle movement.

In simple terms:

- Queue / Scheduler Manager = when pending work may dispatch

- Run Controller = what lifecycle action occurs when it dispatches

The Queue / Scheduler Manager does not replace the Run Controller.

---

18. Relationship to State Machine Governor

The State Machine Governor decides whether a transition is legal.

The Queue / Scheduler Manager decides whether eligible pending work should be dispatched now, later, or not at all.

In simple terms:

- State Machine Governor = structurally legal?
- Queue / Scheduler Manager = dispatch now or wait?

Both may be required before execution proceeds.

---

19. Relationship to Governance Control Plane

The Queue / Scheduler Manager may require governance signals before dispatch, such as:

- quota clearance
- approval satisfaction
- policy clearance
- authorization for privileged queue actions

It does not perform those evaluations itself, but it must honor them in dispatch control.

---

20. Relationship to Adapter / Environment Manager

The Queue / Scheduler Manager must coordinate with environment routing and capacity information.

The Adapter / Environment Manager answers:

- where execution can go
- what capabilities exist
- what targets are valid

The Queue / Scheduler Manager answers:

- when a run should be offered for dispatch into that environment path

---

21. Audit Requirements

The Queue / Scheduler Manager must ensure auditable coverage for important pending-execution actions.

At minimum, audit-relevant actions include:

- enqueue run
- remove from queue
- schedule run
- cancel schedule
- reprioritize run
- dispatch run
- defer dispatch
- retry dispatch
- expire scheduled work

The audit trail should preserve:

- actor or system source
- action type
- target run
- prior and resulting queue/schedule state
- timing metadata
- reason metadata
- AI participation metadata where applicable

---

22. AI Assistance Boundary

The Queue / Scheduler Manager may be AI-assisted in bounded ways.

AI may help with:

- queue pressure forecasting
- suggesting retry timing
- recommending priority changes for operator review
- identifying starvation risk
- summarizing blocked queue reasons
- recommending capacity balancing actions

AI may not:

- directly dispatch protected work
- bypass quota or policy limits

- silently reprioritize privileged work
- invent eligibility
- override deterministic queue or schedule rules

The Queue / Scheduler Manager remains the authoritative pending-execution controller.

---

23. Failure Handling

The Queue / Scheduler Manager must fail safely.

23.1 Invalid queue request

Reject enqueue/schedule request and preserve reason.

23.2 Duplicate active entry conflict

Reject or reconcile according to explicit duplication rules.

23.3 Capacity unavailable

Do not dispatch; preserve blocked/deferred reason.

23.4 Missing dispatch dependency

Do not dispatch; preserve dependency-blocked status.

23.5 Retry exhaustion

Stop retry loop and surface deterministic exhaustion result.

23.6 AI recommendation failure

Ignore AI assistance and preserve deterministic queue behavior.

---

24. Required Guarantees

The Queue / Scheduler Manager must guarantee:

- queued work is explicitly represented
- scheduled work is explicitly represented
- dispatch order is deterministic
- capacity and throttling are honored
- retries follow explicit backoff rules
- blocked work preserves clear reason state
- pending execution behavior remains safe without AI

---

25. Success Criteria

The Queue / Scheduler Manager is correctly established when:

- runs can be queued and scheduled through explicit contracts
- dispatch order is governed and inspectable
- dispatch eligibility is explicit
- priority handling is deterministic
- retries and backoff are governed
- queue pressure and starvation are managed intentionally
- AI remains advisory and non-dispatch-authoritative

---

26. Summary Statement

The Queue / Scheduler Manager is the pending-execution control role within the Execution Control Plane.

It governs queueing, scheduling, prioritization, throttling, retry timing, and dispatch readiness through deterministic rules.

It may use AI for forecasting and recommendation support, but it does not delegate dispatch authority to AI.

ICP-05 — Adapter / Environment Manager

ICP-05 — Adapter / Environment Manager

1. Purpose

The Adapter / Environment Manager is the role responsible for **governing execution target selection, environment capability truth, and execution-route suitability** inside the Execution Control Plane.

Its purpose is to ensure that runs are executed only in environments that are valid, supported, available, and compatible with the run's requirements.

The Adapter / Environment Manager governs **where execution may occur**.

It is the role that determines whether a run can be routed to a given execution target.

---

2. Objective

The objective of the Adapter / Environment Manager is to ensure that:

- execution targets are explicitly registered
- environment capabilities are explicitly represented
- route selection is deterministic
- unsupported environments are rejected
- capability mismatches are surfaced clearly
- environment readiness is checked before dispatch
- execution-routing decisions are inspectable and auditable
- AI may assist with recommendation, but not invent environment truth

---

3. Problem It Solves

Without a dedicated adapter/environment role, execution targeting becomes fragmented across:

- local scripts
- CI assumptions
- machine-specific configuration
- implicit container/runtime expectations
- manual operator choices
- ad hoc environment checks

That fragmentation causes:

- invalid dispatch targets
- environment mismatch failures
- hidden runtime dependencies
- inconsistent capability assumptions
- poor portability
- weak operator trust in why a run was routed somewhere

The Adapter / Environment Manager solves this by making execution-target authority explicit.

---

4. Scope

The Adapter / Environment Manager governs:

- adapter registration
- execution target identity
- environment profile definition

- capability discovery and representation
- environment readiness validation
- route selection rules
- route rejection rules
- target suitability evaluation
- execution environment metadata
- environment compatibility checks

The Adapter / Environment Manager does not govern:

- queue priority
- policy approval
- permission approval
- canonical run persistence
- transition legality
- artifact integrity truth
- release authority

Those are handled by other roles.

---

5. Core Principle

The Adapter / Environment Manager is the **execution-target authority**, not the executor itself.

It defines what execution environments exist, what they can do, whether they are ready, and whether a run may be routed to them.

This means:

- it governs target truth
- it governs capability truth
- it governs route suitability
- it does not itself run the stages
- it does not itself approve policy exceptions
- it does not itself dispatch queue order

It is the deterministic routing and environment-compatibility authority for execution.

---

6. Primary Responsibilities

6.1 Register adapters

- define execution adapters
- assign stable adapter identity
- classify adapter type
- preserve adapter metadata

6.2 Represent environment profiles

- define environment profile contract
- preserve runtime attributes
- preserve capacity or readiness signals where relevant
- preserve compatibility metadata

6.3 Discover or declare capabilities

- represent supported capabilities
- expose capability truth to dependent roles
- distinguish required vs optional capabilities
- preserve version-sensitive capability data where relevant

6.4 Validate environment readiness

- determine whether a target is currently eligible
- surface readiness failures
- surface missing prerequisites
- preserve environment validation result

6.5 Select execution targets

- choose valid target environments using explicit rules
- reject unsupported targets
- preserve routing decisions
- expose why one environment was selected over another

6.6 Support routing inspection

- list valid targets
- explain route rejection
- expose target suitability summaries

---

7. Authority

The Adapter / Environment Manager has authority to:

- register and describe execution targets
- define environment capability truth
- validate target readiness
- determine whether a route is compatible
- approve or reject environment selection based on deterministic rules

The Adapter / Environment Manager does not have authority to:

- override queue order
- bypass policy restrictions
- authorize actors
- bypass state machine rules
- mutate canonical run truth directly
- fabricate readiness signals
- invent capabilities not actually supported

It decides environment validity and routing suitability, not overall execution permission in the broadest sense.

---

8. Environment Model

The Adapter / Environment Manager should formalize at least these concepts.

8.1 Adapter

A registered execution mechanism or target pathway.

Examples may include:

- local adapter
- CI adapter
- container adapter
- remote worker adapter
- sandbox adapter

8.2 Environment profile

A structured description of an execution context.

Examples of environment profile data may include:

- runtime type
- OS/platform
- tool availability
- network posture
- filesystem properties
- container/runtime support
- secrets availability class
- execution constraints

8.3 Capability set

A structured set of supported execution capabilities.

Examples may include:

- shell execution
- file write/read

- network access class
- container build support
- package installation allowance
- GPU/accelerator access
- test/runtime dependencies
- artifact packaging support

8.4 Readiness state

A deterministic status indicating whether a target is currently usable.

Examples may include:

- ready
- degraded
- unavailable
- restricted
- unknown

---

9. Inputs

The Adapter / Environment Manager consumes:

9.1 Target-definition inputs

- adapter registration request
- environment profile definition
- capability declaration

- readiness signal input

9.2 Run-requirement inputs

- run execution requirements
- stage capability requirements
- environment restrictions
- runtime dependency requirements
- quota/capacity constraints where relevant

9.3 Operational inputs

- current target availability
- failure history where relevant
- dispatch request context
- policy constraints affecting environment use

---

10. Outputs

The Adapter / Environment Manager produces:

10.1 Adapter records

- adapter registry entry
- environment profile record
- capability snapshot

10.2 Validation outputs

- readiness result
- capability match result
- environment rejection reason
- route suitability result

10.3 Routing outputs

- selected execution target
- ranked target candidates where allowed
- route denial result
- target decision explanation

---

11. Primary Interfaces

The Adapter / Environment Manager should expose, at minimum:

11.1 `registerAdapter`

Register a new execution adapter.

11.2 `listAdapters`

Return available registered adapters.

11.3 `getAdapter`

Return a specific adapter definition and environment profile.

11.4 `getCapabilities`

Return capability data for an adapter/environment.

11.5 `validateEnvironment`

Validate current target readiness and compatibility.

11.6 `selectExecutionTarget`

Choose the best valid target under explicit routing rules.

11.7 `explainRouteDecision`

Return structured explanation of why a target was selected or rejected.

---

12. Adapter Registration Rules

The Adapter / Environment Manager must define how adapters become valid system targets.

At minimum:

12.1 Stable identity rule

Each adapter must have stable identity.

12.2 Adapter metadata rule

Each adapter must preserve enough metadata to support routing and validation.

12.3 Explicit registration rule

Execution targets must not exist only as implicit hardcoded assumptions.

12.4 Adapter type rule

Adapter classification must be explicit.

12.5 Compatibility declaration rule

An adapter must declare or derive what execution classes it supports.

---

13. Capability Rules

Capabilities must be explicit and structured.

The Adapter / Environment Manager must define:

- capability names or classes
- required vs optional capabilities
- compatibility matching behavior
- capability version sensitivity where relevant
- unsupported capability handling

A run must not be routed to an environment solely on optimistic assumption.

---

14. Readiness Rules

Readiness must be governed explicitly.

The Adapter / Environment Manager must define:

- how readiness is determined
- when readiness expires
- how degraded state is represented
- what failures make a target unavailable
- how stale readiness information is handled
- whether readiness applies globally, per adapter, or per environment instance

A target that is capability-compatible but not ready must not be treated as fully dispatchable.

---

15. Routing Rules

Routing must be deterministic.

The Adapter / Environment Manager must define:

- how candidate environments are generated
- how invalid candidates are rejected
- how tie-breaking works between valid candidates
- how policy or quota restrictions reduce eligible targets
- how preferred target selection works
- how fallback targets are selected

Route selection cannot rely on hidden operator guesswork.

---

16. Rejection Rules

The Adapter / Environment Manager must return explicit route rejection reasons when routing fails.

Examples of rejection classes may include:

- unsupported capability
- environment unavailable
- readiness stale
- restricted by policy
- capacity unavailable
- runtime mismatch
- required dependency absent
- adapter disabled

The exact codes may vary, but rejection must be inspectable.

---

17. Relationship to Queue / Scheduler Manager

The Queue / Scheduler Manager governs when pending work should dispatch.

The Adapter / Environment Manager governs where that dispatch may legally land.

In simple terms:

- Queue / Scheduler Manager = when to dispatch
- Adapter / Environment Manager = where to dispatch

A run may be eligible in queue terms but still unroutable due to environment incompatibility.

---

18. Relationship to Run Controller

The Run Controller governs lifecycle movement.

The Adapter / Environment Manager supplies validated target selection and route suitability that the Run Controller or dispatch path uses.

In simple terms:

- Run Controller = governs run movement
- Adapter / Environment Manager = governs target suitability for that movement

The Run Controller must not assume target validity without environment validation.

---

19. Relationship to Governance Control Plane

The Adapter / Environment Manager may consume governance constraints such as:

- restricted environments

- authorization-sensitive targets
- quota-restricted environments
- policy constraints around where execution may occur

It does not evaluate those policies itself, but it must honor their outcomes in route selection.

---

20. Relationship to Evidence Control Plane

The Adapter / Environment Manager may emit route decisions and validation results that need:

- audit capture
- notification triggers
- artifact provenance linkage where relevant

But it does not own those evidence systems.

---

21. Audit Requirements

The Adapter / Environment Manager must ensure auditable coverage for important environment decisions.

At minimum, audit-relevant actions include:

- register adapter
- disable adapter
- route selection
- route rejection
- environment readiness failure

- privileged target override
- capability declaration change

The audit trail should preserve:

- actor or system source
- action
- target adapter/environment
- resulting readiness/routing state
- reason metadata
- AI participation metadata where applicable

---

22. AI Assistance Boundary

The Adapter / Environment Manager may be AI-assisted in bounded ways.

AI may help with:

- recommending a best-fit target among already valid candidates
- explaining capability mismatch
- summarizing repeated environment failures
- predicting likely target instability
- suggesting fallback routes for operator review

AI may not:

- invent capabilities
- declare an unready target ready

- bypass environment restrictions
- select unsupported routes as authoritative truth
- overrule deterministic compatibility checks

The Adapter / Environment Manager remains the authoritative source of route validity.

---

23. Failure Handling

The Adapter / Environment Manager must fail safely.

23.1 Unknown adapter

Reject route selection and preserve reason.

23.2 Missing capability declaration

Fail closed or treat capability as unsupported until explicit truth exists.

23.3 Readiness unavailable

Do not assume readiness; preserve blocked state.

23.4 No compatible target found

Return explicit unroutable result.

23.5 Policy-restricted target

Reject target from eligible set.

23.6 AI recommendation failure

Ignore AI assistance and preserve deterministic route evaluation.

---

24. Required Guarantees

The Adapter / Environment Manager must guarantee:

- execution targets are explicitly represented
- environment capabilities are explicitly represented
- target readiness is explicitly represented
- route decisions are deterministic
- unsupported environments are rejected
- route rejection reasons are inspectable
- environment-routing behavior remains safe without AI

---

25. Success Criteria

The Adapter / Environment Manager is correctly established when:

- execution targets are first-class registered system objects
- capability matching is explicit
- readiness is explicit
- routing decisions are deterministic
- incompatible targets are rejected consistently
- queue dispatch can depend on target suitability cleanly
- AI remains advisory and non-authoritative for routing truth

---

26. Summary Statement

The Adapter / Environment Manager is the execution-target governance role within the Execution Control Plane.

It governs adapter registration, environment capability truth, readiness validation, and routing suitability through explicit deterministic rules.

It may use AI for recommendation and explanation support, but it does not delegate environment authority to AI.

GCP Artifact Pack

GCP-00 — Purpose + Boundaries

1. Purpose

The Governance Control Plane (GCP) is the control plane responsible for **deciding whether governed actions are allowed to occur** inside Axion.

Its purpose is to provide the authoritative governance layer for:

- policy evaluation
- approval routing
- override handling
- permissions and authorization
- quota and usage governance

The GCP exists to govern *permission to act*.

It is the plane that determines whether a requested action is allowed, blocked, deferred, escalated, or requires additional evidence or approval before execution may continue.

---

2. Objective

The objective of the GCP is to ensure that:

- governed actions are evaluated against explicit rules
- protected actions cannot occur without required checks
- approvals are routed through explicit authority paths
- overrides are controlled, time-bounded, and auditable
- permissions are explicit and enforceable

- quotas and usage constraints are respected
- deterministic governance remains authoritative even when AI assists with analysis

---

3. Problem It Solves

Without a formal governance control plane, governance logic becomes fragmented across:

- controller code
- UI conditions
- operator habits
- manual signoff processes
- ad hoc policy checks
- undocumented permission assumptions
- inconsistent quota enforcement

That fragmentation causes:

- unclear action authority
- hidden approval requirements
- policy bypass risk
- inconsistent override handling
- weak operator trust
- poor auditability of why something was allowed or denied

The GCP solves this by making governance authority explicit.

---

4. What the GCP Governs

The GCP governs:

- action allow/deny decisions
- policy evaluation
- protected-action detection
- approval-required routing
- approval decision linkage
- override request and activation rules
- actor authorization
- separation-of-duties constraints
- quota enforcement signals
- governance denial reasons
- governance dependency satisfaction for downstream actions

---

5. What the GCP Does Not Govern

The GCP does **not** govern:

- run persistence implementation
- execution-state legality
- queue ordering
- dispatch timing
- environment capability truth

- artifact hash truth
- append-only audit storage mechanics
- release evidence storage
- artifact lineage truth

Those belong to the Execution or Evidence Control Planes.

The GCP may block or condition their actions, but it does not absorb their role responsibilities.

---

6. Core Principle

The GCP is the **governance authority**, not the execution engine and not the evidence store.

It decides whether an action is allowed under:

- policy
- approval requirements
- authorization requirements
- override rules
- quota rules

This means:

- it governs permission
- it governs exceptions
- it governs protected-action handling
- it does not itself execute stages
- it does not itself persist canonical run state

- it does not itself define artifact integrity truth

---

7. Governing Model

The GCP follows this model:

requested action → governance classification → required checks → governance result

That result may be:

- allowed
- denied
- approval required
- blocked pending evidence
- blocked pending override
- blocked pending quota availability
- expired or invalid

The GCP is the plane that transforms raw action intent into governed permission outcome.

---

8. Primary Responsibilities

The GCP is responsible for:

8.1 Policy governance

- load policies
- determine applicable policies
- evaluate action requests against policies

- preserve denial reasons and required evidence

8.2 Approval governance

- detect when approval is required
- create approval requests
- route requests to valid approvers
- preserve approval status
- enforce escalation and expiry where defined

8.3 Override governance

- handle policy exception requests
- require evidence and justification
- enforce time-bounded override behavior
- preserve override status and expiry

8.4 Permission governance

- evaluate actor permission to perform protected actions
- enforce role-based or rule-based action authority
- enforce separation of duties

8.5 Quota governance

- evaluate resource governance constraints
- meter or consume quota signals where required
- deny, throttle, or defer governed actions based on allowed quota behavior

---

9. GCP Role Set

The GCP is composed of five roles:

GCP-01 Policy Engine

Owns policy evaluation and policy-result generation.

GCP-02 Approval Router

Owns approval request creation, routing, and decision linkage.

GCP-03 Override Manager

Owns controlled exception handling.

GCP-04 Permissions / AuthZ Manager

Owns actor authorization and protected-action access control.

GCP-05 Quota Manager

Owns usage governance and limit enforcement signals.

---

10. Inputs

The GCP consumes the following input classes:

10.1 Action requests

- run lifecycle action request
- stage rerun request
- release action request
- artifact action request

- administrative action request
- queue reprioritization request
- override request
- approval decision request

10.2 Governance context

- actor identity
- actor role
- workspace/project scope
- risk class
- current state of governed object
- requested change

10.3 Supporting signals

- policy definitions
- approval maps
- quota state
- override records
- prior governance decisions
- evidence references where required

---

11. Outputs

The GCP produces:

11.1 Governance decisions

- allowed
- denied
- approval required
- authorization denied
- quota denied
- override active/inactive
- blocked pending evidence

11.2 Governance records

- policy evaluation result
- approval request
- approval decision
- override record
- authz decision
- quota enforcement result

11.3 Governance events

- approval requested
- approval expired
- override activated
- override denied
- permission denied

- quota exceeded
- policy denied

---

12. Dependencies

The GCP depends on other system layers but remains governance-authoritative within its scope.

12.1 Execution dependencies

The GCP may consume:

- current run state
- stage state
- requested transition context
- queue context
- environment routing context

These are inputs to governance decisions, not replacements for execution roles.

12.2 Evidence dependencies

The GCP depends on the Evidence Control Plane for:

- audit logging of governance decisions
- preserved evidence references
- release and artifact evidence where relevant
- notification delivery for approval and denial events

---

13. Authority Boundaries

The GCP has authority to:

- allow or deny governed actions
- require approval
- require evidence
- require override
- reject unauthorized actors
- enforce quota constraints
- expire invalid governance permissions

The GCP does **not** have authority to:

- directly execute the underlying action by itself
- mutate canonical run state outside governed service paths
- rewrite audit truth
- fabricate evidence
- redefine execution-state legality
- claim environment readiness or capability truth

---

14. Protected Actions

The GCP must explicitly govern protected actions.

Examples may include:

- rerunning privileged stages

- publishing or revoking releases
- applying overrides
- modifying quota or permission-sensitive settings
- unpinning protected artifacts
- dispatching work into restricted environments
- reprioritizing privileged queue items
- resuming a held/gated run under approval dependency

The exact list may evolve, but the GCP must define what counts as protected and how those actions are governed.

---

15. AI Assistance Boundary

The GCP may be AI-assisted, but AI is not the GCP.

AI may help with:

- summarizing policy conflicts
- drafting readable denial reasons
- identifying likely approver paths
- summarizing override risk
- detecting suspicious permission patterns
- forecasting quota pressure
- preparing governance review packets

AI may not:

- approve protected actions by itself

- become policy truth
- grant permissions directly
- activate overrides directly
- overrule deterministic governance results
- bypass quota rules

The GCP remains the authoritative governance layer.

---

16. Operating Modes

The GCP should support these governance modes:

16.1 Deterministic-only mode

All governance decisions occur without AI assistance.

16.2 AI-assisted advisory mode

AI summarizes or recommends, but governance outcomes are determined by formal rules and authorized humans where applicable.

16.3 AI-assisted validate-first mode

AI proposes structured governance recommendations that must still pass deterministic governance checks before any result is used.

16.4 Restricted low-risk automation mode

Only explicitly whitelisted low-risk governance support actions may auto-apply, and even then not in ways that grant protected authority beyond allowed rules.

---

17. Failure Handling Expectations

The GCP must fail safely.

17.1 Missing policy truth

Fail closed or block governed action until explicit policy state exists.

17.2 Missing authorization context

Reject or defer action until actor identity and permission context are available.

17.3 Missing approval result

Block protected action until approval status is satisfied.

17.4 Missing quota signal

Do not assume permission to consume resource-governed actions.

17.5 AI failure

Ignore AI assistance and continue deterministic governance behavior.

---

18. Required Guarantees

The GCP must guarantee:

- governed actions are evaluated explicitly
- protected actions cannot bypass governance checks
- approvals are first-class governed objects
- overrides are controlled and auditable
- actor authority is explicit
- quota restrictions are enforceable
- governance behavior remains safe without AI

---

19. Success Criteria

The GCP is correctly established when:

- policy, approval, override, authz, and quota responsibilities are centralized into first-class roles
- allowed/denied decisions are inspectable
- protected actions are clearly governed
- approval requirements are explicit
- override handling is controlled and time-bounded
- permissions are enforced explicitly
- AI assistance remains bounded and non-authoritative

---

20. Relationship to Other Planes

GCP vs ICP

- GCP governs **whether the action is allowed**
- ICP governs **how execution movement occurs**

GCP vs ECP

- GCP governs **permission and exception logic**
- ECP governs **evidence, integrity, and proof-bearing records**

GCP vs AIG

- GCP governs **governance authority**

- AIG governs **how AI may assist governance authority**

---

21. Summary Statement

The Governance Control Plane is the Axion governance authority layer.

It governs policy, approvals, overrides, authorization, and quota enforcement through deterministic rules and explicit decision paths.

It may be assisted by AI, but it does not surrender governance authority to AI.

GCP-01 — Policy Engine.

GCP-01 — Policy Engine

1. Purpose

The Policy Engine is the role responsible for **evaluating governed actions against explicit policy** inside the Governance Control Plane.

Its purpose is to determine whether a requested action is:

- allowed
- denied
- conditionally allowed
- blocked pending evidence
- blocked pending approval
- blocked pending override
- blocked pending quota or other governance dependency

The Policy Engine governs **policy truth application**.

It is the role that decides how written governance rules apply to a specific action context.

---

2. Objective

The objective of the Policy Engine is to ensure that:

- policies are explicit and loadable
- applicable policies can be identified for a given action
- policy results are deterministic

- denial reasons are inspectable
- required evidence can be surfaced
- policy precedence and conflict behavior are defined
- downstream roles can consume policy results safely
- AI may assist explanation and recommendation, but not replace policy truth

---

3. Problem It Solves

Without a dedicated policy role, policy logic tends to become fragmented across:

- controllers
- UI conditions
- operator conventions
- ad hoc checks
- hidden config flags
- inconsistent exception handling

That fragmentation causes:

- unclear allow/deny rules
- policy drift
- inconsistent enforcement
- hidden policy conflicts
- weak trust in why an action was permitted or denied

The Policy Engine solves this by making policy evaluation explicit and centralized.

4. Scope

The Policy Engine governs:

- policy loading
- policy applicability determination
- policy evaluation
- policy precedence handling
- policy result generation
- denial reason generation
- required evidence declaration
- policy snapshot preservation where required
- policy-result linkage to downstream governed actions

The Policy Engine does not govern:

- human approval decisions
- override activation decisions
- actor authorization by itself
- canonical run persistence
- audit storage
- release artifact truth
- environment capability truth

Those are handled by other roles.

---

5. Core Principle

The Policy Engine is the **policy-application authority**, not the entire governance plane.

It applies explicit policy to action context and returns a deterministic policy result.

This means:

- it evaluates rules
- it explains rule outcomes
- it does not itself execute the action
- it does not itself approve overrides
- it does not itself grant permissions
- it does not replace human approval when required

It is the formal rule-evaluation layer for governance.

---

6. Primary Responsibilities

6.1 Load policies

- retrieve active policy definitions
- preserve policy identity and version
- support scoped policy selection

6.2 Determine applicability

- identify which policies apply to a requested action
- filter by action class, scope, risk class, object type, or environment context as defined

6.3 Evaluate policies

- evaluate single policy
- evaluate multiple applicable policies
- produce deterministic allow/deny/conditional outcomes

6.4 Resolve policy interactions

- enforce precedence
- handle conflicting policy outcomes
- fail closed where required if conflict is unresolved

6.5 Produce usable policy results

- return structured result
- preserve denial reasons
- preserve evidence requirements
- preserve blocking conditions
- expose readable explanation data

---

7. Authority

The Policy Engine has authority to:

- determine which policies apply
- evaluate explicit policy conditions
- return policy pass/fail/conditional outcomes

- require evidence or approval when policy dictates
- block action when policy denies it

The Policy Engine does not have authority to:

- rewrite policy outside governed update paths
- approve protected actions by itself where approval is separately required
- grant permissions directly
- activate overrides directly
- mutate canonical run state
- fabricate evidence
- ignore active policy

It decides policy outcome, not all governance behavior.

---

8. Inputs

The Policy Engine consumes:

8.1 Action inputs

- requested action
- action type
- object type
- target object ID
- requested change set where applicable

8.2 Governance context inputs

- actor identity and role where relevant
- workspace/project scope
- risk class
- environment class
- current object state
- release/run/stage context where applicable

8.3 Policy inputs

- active policy definitions
- policy versions
- precedence metadata
- scope metadata
- exception/override metadata where applicable

8.4 Supporting evidence inputs

- supplied evidence refs
- approval state if policy depends on it
- override state if policy depends on it
- quota state where policy consumes quota signals

---

9. Outputs

The Policy Engine produces:

9.1 Policy decision result

- allowed
- denied
- conditionally allowed
- approval required
- evidence required
- override required
- quota-blocked, if modeled through policy

9.2 Policy explanation data

- applied policy IDs
- denial reasons
- blocking predicates
- readable explanation summary

9.3 Policy evidence data

- required evidence set
- missing evidence set
- policy snapshot references where needed

---

10. Primary Interfaces

The Policy Engine should expose, at minimum:

10.1 `loadPolicies`

Load active policy definitions for a relevant scope.

10.2 `getApplicablePolicies`

Return policies applicable to a given action context.

10.3 `evaluatePolicy`

Evaluate a single policy against context.

10.4 `evaluatePolicies`

Evaluate all applicable policies and return a combined result.

10.5 `explainPolicyResult`

Return readable explanation for a structured policy result.

10.6 `getPolicySnapshot`

Return or reference the policy snapshot used for a decision where required.

---

11. Policy Model

The Policy Engine must require an explicit policy model.

At minimum, policies should have:

- stable policy ID
- policy version
- policy scope
- policy action/object applicability
- predicate or condition set
- effect type
- precedence metadata

- evidence requirement metadata where applicable

The exact syntax may vary, but policy shape must be explicit and structured.

---

12. Applicability Rules

The Policy Engine must define how a policy becomes applicable.

Applicability may depend on:

- action type
- object type
- workspace/project scope
- risk class
- environment type
- actor role
- lifecycle state
- release stage or run stage
- protected-action classification

Policies must not be applied through hidden guesswork.

---

13. Evaluation Rules

The Policy Engine must define how policy evaluation works.

At minimum it must define:

- what constitutes pass

- what constitutes fail
- what constitutes conditional pass
- when evidence is required
- when approval is required
- when override is required
- how missing data is treated
- whether unknown evaluation conditions fail closed

Policy evaluation must be deterministic.

---

14. Precedence and Conflict Rules

The Policy Engine must define explicit conflict handling.

Examples of questions it must answer:

- do deny rules override allow rules?
- do narrower-scope policies override broader ones?
- how are version conflicts handled?
- what happens when multiple policies produce incompatible conditions?
- what happens when required context is missing?

If conflict cannot be resolved deterministically, the system should fail closed or block pending resolution.

---

15. Evidence Requirement Rules

Policies may require evidence before an action is allowed.

The Policy Engine must define:

- how evidence requirements are declared
- how missing evidence affects outcome
- whether evidence can satisfy conditional allow
- whether approval or override depends on evidence completeness
- how evidence requirements are surfaced downstream

The Policy Engine does not store evidence truth, but it must declare evidence needs clearly.

---

16. Conditional Outcomes

Not all policy results are simple allow/deny.

The Policy Engine must support conditional governance outcomes such as:

- allowed if approval is obtained
- allowed if override is active
- allowed if evidence is provided
- denied unless quota becomes available
- denied while object is in forbidden state

Conditional outcomes must be explicit and machine-usable.

---

17. Relationship to Approval Router

The Policy Engine determines whether policy requires approval.

The Approval Router handles the approval object and decision path.

In simple terms:

- Policy Engine = does policy require approval?
- Approval Router = route and resolve the approval

The Policy Engine must not pretend to replace approval workflow.

---

18. Relationship to Override Manager

The Policy Engine determines whether override is required or whether an active override changes policy outcome.

The Override Manager governs actual override objects and lifecycle.

In simple terms:

- Policy Engine = policy says override needed or active override applies
- Override Manager = governs override existence and validity

---

19. Relationship to Permissions / AuthZ Manager

Policy and authorization are related but distinct.

A policy may allow an action in abstract terms while the actor still lacks permission.

In simple terms:

- Policy Engine = is the action allowed under policy?
- AuthZ Manager = is this actor allowed to do it?

Both may need to pass before execution proceeds.

---

20. Relationship to Quota Manager

The Policy Engine may consume quota state as an input, or policy may define that certain actions require quota availability.

The Quota Manager governs quota truth and usage enforcement state.

The Policy Engine must not fabricate quota status.

---

21. Relationship to Evidence Control Plane

The Policy Engine may require:

- audit linkage for policy decisions
- evidence references for decision context
- notification for policy denial events

But it does not own evidence storage or audit mechanics.

---

22. Audit Requirements

The Policy Engine must ensure auditable coverage for important policy events.

At minimum, audit-relevant events include:

- policy evaluation
- policy denial
- policy-required approval condition
- policy-required evidence condition
- policy-required override condition
- policy conflict or fail-closed event
- policy version used for critical decision

The audit trail should preserve:

- actor or system source
- action being evaluated
- applied policy IDs/versions
- policy outcome
- denial or blocking reasons
- evidence requirement metadata
- AI participation metadata where applicable

---

23. AI Assistance Boundary

The Policy Engine may be AI-assisted in bounded ways.

AI may help with:

- summarizing policy conflicts
- producing readable policy explanations
- drafting operator-friendly denial summaries
- suggesting which policies are likely applicable before deterministic evaluation
- clustering recurring denial patterns

AI may not:

- define policy truth
- invent policy predicates
- overrule policy denial

- reinterpret forbidden actions as allowed
- activate override as a substitute for policy result

The Policy Engine remains the deterministic source of policy outcome.

---

24. Failure Handling

The Policy Engine must fail safely.

24.1 Missing policy definition

Fail closed or block until policy truth exists.

24.2 Ambiguous applicability

Fail closed or return explicit unresolved-policy state.

24.3 Conflicting policies without precedence resolution

Return conflict-blocked or fail-closed result.

24.4 Missing required evaluation context

Do not guess; return blocked or denied outcome according to explicit rule.

24.5 AI explanation failure

Ignore AI assistance and preserve deterministic policy result.

---

25. Required Guarantees

The Policy Engine must guarantee:

- policy applicability is explicit
- policy results are deterministic

- denial reasons are inspectable
- conditional outcomes are structured
- evidence requirements are surfaced clearly
- policy conflicts are handled explicitly
- policy evaluation remains safe without AI

---

26. Success Criteria

The Policy Engine is correctly established when:

- policies are first-class structured system objects
- applicable policies can be identified explicitly
- policy evaluation is centralized and deterministic
- denial reasons and blocking conditions are visible
- approval/evidence/override requirements can be produced structurally
- policy conflicts do not result in silent ambiguous allowance
- AI remains advisory and non-authoritative for policy truth

---

27. Summary Statement

The Policy Engine is the policy-application role within the Governance Control Plane.

It evaluates explicit policy against governed action context, returns deterministic policy outcomes, and surfaces denial reasons, conditions, and evidence requirements for downstream governance and execution roles.

It may use AI for explanation and recommendation support, but it does not delegate policy truth to AI.

GCP-02 — Approval Router

GCP-02 — Approval Router

1. Purpose

The Approval Router is the role responsible for **creating, routing, tracking, and resolving approval-required governance decisions** inside the Governance Control Plane.

Its purpose is to ensure that actions requiring human or delegated authority approval are:

- detected
- represented as explicit approval objects
- routed to the correct approvers
- tracked through a governed lifecycle
- linked back to the action they authorize or deny

The Approval Router governs **approval flow**.

It is the role that turns “this needs approval” into an explicit, auditable decision path.

---

2. Objective

The objective of the Approval Router is to ensure that:

- approval-required actions are represented explicitly
- valid approvers are selected through defined rules
- approval states are visible and durable
- approval decisions are linked to governed actions
- approvals can expire, escalate, or be rejected through explicit rules

- separation-of-duties expectations are enforceable
- AI may assist summarization and routing support, but may not become approval authority

---

3. Problem It Solves

Without a dedicated approval role, approvals become fragmented across:

- informal operator messages
- UI-only flags
- direct controller exceptions
- manual signoff conventions
- hidden admin actions
- undocumented reviewer expectations

That fragmentation causes:

- unclear who must approve what
- inconsistent approval trails
- weak protected-action controls
- poor expiry/escalation behavior
- weak linkage between approval and resulting action

The Approval Router solves this by making approval flow explicit.

---

4. Scope

The Approval Router governs:

- approval request creation
- approval request identity
- approver resolution
- approval assignment
- approval state tracking
- approval decision capture
- approval expiry handling
- approval escalation handling
- linkage of approval result to downstream action eligibility
- approval visibility status

The Approval Router does not govern:

- whether policy requires approval in the first place
- actor permission truth
- override object lifecycle
- canonical run persistence
- audit storage mechanics
- notification delivery mechanics
- release evidence truth

Those are handled by adjacent roles.

---

5. Core Principle

The Approval Router is the **approval workflow authority**, not the ultimate source of policy truth and not the executor of the protected action.

It governs how approval-required actions move through a decision path.

This means:

- it creates approval objects
- it routes them
- it captures decisions
- it does not invent approval requirements outside governance rules
- it does not itself execute the protected action
- it does not replace authorization or policy evaluation

It is the formal approval-flow layer for governed actions.

---

6. Primary Responsibilities

6.1 Create approval requests

- create stable approval IDs
- bind approval request to target action
- preserve approval type and scope
- preserve required approval metadata

6.2 Resolve approvers

- determine valid approver candidates

- honor scope, role, ownership, and separation-of-duties rules
- reject invalid approver selection paths

6.3 Route approvals

- assign or publish approval requests to appropriate approvers
- preserve route state
- support multi-step or multi-actor approval flows where defined

6.4 Track approval lifecycle

- preserve pending state
- preserve approved/rejected state
- preserve expired/escalated/cancelled state
- preserve timestamps and decision metadata

6.5 Link decisions to governed actions

- mark when an action is now authorized by approval
- mark when action remains blocked
- preserve decision reference for downstream enforcement

6.6 Handle expiry and escalation

- expire stale approvals
- escalate according to defined rules
- preserve escalation history

---

7. Authority

The Approval Router has authority to:

- create approval objects
- determine valid approval routes
- assign approval requests
- accept governed approval or rejection decisions from authorized approvers
- expire or escalate approval requests according to explicit rules
- publish approval status for downstream use

The Approval Router does not have authority to:

- decide that approval is required without upstream governance basis
- grant permission outside valid approver authority
- bypass policy, authz, or override checks
- directly execute protected actions
- fabricate approval decisions
- rewrite audit truth

It governs approval flow, not all governance law.

---

8. Inputs

The Approval Router consumes:

8.1 Approval-triggering inputs

- policy result indicating approval required

- protected-action classification
- override request requiring approval
- release action requiring approval
- privileged administrative request

8.2 Request context

- target action
- target object
- object scope
- actor requesting action
- risk class
- reason/justification
- evidence refs where applicable

8.3 Approver context

- approver mapping rules
- role definitions
- scope/ownership mapping
- separation-of-duties constraints
- escalation rules

8.4 Existing approval state

- current pending approvals

- prior decisions
- expired approvals
- superseded approvals

---

9. Outputs

The Approval Router produces:

9.1 Approval objects

- approval request
- assignment record
- escalation record
- decision record

9.2 Approval decisions

- pending
- approved
- rejected
- expired
- cancelled
- escalated

9.3 Downstream governance outputs

- approval satisfied

- approval unsatisfied
- approval invalid
- approval expired
- approval blocked by missing approver

---

10. Primary Interfaces

The Approval Router should expose, at minimum:

10.1 `createApprovalRequest`

Create a new approval request for a governed action.

10.2 `resolveApprovers`

Return valid approvers for an approval request.

10.3 `assignApproval`

Assign approval request to one or more approvers.

10.4 `approveRequest`

Record approval decision from an authorized approver.

10.5 `rejectRequest`

Record rejection decision from an authorized approver.

10.6 `expireRequest`

Expire a pending approval request according to explicit rules.

10.7 `escalateRequest`

Escalate a pending approval request according to explicit rules.

10.8 `getApprovalStatus`

Return current approval lifecycle state.

10.9 `listPendingApprovals`

Return pending approvals by scope, actor, or queue.

---

11. Approval Object Model

The Approval Router must require an explicit approval object model.

At minimum, an approval request should include:

- approval_id
- target action reference
- target object reference
- requesting actor
- required approval type
- scope
- approver requirements
- current state
- created time
- expiry time where applicable
- justification
- evidence refs where applicable
- route history
- decision history

The exact schema may vary, but approval must be a first-class governed object.

---

12. Approver Resolution Rules

The Approval Router must define how valid approvers are determined.

Approver selection may depend on:

- workspace/project scope
- object ownership
- risk level
- action type
- actor role
- escalation hierarchy
- separation-of-duties rules
- minimum approver count where required

Approver resolution must not rely on hidden informal convention.

---

13. Decision Rules

The Approval Router must define what constitutes a valid approval decision.

At minimum, it must define:

- who may approve
- who may reject
- whether multiple approvals are required

- whether approval order matters
- what invalidates an approval
- whether approval may be withdrawn
- how superseded approval requests are handled

Approval decisions must be machine-usable and inspectable.

---

14. Expiry Rules

The Approval Router must define approval expiry behavior explicitly.

It must answer:

- when an approval request expires
- whether expiry is absolute or relative
- whether expiry resets on escalation
- whether expired approvals can be reused
- whether action context changes invalidate an otherwise valid approval

A stale approval must not silently remain valid forever.

---

15. Escalation Rules

The Approval Router must define when escalation occurs.

Examples:

- pending beyond threshold
- missing required approver response

- high-risk request not resolved in time
- escalation due to unavailable approver

It must also define:

- who receives escalation
- whether escalation changes approver authority
- whether prior decisions remain valid
- how escalation history is recorded

---

16. Separation-of-Duties Rules

The Approval Router must support separation-of-duties enforcement where required.

Examples:

- requester cannot self-approve
- override requester cannot be sole override approver
- release creator cannot be sole release approver
- actor performing privileged action may require independent reviewer

The Approval Router may depend on AuthZ/role truth, but must enforce the approval-side rule.

---

17. Relationship to Policy Engine

The Policy Engine determines whether approval is required.

The Approval Router governs the approval object and decision path.

In simple terms:

- Policy Engine = approval required?
- Approval Router = who approves, how, and what state is the approval in?

The Approval Router must not replace policy truth.

---

18. Relationship to Override Manager

Some approvals may authorize an override request, but the override lifecycle is governed by the Override Manager.

In simple terms:

- Approval Router = route and resolve approval
- Override Manager = govern override object lifecycle and validity

Approval alone is not necessarily override activation until override rules are satisfied.

---

19. Relationship to Permissions / AuthZ Manager

The Approval Router depends on authorization truth to validate:

- who is an allowed approver
- who may act on an approval request
- whether requester and approver separation is satisfied

In simple terms:

- AuthZ Manager = may this actor approve?
- Approval Router = apply that to approval flow

---

20. Relationship to Notification Router

The Approval Router may create events such as:

- approval requested
- approval expiring
- approval escalated
- approval rejected
- approval granted

The Notification Router handles delivery, but the Approval Router must produce the governed event intent.

---

21. Relationship to Audit Logger

Approval decisions are governance-critical and must be audit-linked.

The Approval Router must ensure audit coverage for:

- request creation
- assignment
- approval
- rejection
- expiry
- escalation
- cancellation/supersession

The audit trail should preserve:

- actor
- action
- target approval
- target governed object
- decision state
- route metadata
- reason metadata
- AI participation metadata where applicable

---

22. AI Assistance Boundary

The Approval Router may be AI-assisted in bounded ways.

AI may help with:

- summarizing request context for approvers
- drafting approval packet summaries
- recommending likely approver route for deterministic validation
- flagging incomplete evidence
- prioritizing pending approval queues for operator review

AI may not:

- approve protected actions by itself
- reject requests authoritatively by itself
- invent approver authority

- bypass separation-of-duties rules
- convert incomplete approval into valid approval

The Approval Router remains the authoritative approval-flow role.

---

23. Failure Handling

The Approval Router must fail safely.

23.1 No valid approver found

Block approval flow and preserve explicit unresolved-approver reason.

23.2 Invalid approver action

Reject approval decision and preserve reason.

23.3 Expired approval used

Reject downstream approval satisfaction and require new approval flow.

23.4 Superseded request reused

Reject stale approval reference.

23.5 Missing approval context

Do not guess; preserve blocked state.

23.6 AI assistance failure

Ignore AI assistance and preserve deterministic approval behavior.

---

24. Required Guarantees

The Approval Router must guarantee:

- approval-required actions become explicit approval objects
- approver resolution is rule-based
- approval lifecycle state is durable and inspectable
- stale or invalid approvals do not silently authorize actions
- separation-of-duties rules can be enforced
- approval flow remains safe without AI

---

25. Success Criteria

The Approval Router is correctly established when:

- approval is a first-class governed object
- protected actions requiring approval cannot proceed without valid approval state
- approvers are resolved explicitly
- approval expiry and escalation are explicit
- approval decisions are linked to governed actions
- operators can inspect pending and resolved approvals clearly
- AI remains advisory and non-authoritative for approval truth

---

26. Summary Statement

The Approval Router is the approval-flow role within the Governance Control Plane.

It creates, routes, tracks, and resolves approval-required governance decisions through explicit, durable, and auditable approval objects and decision paths.

It may use AI for summarization and routing support, but it does not delegate approval authority to AI.

GCP-03 — Override Manager

GCP-03 — Override Manager

1. Purpose

The Override Manager is the role responsible for **governing controlled exceptions to normal policy outcomes** inside the Governance Control Plane.

Its purpose is to ensure that when a policy-denied or policy-constrained action needs to proceed under exceptional circumstances, the exception is handled through a **formal, bounded, auditable override path** rather than informal bypass.

The Override Manager governs **exception authority**.

It is the role that turns “this cannot proceed normally” into a governed exception workflow.

---

2. Objective

The objective of the Override Manager is to ensure that:

- overrides are explicit first-class governed objects
- overrides are only used when eligible
- overrides require justification and evidence
- override activation follows approval and authorization rules where required
- overrides are time-bounded and scope-bounded
- override usage is auditable and inspectable
- expired or invalid overrides cannot silently authorize actions
- AI may assist risk review and summarization, but may not become override authority

---

3. Problem It Solves

Without a dedicated override role, exceptions tend to become fragmented across:

- manual admin intervention
- hidden flags
- informal approval messages
- emergency scripts
- undocumented operator conventions
- direct code path bypasses

That fragmentation causes:

- uncontrolled policy bypass
- unclear exception scope
- override reuse beyond intended context
- weak expiry handling
- poor auditability of why an exception existed

The Override Manager solves this by making exceptions governed and explicit.

---

4. Scope

The Override Manager governs:

- override request creation
- override eligibility evaluation
- override object identity
- override scope definition

- override justification requirements
- override evidence requirements
- override approval linkage
- override activation rules
- override expiry rules
- override revocation rules
- override consumption/usage tracking
- override status visibility

The Override Manager does not govern:

- the base policy result itself
- approval routing mechanics
- actor permission truth
- canonical run persistence
- audit storage mechanics
- notification delivery mechanics

Those are handled by adjacent roles.

---

5. Core Principle

The Override Manager is the **exception-governance authority**, not a general bypass switch.

An override is not an informal permission to ignore the system. It is a tightly scoped, justified, time-bounded governance object that modifies whether a blocked action may proceed under explicit exception rules.

This means:

- overrides must be explicit
- overrides must be bounded
- overrides must be reviewable
- overrides must expire
- overrides must not become a hidden permanent alternate policy path

---

6. Primary Responsibilities

6.1 Create override requests

- create stable override IDs
- bind override to target action or target class of actions
- preserve override type and scope
- preserve requester identity

6.2 Validate override eligibility

- determine whether the requested exception is override-eligible
- reject non-overridable actions
- reject invalid exception classes

6.3 Require justification and evidence

- require explicit reason for exception
- require evidence where applicable
- preserve evidence linkage

- reject insufficient override basis where rules require it

6.4 Coordinate approval requirements

- require approval path where policy or risk class requires it
- refuse activation until approval is satisfied

6.5 Activate and enforce override scope

- mark override active when all requirements are met
- enforce scope limits
- expose when an override may be consumed downstream

6.6 Expire and revoke overrides

- expire overrides automatically by rule
- revoke invalid overrides
- invalidate superseded or stale exceptions
- prevent post-expiry use

6.7 Preserve override history

- track lifecycle state
- track activations
- track use events
- track expiry/revocation events

---

7. Authority

The Override Manager has authority to:

- create override objects
- determine whether override is eligible
- activate override only when all required conditions are met
- expire or revoke override objects
- declare whether an override is currently valid for downstream use

The Override Manager does not have authority to:

- invent override eligibility outside defined rules
- replace policy evaluation
- replace approval routing
- grant actor permissions directly
- directly execute protected actions
- rewrite audit truth
- silently convert denied actions into permanently allowed actions

It governs exception objects, not all governance logic.

---

8. Inputs

The Override Manager consumes:

8.1 Override request inputs

- requested action
- target object

- requester identity
- requested scope
- requested duration
- override reason
- requested exception type

8.2 Governance inputs

- policy result
- protected-action classification
- risk class
- approval requirement
- authorization requirement
- quota or environment constraints where relevant

8.3 Evidence inputs

- justification text
- supporting evidence refs
- required evidence completion state
- prior related decisions

8.4 Approval/authz inputs

- approval state
- approver decision linkage

- requester/approver roles
- permission validation where needed

---

9. Outputs

The Override Manager produces:

9.1 Override objects

- override request
- active override record
- revoked override record
- expired override record

9.2 Override status decisions

- eligible
- ineligible
- pending approval
- pending evidence
- active
- denied
- expired
- revoked

9.3 Downstream governance signals

- override valid for action
- override invalid for action
- override scope exceeded
- override expired
- override cannot satisfy requested exception

---

10. Primary Interfaces

The Override Manager should expose, at minimum:

10.1 `createOverrideRequest`

Create a new override request.

10.2 `validateOverrideEligibility`

Return whether the requested exception is override-eligible.

10.3 `activateOverride`

Activate override when all requirements are satisfied.

10.4 `revokeOverride`

Revoke an active or pending override under valid conditions.

10.5 `expireOverride`

Expire override according to explicit time or rule conditions.

10.6 `getOverrideStatus`

Return the current lifecycle status of an override.

10.7 `listOverrides`

Return overrides by scope, state, or target object.

10.8 `validateOverrideUse`

Return whether an active override can satisfy a specific downstream action request.

---

11. Override Object Model

The Override Manager must require an explicit override object model.

At minimum, an override record should include:

- override_id
- target action or action class
- target object or scope
- requester
- reason/justification
- evidence refs
- risk class
- approval requirement state
- current status
- created_at
- activated_at
- expires_at
- revoked_at
- allowed use scope
- usage history

- denial/revocation/expiry reason where applicable

Overrides must be first-class governed records, not hidden flags.

---

12. Eligibility Rules

The Override Manager must define explicit eligibility rules.

It must answer:

- which action classes are overridable
- which are never overridable
- which scopes may be overridden
- which risk classes require stricter controls
- whether an override may apply to a single action, a bounded action set, or a narrow time window only

Examples of likely non-overridable classes may include:

- illegal state transitions
- unauthorized actor actions without valid authz path
- evidence truth fabrication
- audit rewrite attempts

The exact list is system-defined, but eligibility must be explicit.

---

13. Scope Rules

Overrides must be scope-bounded.

The Override Manager must define scope dimensions such as:

- single action
- single target object
- single run
- single release
- narrow admin window
- time-limited environment exception

Overrides must not default to broad indefinite authority.

The role must also define:

- whether scope may be reused
- whether scope may cover multiple actions
- whether scope shrinks after first use
- what counts as scope violation

---

14. Justification and Evidence Rules

Override requests must require explicit justification.

The Override Manager must define:

- minimum justification requirements
- evidence classes required by override type
- evidence sufficiency expectations
- whether missing evidence blocks activation
- whether weak evidence requires stronger approval path

An override without reason should be invalid by default.

---

15. Approval Rules

Most meaningful overrides should require approval.

The Override Manager must define:

- when approval is mandatory
- whether multiple approvers are required for some override classes
- whether approval must precede activation
- whether approval expires alongside override validity
- whether change in override scope invalidates prior approval

The Approval Router manages the approval object, but the Override Manager must enforce the dependency.

---

16. Activation Rules

The Override Manager must define what makes an override active.

At minimum, activation should depend on:

- eligibility satisfied
- required justification present
- required evidence present
- required approval satisfied
- requester authorized to request override
- override not expired or revoked

An override must not become active simply because it was requested.

---

17. Expiry Rules

Overrides must expire explicitly.

The Override Manager must define:

- default duration rules
- maximum duration rules
- absolute vs relative expiry
- whether unused overrides expire automatically
- whether consumed overrides remain reusable
- what happens when underlying context changes before expiry

An expired override must not remain silently valid.

---

18. Revocation Rules

The Override Manager must define revocation behavior.

Revocation may occur because:

- new policy state invalidates the exception
- approval is withdrawn or invalidated
- misuse is detected
- scope was exceeded
- operator/admin revocation is required by rule

- evidence is later found insufficient or false

The role must define:

- who may revoke
- what revocation does to downstream action eligibility
- whether past approved uses remain valid as historical fact
- how revocation is surfaced and audited

---

19. Usage Rules

Overrides should track usage.

The Override Manager must define:

- whether an override may be used once or multiple times
- whether use consumes the override
- how each use is recorded
- whether use outside declared scope is invalid
- whether partially consumed overrides remain active

Overrides should not become invisible reusable permissions.

---

20. Relationship to Policy Engine

The Policy Engine determines whether:

- override is required

- active override changes policy outcome
- action is denied without valid override

The Override Manager governs the existence, validity, scope, and lifecycle of the override itself.

In simple terms:

- Policy Engine = policy says override needed or active override may satisfy condition
- Override Manager = is there a valid override, and what does it cover?

---

21. Relationship to Approval Router

The Approval Router handles approval flow for overrides that require signoff.

The Override Manager must not activate override until required approval is satisfied.

In simple terms:

- Approval Router = approval object and decision
- Override Manager = exception object and validity

---

22. Relationship to Permissions / AuthZ Manager

The Override Manager depends on authz truth for:

- whether actor may request override
- whether actor may revoke override
- whether approval actors are valid
- whether separation-of-duties expectations are satisfied

An override must not substitute for base authorization unless the system explicitly defines that narrow behavior.

---

23. Relationship to Audit Logger

Override activity is high-sensitivity governance behavior and must be audit-linked.

The Override Manager must ensure audit coverage for:

- override request creation
- eligibility denial
- activation
- revocation
- expiry
- usage
- scope violation
- override-based protected action satisfaction

The audit trail should preserve:

- actor
- action
- target override
- related governed object
- scope
- expiry
- reason

- evidence refs
- approval linkage
- AI participation metadata where applicable

---

24. AI Assistance Boundary

The Override Manager may be AI-assisted in bounded ways.

AI may help with:

- summarizing override risk
- drafting reviewer context
- clustering similar override cases
- flagging weak justification or missing evidence
- highlighting possible scope overbreadth for review

AI may not:

- activate override by itself
- redefine override eligibility
- expand override scope authoritatively
- convert insufficient evidence into valid evidence
- overrule deterministic override denial

The Override Manager remains the authoritative exception-governance role.

---

25. Failure Handling

The Override Manager must fail safely.

25.1 Ineligible override request

Reject request and preserve explicit denial reason.

25.2 Missing approval

Do not activate override; preserve pending-approval state.

25.3 Missing evidence

Do not activate override where evidence is required.

25.4 Expired override used

Reject downstream use and require new governed path.

25.5 Scope mismatch

Reject use and preserve explicit scope-violation result.

25.6 AI assistance failure

Ignore AI assistance and preserve deterministic override behavior.

---

26. Required Guarantees

The Override Manager must guarantee:

- overrides are explicit first-class objects
- overrides cannot activate without required conditions
- override scope is bounded and inspectable
- override expiry is enforced
- override misuse can be detected and rejected
- expired or revoked overrides cannot silently authorize future actions

- override behavior remains safe without AI

---

27. Success Criteria

The Override Manager is correctly established when:

- policy exceptions are handled through formal override objects
- override eligibility is explicit
- justification and evidence requirements are explicit
- approval dependencies are explicit
- override activation, expiry, revocation, and usage are all inspectable
- protected actions cannot rely on stale or out-of-scope overrides
- AI remains advisory and non-authoritative for override truth

---

28. Summary Statement

The Override Manager is the exception-governance role within the Governance Control Plane.

It creates, validates, activates, scopes, expires, revokes, and tracks overrides as bounded, auditable governance objects for controlled policy exceptions.

It may use AI for risk review and summarization support, but it does not delegate override authority to AI.

GCP-04 — Permissions / AuthZ Manager

GCP-04 — Permissions / AuthZ Manager

1. Purpose

The Permissions / AuthZ Manager is the role responsible for **determining whether an actor is authorized to perform a governed action** inside the Governance Control Plane.

Its purpose is to enforce explicit action authority over protected system operations by evaluating:

- actor identity
- actor role or permission set
- action type
- scope
- target object
- separation-of-duties constraints
- privileged-action rules

The Permissions / AuthZ Manager governs **actor authority**.

It is the role that decides whether a specific actor may perform a specific action in a specific context.

---

2. Objective

The objective of the Permissions / AuthZ Manager is to ensure that:

- actors are evaluated through explicit authorization rules
- privileged actions are protected

- role or permission-based access is deterministic
- scope-sensitive authorization is enforceable
- separation-of-duties rules are respected
- authorization denial reasons are inspectable
- downstream roles can rely on authorization results safely
- AI may assist with explanation and anomaly detection, but may not become permission authority

---

3. Problem It Solves

Without a dedicated authz role, action authority tends to become fragmented across:

- UI visibility rules
- hidden admin checks
- controller-level conditionals
- operator assumptions
- informal “trusted user” behavior
- partial role checks scattered through the system

That fragmentation causes:

- unclear who is allowed to do what
- inconsistent protected-action enforcement
- privilege creep
- weak separation-of-duties enforcement

- poor auditability of why an action was allowed or denied

The Permissions / AuthZ Manager solves this by making actor authority explicit.

---

4. Scope

The Permissions / AuthZ Manager governs:

- actor authorization evaluation
- permission or role resolution
- scope-sensitive access control
- protected-action checks
- separation-of-duties enforcement
- privileged-action gating
- authorization decision generation
- authorization denial reasons
- authorization decision linkage to downstream action flows

The Permissions / AuthZ Manager does not govern:

- whether policy allows the action in abstract terms
- approval object lifecycle
- override object lifecycle
- canonical run persistence
- audit storage mechanics
- environment capability truth

- artifact integrity truth

Those are handled by adjacent roles.

---

5. Core Principle

The Permissions / AuthZ Manager is the **actor-authority layer**, not the full governance plane and not a substitute for policy.

It answers:

“May this actor perform this action here?”

This means:

- it evaluates actor authority
- it enforces scope and privilege boundaries
- it does not itself perform the action
- it does not itself decide general policy truth
- it does not itself grant approval where approval is separately required

It is the formal authorization layer for governed action execution.

---

6. Primary Responsibilities

6.1 Resolve actor authority

- identify actor role(s)
- identify actor permission set
- identify actor scope and access boundaries

6.2 Evaluate action authorization

- validate whether actor may perform requested action
- validate whether actor may act on target object within target scope
- enforce privileged-action restrictions

6.3 Enforce separation of duties

- reject self-approval where forbidden
- reject self-exception where forbidden
- reject conflicting requester/reviewer/operator combinations where rules require independence

6.4 Produce usable authz decisions

- return allow/deny result
- return structured denial reason
- return effective authority snapshot where needed
- expose role/permission basis used for decision

6.5 Support downstream governance and execution

- allow other roles to consume authz decisions before acting
- ensure authorization remains explicit and inspectable

---

7. Authority

The Permissions / AuthZ Manager has authority to:

- determine whether an actor is authorized for a governed action
- reject unauthorized actions
- enforce scope restrictions
- enforce privileged-action restrictions
- enforce separation-of-duties constraints where authorization participates

The Permissions / AuthZ Manager does not have authority to:

- redefine policy truth
- grant approval in place of approvers
- activate override objects by itself
- mutate canonical run state
- fabricate actor identity
- rewrite audit truth
- invent roles or permissions outside defined system mappings

It governs authorization, not general governance law.

---

8. Inputs

The Permissions / AuthZ Manager consumes:

8.1 Actor inputs

- actor ID
- actor role(s)
- actor permission set

- actor group or org membership where relevant
- actor scope bindings

8.2 Action inputs

- requested action type
- target object type
- target object ID
- target workspace/project/scope
- action context metadata

8.3 Governance context

- protected-action classification
- role requirement rules
- separation-of-duties rules
- prior related decision context where relevant

8.4 Permission model inputs

- role definitions
- permission definitions
- access mappings
- delegation rules where supported

---

9. Outputs

The Permissions / AuthZ Manager produces:

9.1 Authorization decision

- allowed
- denied
- conditionally allowed if modeled with dependent governance signals

9.2 Authorization explanation data

- denial reason
- missing role/permission
- scope mismatch
- privileged-action restriction
- separation-of-duties conflict

9.3 Authorization support data

- effective actor role snapshot
- effective permission snapshot
- authorization basis metadata

---

10. Primary Interfaces

The Permissions / AuthZ Manager should expose, at minimum:

10.1 `authorizeAction`

Return whether actor may perform requested action in context.

10.2 `getActorRoles`

Return resolved actor roles.

10.3 `getActorPermissions`

Return resolved actor permissions.

10.4 `validateScopeAccess`

Return whether actor may act within requested scope.

10.5 `validateSeparationOfDuties`

Return whether action violates independence constraints.

10.6 `explainAuthzResult`

Return readable explanation for an authorization decision.

---

11. Authorization Model

The Permissions / AuthZ Manager must require an explicit authorization model.

At minimum, the model should define:

- actor identity
- role definitions
- permission definitions
- action-to-permission mapping
- scope model
- privileged-action classes
- delegation rules if supported

- separation-of-duties rules

The exact syntax may vary, but authorization must not rely on hidden assumptions.

---

12. Scope Rules

Authorization must be scope-sensitive where the system has scoped objects.

The Permissions / AuthZ Manager must define how access depends on:

- workspace
- project
- run
- release
- environment
- administrative boundary
- artifact or registry domain where relevant

An actor may be authorized in one scope and unauthorized in another.

Scope mismatch must be explicit.

---

13. Protected-Action Rules

The Permissions / AuthZ Manager must define which actions are privileged or protected from ordinary users.

Examples may include:

- reprioritizing privileged queue items
- resuming or cancelling protected runs

- activating overrides
- approving sensitive releases
- changing quota-related settings
- unpinning protected artifacts
- dispatching to restricted environments
- modifying permission-sensitive governance objects

The exact list may vary, but protected-action rules must be explicit.

---

14. Separation-of-Duties Rules

The Permissions / AuthZ Manager must support explicit independence constraints.

Examples:

- requester may not approve own request
- override requester may not be sole override approver
- release creator may not be sole release approver
- actor who performs privileged mutation may require independent reviewer
- auditor role may be restricted from mutating certain operational objects

The role must define:

- which combinations are disallowed
- what constitutes conflict
- how conflict is surfaced

---

15. Delegation Rules

If the system supports delegated authority, the Permissions / AuthZ Manager must define:

- who may delegate
- which permissions are delegable
- scope bounds of delegation
- time bounds of delegation
- revocation behavior
- auditability requirements

If delegation is not supported, the role should state that explicitly.

Delegation must never become an informal bypass path.

---

16. Relationship to Policy Engine

Policy and authorization are distinct.

A policy may allow an action generally, while the actor is still unauthorized.

In simple terms:

- Policy Engine = is the action allowed under policy?
- AuthZ Manager = may this actor do it?

Both may need to pass for action execution.

---

17. Relationship to Approval Router

Approval and authorization are distinct.

An actor may be authorized to approve, or unauthorized to approve, regardless of whether approval is required.

In simple terms:

- Approval Router = approval flow exists and is pending/resolved
- AuthZ Manager = may this actor act as approver or requester in that flow?

The Approval Router depends on authz truth for valid approver actions.

---

18. Relationship to Override Manager

Override and authorization are distinct.

An actor may be authorized or unauthorized to:

- request override
- approve override
- revoke override
- use override

The Override Manager depends on authz truth for those action paths.

An override should not become a substitute for missing base actor authority unless explicitly designed as such.

---

19. Relationship to Quota Manager

Quota and authorization are also distinct.

An actor may be authorized to request an action, but quota may still block it.

In simple terms:

- AuthZ Manager = may this actor attempt the action?
- Quota Manager = is resource-governed execution of that action available?

---

20. Relationship to Audit Logger

Authorization decisions, especially for protected actions, must be audit-linked.

The Permissions / AuthZ Manager must ensure audit coverage for:

- authorization denial for protected action
- authorization success for sensitive action where required
- role/permission changes if governed here or linked through adjacent systems
- separation-of-duties denial
- delegated authority use or denial, if supported

The audit trail should preserve:

- actor
- action requested
- target object
- scope
- authz result
- denial reason or authorization basis
- AI participation metadata where applicable

---

21. AI Assistance Boundary

The Permissions / AuthZ Manager may be AI-assisted in bounded ways.

AI may help with:

- generating readable explanations for denied access
- identifying suspicious permission usage patterns
- summarizing unusual protected-action attempts
- clustering repeated denial reasons
- recommending review of role/permission anomalies for operators

AI may not:

- grant permission directly
- invent actor roles
- authorize protected actions by itself
- waive separation-of-duties constraints
- overrule deterministic authorization denial

The Permissions / AuthZ Manager remains the authoritative actor-authority role.

---

22. Failure Handling

The Permissions / AuthZ Manager must fail safely.

22.1 Unknown actor

Deny or block until actor identity is resolved according to explicit rule.

22.2 Missing role/permission data

Fail closed or block until explicit authorization context exists.

22.3 Ambiguous scope mapping

Do not guess; deny or block according to explicit rule.

22.4 Separation-of-duties conflict

Reject action and preserve conflict reason.

22.5 AI explanation failure

Ignore AI assistance and preserve deterministic authz result.

---

23. Required Guarantees

The Permissions / AuthZ Manager must guarantee:

- actor authority is evaluated explicitly
- scope restrictions are enforceable
- protected actions cannot bypass authorization checks
- separation-of-duties conflicts can be detected
- denial reasons are inspectable
- authorization behavior remains safe without AI

---

24. Success Criteria

The Permissions / AuthZ Manager is correctly established when:

- actor authorization is centralized and explicit
- protected actions require explicit authorization result
- scope-aware access control works deterministically
- separation-of-duties rules are enforceable

- downstream roles can rely on authz decisions cleanly
- AI remains advisory and non-authoritative for permission truth

---

25. Summary Statement

The Permissions / AuthZ Manager is the actor-authority role within the Governance Control Plane.

It determines whether a specific actor may perform a specific governed action within a specific scope, and it enforces explicit role, permission, privilege, and separation-of-duties rules.

It may use AI for explanation and anomaly-review support, but it does not delegate authorization authority to AI.

GCP-05 — Quota Manager

GCP-05 — Quota Manager

1. Purpose

The Quota Manager is the role responsible for **governing resource and usage limits for governed actions** inside the Governance Control Plane.

Its purpose is to ensure that execution and administrative actions consume resources only within explicit limits, budgets, and throttling rules.

The Quota Manager governs **resource authority**.

It is the role that decides whether a requested action may proceed from a resource-governance perspective.

---

2. Objective

The objective of the Quota Manager is to ensure that:

- resource limits are explicit
- usage is metered consistently
- overages are blocked, throttled, or deferred through defined rules
- quota state is visible and inspectable
- actions cannot silently exceed intended resource governance boundaries
- quota decisions are usable by execution and governance roles
- AI may assist forecasting and explanation, but may not become quota authority

---

3. Problem It Solves

Without a dedicated quota role, resource governance tends to become fragmented across:

- controller-side counters
- environment-specific limits
- informal operator judgment
- hidden config thresholds
- manual pause decisions
- ad hoc retry behavior

That fragmentation causes:

- unpredictable resource consumption
- inconsistent throttling
- quota bypass risk
- poor visibility into why work was blocked
- weak control over system growth and pressure

The Quota Manager solves this by making resource governance explicit.

---

4. Scope

The Quota Manager governs:

- quota definition consumption and enforcement
- usage metering
- quota-state tracking
- overage detection
- throttle/defer/deny decisions

- quota availability signaling
- burst handling if supported
- cooldown/reset handling if supported
- quota visibility outputs
- quota decision linkage to downstream actions

The Quota Manager does not govern:

- general policy truth
- actor permission truth
- approval routing
- canonical run persistence
- queue ordering by itself
- environment capability truth
- audit storage mechanics

Those are handled by adjacent roles.

---

5. Core Principle

The Quota Manager is the **resource-governance authority**, not the execution scheduler and not the billing system unless explicitly extended that far.

It answers:

“Is sufficient governed resource available for this action right now?”

This means:

- it meters usage

- it evaluates limits
- it returns allow/throttle/defer/deny outcomes
- it does not itself execute the action
- it does not itself define policy outside quota rules
- it does not itself dispatch work order

It is the formal limit-enforcement layer for governed consumption.

---

6. Primary Responsibilities

6.1 Maintain quota state

- track current quota availability
- preserve quota scope
- preserve usage state
- preserve limit metadata

6.2 Meter usage

- consume or reserve quota when required
- track usage events
- classify usage by resource type and scope

6.3 Enforce limits

- deny actions that exceed hard limits
- throttle actions when throttling is the correct response

- defer actions where allowed
- expose remaining quota

6.4 Support reset/cooldown behavior

- apply rolling or fixed-window reset rules
- expose reset timing
- preserve exhaustion state where relevant

6.5 Provide downstream governance signals

- quota available
- quota exhausted
- throttle required
- defer until reset/release
- quota reservation granted/denied

---

7. Authority

The Quota Manager has authority to:

- meter usage for governed actions
- determine whether quota is available
- deny or throttle actions due to resource limits
- expose remaining quota state
- reserve or consume capacity where quota semantics require it

The Quota Manager does not have authority to:

- redefine authorization rules
- override policy denials
- reorder queue items by itself
- invent extra resource capacity
- mutate canonical run truth directly
- fabricate usage or reset history
- silently ignore hard limits

It governs resource availability, not all aspects of action permission.

---

8. Inputs

The Quota Manager consumes:

8.1 Action inputs

- requested action
- resource class required
- scope of usage
- estimated or declared usage amount where applicable

8.2 Scope inputs

- workspace/project scope
- actor scope if relevant
- environment scope if relevant

- run/release/action class scope

8.3 Quota-definition inputs

- hard limits
- soft limits
- burst rules
- reset rules
- throttle rules
- reservation rules

8.4 Usage inputs

- prior usage ledger
- active reservations
- in-flight consumption
- cooldown state
- exhaustion state

---

9. Outputs

The Quota Manager produces:

9.1 Quota decisions

- allowed
- denied

- throttled
- deferred
- reservation granted
- reservation denied

9.2 Quota state

- remaining quota
- exhausted quota
- reserved capacity
- reset timing
- throttle state

9.3 Explanation data

- over-limit reason
- scope-limited reason
- reset/cooldown reason
- burst allowance basis
- usage summary

---

10. Primary Interfaces

The Quota Manager should expose, at minimum:

10.1 `getQuotaState`

Return current quota state for a given scope and resource class.

10.2 `meterUsage`

Record or reserve governed resource usage.

10.3 `enforceQuota`

Return whether the requested action may proceed under quota rules.

10.4 `reserveQuota`

Reserve quota for a future or in-flight action where required.

10.5 `releaseQuota`

Release reservation where action is cancelled, fails, or does not consume reserved capacity.

10.6 `getResetInfo`

Return reset/cooldown timing.

10.7 `forecastPressure`

Optional interface for advisory forecasting support.

---

11. Quota Model

The Quota Manager must require an explicit quota model.

At minimum, quotas should define:

- quota ID or class
- scope
- resource type
- limit value
- window or period

- hard vs soft behavior
- throttle/defer behavior
- reset rules
- reservation semantics

The exact schema may vary, but quota must be a first-class governed object, not scattered counters.

---

12. Resource Classes

The Quota Manager must define what kinds of resources are governed.

Examples may include:

- runs per time window
- stage executions per time window
- compute budget
- storage budget
- artifact packaging/export budget
- high-cost environment usage
- AI model invocation budget, if governed here
- concurrency slot usage, if governed here

The exact set may vary, but the resource classes must be explicit.

---

13. Scope Rules

Quota must be scope-aware.

The Quota Manager must define how quota applies to:

- workspace
- project
- actor
- environment
- run class
- release class
- global system capacity
- tenant/organization boundary if applicable

A quota satisfied in one scope must not automatically imply satisfaction in another.

---

14. Enforcement Rules

The Quota Manager must define how limits are enforced.

At minimum it must answer:

- when to deny
- when to throttle
- when to defer
- when to allow burst
- when to reserve rather than consume immediately
- when to release unused reservation
- how hard limits differ from soft limits

Quota enforcement must not rely on undocumented behavior.

---

15. Reset and Cooldown Rules

The Quota Manager must define:

- reset window type
- fixed vs rolling window behavior
- cooldown state after exhaustion
- whether partial quota becomes available before full reset
- how reset timing is exposed downstream

Stale or ambiguous reset handling should fail safely.

---

16. Reservation Rules

If quota reservation is supported, the Quota Manager must define:

- when reservation is required
- how long reservation remains valid
- what consumes reserved quota
- what releases reserved quota
- whether reservation blocks other actions
- what happens on action failure or cancellation

Reservation must not become hidden unbounded locking.

---

17. Burst Rules

If burst behavior is supported, the Quota Manager must define:

- who may burst
- what resource classes allow burst
- maximum burst amount
- whether burst requires approval or elevated policy
- how burst is audited and reset

If burst is not supported, the role should state that explicitly.

---

18. Relationship to Policy Engine

Quota and policy are related but distinct.

A policy may require quota availability, and quota state may affect policy outcome.

In simple terms:

- Policy Engine = is the action allowed under policy?
- Quota Manager = is sufficient governed resource available?

Both may need to pass before action execution.

---

19. Relationship to Queue / Scheduler Manager

The Queue / Scheduler Manager may use quota state to decide whether work may dispatch now or must remain deferred.

In simple terms:

- Quota Manager = resource available?
- Queue / Scheduler Manager = if not, defer/throttle; if yes, dispatch may proceed

The Quota Manager does not decide queue order by itself.

---

20. Relationship to Adapter / Environment Manager

Some quota classes may be environment-specific.

The Adapter / Environment Manager defines target capability and readiness.

The Quota Manager defines whether resource governance allows usage of that target.

In simple terms:

- Adapter / Environment Manager = target can run this
- Quota Manager = target usage is currently allowed under resource limits

---

21. Relationship to Audit Logger

Quota decisions for meaningful governed actions must be audit-linked.

The Quota Manager must ensure audit coverage for:

- quota denial
- throttle/defer decision
- reservation grant/denial
- burst use if supported
- exhaustion event
- reset event where relevant to governance traceability

The audit trail should preserve:

- actor or system source

- action requested
- scope
- resource class
- quota result
- remaining/exhausted state
- reason metadata
- AI participation metadata where applicable

---

22. AI Assistance Boundary

The Quota Manager may be AI-assisted in bounded ways.

AI may help with:

- forecasting quota pressure
- summarizing resource spike causes
- recommending non-authoritative defer windows
- identifying recurring exhaustion patterns
- generating readable explanations for quota-based denials

AI may not:

- increase quota directly
- ignore hard limits
- authorize over-limit actions by itself
- fabricate resource availability

- overrule deterministic quota denial

The Quota Manager remains the authoritative resource-governance role.

---

23. Failure Handling

The Quota Manager must fail safely.

23.1 Missing quota definition

Fail closed or block until explicit quota truth exists for governed action class.

23.2 Missing usage ledger state

Do not guess availability if the system requires explicit usage truth.

23.3 Reservation conflict

Reject or block conflicting reservation according to explicit rules.

23.4 Reset ambiguity

Do not silently reset quota without explicit rule support.

23.5 AI forecasting failure

Ignore AI assistance and preserve deterministic quota behavior.

---

24. Required Guarantees

The Quota Manager must guarantee:

- quota classes and scopes are explicit
- resource usage is metered through governed paths
- hard limits are enforceable

- throttle/defer behavior is inspectable
- stale or ambiguous quota state does not silently authorize actions
- quota behavior remains safe without AI

---

25. Success Criteria

The Quota Manager is correctly established when:

- quotas are first-class governed objects
- resource-governed actions receive explicit quota decisions
- exhaustion, throttling, and reset behavior are inspectable
- downstream execution and governance roles can consume quota state cleanly
- quota decisions are auditable
- AI remains advisory and non-authoritative for quota truth

---

26. Summary Statement

The Quota Manager is the resource-governance role within the Governance Control Plane.

It meters governed usage, enforces explicit limits, and produces allow/throttle/defer/deny decisions so downstream roles can act within bounded resource authority.

It may use AI for forecasting and explanation support, but it does not delegate quota authority to AI.

ECP Artifact Pack

ECP-00 — Purpose + Boundaries

1. Purpose

The Evidence Control Plane (ECP) is the control plane responsible for **preserving proof-bearing, integrity-bearing, and trust-bearing system records** inside Axion.

Its purpose is to provide the authoritative evidence layer for:

- audit history
- artifact integrity anchoring
- release evidence
- artifact identity and lineage
- notification delivery evidence where governance requires it

The ECP exists to govern *what the system can prove*.

It is the plane that preserves trustworthy records of what happened, what artifacts exist, how they relate, and whether integrity-sensitive system objects remain valid.

---

2. Objective

The objective of the ECP is to ensure that:

- control-plane-relevant actions are evidentially preserved
- integrity-sensitive artifacts can be anchored and verified
- release state is represented as a governed evidence object
- artifact identity and lineage are explicit and inspectable
- important operational events can preserve delivery evidence

- downstream operators and services can rely on evidence-bearing truth
- deterministic evidence authority remains intact even when AI assists with analysis and summarization

---

3. Problem It Solves

Without a formal evidence control plane, trust-bearing records become fragmented across:

- ad hoc logs
- local files
- partial release records
- informal artifact naming
- missing provenance links
- non-governed notification history
- mutable operational state with no clear proof model

That fragmentation causes:

- weak auditability
- weak integrity verification
- missing provenance
- unclear release evidence
- poor operator trust in what can be proven versus what is merely current state

The ECP solves this by making evidence authority explicit.

---

4. What the ECP Governs

The ECP governs:

- append-only or governed audit evidence
- artifact pinning and integrity anchoring
- release evidence records
- artifact identity records
- artifact lineage/provenance relationships
- event/notification delivery evidence where required
- evidence linkage between actions, artifacts, releases, and system decisions

---

5. What the ECP Does Not Govern

The ECP does **not** govern:

- whether an action is allowed under policy
- whether an actor is authorized
- whether approval is required
- whether a transition is structurally legal
- queue priority
- dispatch timing
- environment capability truth
- execution-state authority

Those belong to the Governance or Execution Control Planes.

The ECP may record their results, but it does not replace them.

---

6. Core Principle

The ECP is the **evidence authority**, not the execution engine and not the policy engine.

It preserves trustworthy records of:

- actions
- artifact states
- releases
- provenance
- integrity anchors
- delivery evidence

This means:

- it governs proof-bearing records
- it governs integrity-bearing records
- it governs provenance-bearing records
- it does not invent governance permission
- it does not invent execution legality
- it does not replace canonical execution state

---

7. Governing Model

The ECP follows this model:

governed action or artifact event → evidence object creation/update → preserved trust-bearing record

That record may represent:

- audit evidence
- artifact integrity anchor
- release state evidence
- artifact registry entry
- lineage edge
- notification delivery evidence

The ECP is the plane that transforms important system events into inspectable, governed evidence objects.

---

8. Primary Responsibilities

The ECP is responsible for:

8.1 Audit evidence governance

- preserve append-only or governed audit history
- capture actor/action/target/time/evidence linkage
- support later inspection and verification

8.2 Integrity governance

- anchor artifact identity to integrity references
- support pin verification
- surface artifact drift or mismatch

8.3 Release evidence governance

- preserve release records
- preserve release signatures/approvals/evidence refs
- support publish/revoke evidence history

8.4 Artifact registry and lineage governance

- preserve artifact identity
- preserve producer/consumer relationships
- preserve provenance and lineage

8.5 Notification evidence governance

- preserve evidence that important control-plane events were emitted and delivered where required

---

9. ECP Role Set

The ECP is composed of five roles:

ECP-01 Audit Logger

Owns append-only or governed action evidence.

ECP-02 Pin Manager

Owns artifact integrity anchors and drift verification.

ECP-03 Release Manager

Owns governed release evidence lifecycle.

ECP-04 Artifact Registry / Lineage Governor

Owns artifact identity, provenance, and lineage truth.

ECP-05 Notification Router

Owns event/notification routing with delivery evidence linkage where required.

---

10. Inputs

The ECP consumes the following input classes:

10.1 Action evidence inputs

- execution actions
- governance decisions
- approval decisions
- override events
- release events
- privileged administrative actions

10.2 Artifact inputs

- artifact creation events
- artifact update events
- artifact content hashes
- artifact references
- stage outputs
- packaging outputs

10.3 Release inputs

- release requests
- release signatures
- release evidence refs
- publish/revoke results

10.4 Provenance inputs

- producer stage references
- consumer references
- artifact dependencies
- run/release linkage

10.5 Event inputs

- notification-worthy control-plane events
- delivery state
- retry state
- escalation state where relevant

---

11. Outputs

The ECP produces:

11.1 Evidence objects

- audit entries
- pin records

- release records
- artifact registry entries
- lineage records
- notification delivery records

11.2 Evidence decisions/results

- integrity verified / mismatch
- release active / revoked
- artifact provenance resolved / unresolved
- delivery sent / failed / retried

11.3 Evidence events

- audit append event
- pin drift event
- release publish/revoke event
- lineage update event
- notification delivery event

---

12. Dependencies

The ECP depends on other system layers but remains evidence-authoritative within its scope.

12.1 Execution dependencies

The ECP may consume:

- run IDs
- stage IDs
- run lifecycle events
- stage outputs
- environment route decisions

These are evidence inputs, not execution authority.

12.2 Governance dependencies

The ECP may consume:

- policy decisions
- approval results
- override status
- authorization results
- quota outcomes

These are evidence-bearing governance inputs, not replacements for governance roles.

---

13. Authority Boundaries

The ECP has authority to:

- preserve evidence objects
- define integrity-anchor state
- define release evidence records
- define artifact identity and lineage records

- preserve delivery evidence for important routed events

The ECP does **not** have authority to:

- approve governed actions
- authorize actors
- allow denied policy actions
- execute pipeline movement
- invent missing actor authority
- fabricate evidence for events that did not occur
- rewrite proof-bearing history outside explicitly governed mutation rules

---

14. Trust-Bearing Object Classes

The ECP should explicitly recognize several trust-bearing object classes.

14.1 Audit objects

Records of governed actions.

14.2 Integrity objects

Pins, hashes, and verification records.

14.3 Release objects

Release records, signatures, publish/revoke evidence.

14.4 Artifact identity objects

Artifact registry entries and identity metadata.

14.5 Lineage objects

Producer/consumer relationships and provenance edges.

14.6 Delivery evidence objects

Notification delivery attempts, statuses, and retries where relevant.

---

15. AI Assistance Boundary

The ECP may be AI-assisted, but AI is not the ECP.

AI may help with:

- summarizing audit trails
- explaining likely causes of artifact drift
- summarizing release readiness
- explaining lineage breaks
- clustering notification failures
- producing operator-friendly evidence summaries

AI may not:

- rewrite audit truth
- forge integrity verification
- invent artifact lineage authoritatively
- publish or revoke releases by itself
- fabricate delivery success
- overrule deterministic evidence records

The ECP remains the authoritative evidence layer.

---

16. Operating Modes

The ECP should support these evidence modes:

16.1 Deterministic-only mode

Evidence is preserved and evaluated with no AI assistance.

16.2 AI-assisted advisory mode

AI summarizes or explains evidence while the ECP remains the authority.

16.3 AI-assisted validate-first mode

AI may propose categorization or interpretation that must still be validated against evidence truth before use.

16.4 Restricted low-risk automation mode

Only explicitly whitelisted low-risk evidence support actions may auto-apply, and never in ways that mutate proof-bearing truth without governed rules.

---

17. Failure Handling Expectations

The ECP must fail safely.

17.1 Missing evidence input

Do not invent missing proof-bearing data.

17.2 Integrity ambiguity

Surface unresolved or failed verification rather than silently assuming validity.

17.3 Missing provenance

Mark lineage/provenance as unresolved rather than fabricated.

17.4 Release evidence incomplete

Do not treat incomplete evidence as full release validity.

17.5 AI failure

Ignore AI assistance and continue deterministic evidence behavior.

---

18. Required Guarantees

The ECP must guarantee:

- trust-bearing system records are explicitly represented
- important actions can be tied to evidence objects
- integrity-sensitive artifacts can be verified
- release evidence is inspectable
- artifact provenance is explicit where available
- delivery evidence is preserved where required
- evidence behavior remains safe without AI

---

19. Success Criteria

The ECP is correctly established when:

- audit, pins, releases, artifact registry, lineage, and delivery evidence are first-class roles or objects
- trust-bearing records are inspectable
- integrity verification is explicit
- provenance is explicit
- release evidence is explicit
- AI assistance remains bounded and non-authoritative for evidence truth

---

20. Relationship to Other Planes

ECP vs ICP

- ECP governs **what can be proven about execution-related events and artifacts**
- ICP governs **how execution movement occurs**

ECP vs GCP

- ECP governs **evidence and integrity-bearing truth**
- GCP governs **permission, approval, override, and quota decisions**

ECP vs AIG

- ECP governs **evidence authority**
- AIG governs **how AI may assist evidence authority**

---

21. Summary Statement

The Evidence Control Plane is the Axion evidence authority layer.

It governs audit records, integrity anchors, release evidence, artifact identity, provenance, and delivery evidence through explicit trust-bearing system objects and deterministic evidence rules.

It may be assisted by AI, but it does not surrender evidence authority to AI.

ECP-01 — Audit Logger

ECP-01 — Audit Logger

1. Purpose

The Audit Logger is the role responsible for **preserving append-only or otherwise governed evidence of important system actions** inside the Evidence Control Plane.

Its purpose is to ensure that control-plane-relevant actions can later be inspected, traced, explained, and verified as having occurred in a specific order with specific actors, targets, and outcomes.

The Audit Logger governs **action evidence**.

It is the role that turns important system activity into a durable, reviewable record.

---

2. Objective

The objective of the Audit Logger is to ensure that:

- important actions are recorded explicitly
- audit records preserve who did what, to what, when, and with what result
- audit history is durable and ordered
- tamper-evidence or equivalent integrity protection exists where required
- downstream operators can reconstruct critical system decisions
- audit coverage applies across execution, governance, and evidence roles
- AI may assist summarization and anomaly review, but may not become audit truth

---

3. Problem It Solves

Without a dedicated audit role, action history becomes fragmented across:

- transient logs
- CLI output
- UI history
- scattered JSON files
- partial release notes
- inconsistent event streams
- controller-specific debug logs

That fragmentation causes:

- unclear operator accountability
- weak postmortem traceability
- poor protected-action reviewability
- missing linkage between governance decisions and resulting actions
- weak trust in what the system can prove actually happened

The Audit Logger solves this by making action evidence explicit.

---

4. Scope

The Audit Logger governs:

- audit entry creation
- audit entry identity and ordering
- append-only or governed mutation behavior
- actor/action/target/outcome recording

- event-to-evidence linkage
- integrity chaining or equivalent tamper-evidence model
- audit query surfaces
- audit verification support
- audit coverage requirements for control-plane events

The Audit Logger does not govern:

- whether the underlying action is allowed
- whether a run transition is legal
- whether a release is valid
- whether an artifact pin is correct
- canonical run persistence
- policy truth
- actor permission truth

Those are handled by adjacent roles.

---

5. Core Principle

The Audit Logger is the **historical action-evidence authority**, not the source of current execution truth.

It preserves evidence that an action occurred, not the complete replacement for every operational state store.

This means:

- it records history

- it preserves accountability
- it preserves evidence references
- it does not replace run state
- it does not replace policy evaluation
- it does not replace artifact registry truth

It is the formal evidence history layer for significant actions.

---

6. Primary Responsibilities

6.1 Create audit entries

- create stable audit entries
- preserve event identity and ordering
- capture minimum required action metadata

6.2 Preserve actor/action/target/outcome evidence

- record actor
- record action type
- record target object
- record scope
- record resulting outcome or status
- record timestamp

6.3 Preserve evidence linkage

- link related run/release/artifact/approval/override objects
- link evidence refs where present
- link AI participation metadata where applicable

6.4 Preserve tamper-evidence or equivalent integrity

- chain entries where applicable
- preserve prior-hash/current-hash or equivalent verification data
- support audit verification operations

6.5 Support audit inspection

- query entries
- filter by actor/action/object/scope/time
- support review of protected actions and governance outcomes

---

7. Authority

The Audit Logger has authority to:

- append audit entries
- define audit entry structure
- define ordering and linkage rules
- verify audit-chain integrity where applicable
- reject malformed audit append attempts

The Audit Logger does not have authority to:

- decide whether an action was allowed
- retroactively alter action truth outside explicitly governed correction rules
- fabricate actor identity
- fabricate evidence for events that did not occur
- replace execution-state persistence
- replace artifact identity or release truth

It governs audit evidence, not all system truth.

---

8. Inputs

The Audit Logger consumes:

8.1 Action inputs

- run lifecycle actions
- stage execution outcomes
- policy decisions
- approval decisions
- override events
- authorization denials
- quota denials/throttles
- release actions
- pin actions
- lineage mutations

- notification routing outcomes
- administrative actions

8.2 Context inputs

- actor identity
- action type
- target object type
- target object ID
- scope
- reason/justification where relevant
- resulting status or outcome

8.3 Evidence-link inputs

- run IDs
- release IDs
- artifact IDs
- approval IDs
- override IDs
- policy IDs
- evidence refs
- AI trace refs where applicable

---

9. Outputs

The Audit Logger produces:

9.1 Audit records

- audit entries
- audit chains or ordered logs
- verification records where supported

9.2 Audit query results

- filtered audit lists
- actor-based audit views
- object-based audit views
- time-range audit views
- protected-action audit views

9.3 Verification outcomes

- audit chain valid
- audit chain broken
- entry malformed
- entry missing linkage
- integrity check unresolved

---

10. Primary Interfaces

The Audit Logger should expose, at minimum:

10.1 `appendAuditEntry`

Append a new audit record.

10.2 `getAuditEntry`

Return a specific audit entry by ID.

10.3 `queryAuditEntries`

Return audit entries by filter.

10.4 `listAuditEntries`

Return audit entries in deterministic order.

10.5 `verifyAuditChain`

Return whether audit ordering/integrity chain is valid.

10.6 `explainAuditEntry`

Return readable explanation of an audit entry where helpful.

---

11. Audit Entry Model

The Audit Logger must require an explicit audit entry model.

At minimum, an audit entry should include:

- audit_entry_id
- timestamp
- actor
- action
- target_type
- target_id

- scope
- outcome/result
- reason/summary where applicable
- related object refs
- evidence refs
- prior_hash or equivalent predecessor ref
- current_hash or equivalent integrity ref
- AI participation metadata where applicable

The exact schema may vary, but audit entries must be first-class governed objects.

---

12. Coverage Rules

The Audit Logger must define what actions require audit coverage.

At minimum, audit-relevant classes should include:

- run create/start/advance/pause/resume/cancel/fail/complete
- policy denials and meaningful policy outcomes
- approval create/approve/reject/expire/escalate
- override create/activate/revoke/expire/use
- authz denials for protected actions
- quota denials or throttles for meaningful governed actions
- release create/sign/publish/revoke
- pin create/unpin/verify failure

- artifact registry/lineage mutations
- privileged admin actions
- AI-influenced governed action decisions where applicable

The exact list may expand, but coverage must be explicit.

---

13. Ordering Rules

The Audit Logger must define how audit order is preserved.

It must answer:

- what determines entry order
- how simultaneous events are ordered
- how prior-entry linkage is represented
- how append consistency is ensured
- how missing or broken sequences are surfaced

Audit order must not rely on ambiguous incidental behavior.

---

14. Integrity / Tamper-Evidence Rules

The Audit Logger must define how audit integrity is preserved.

Examples may include:

- hash chaining
- signed segments
- append-only ledger semantics

- immutable storage windows
- verification snapshots

The exact implementation may vary, but the role must define:

- how tamper-evidence works
- what invalidates verification
- how broken chains are surfaced
- who may repair or annotate corrupted histories, if ever allowed

The Audit Logger must not silently tolerate evidence corruption.

---

15. Mutation Rules

Audit history should be append-only by default.

The Audit Logger must define:

- whether entries are immutable
- whether correction annotations are allowed
- whether redaction is ever allowed and under what governed rules
- how supersession or correction is represented without erasing history

Corrections must not become silent deletion or rewrite.

---

16. Query Rules

The Audit Logger must support inspectable audit retrieval.

At minimum, audit queries should support:

- actor
- action type
- target object
- scope
- time range
- related run/release/artifact/approval/override IDs
- protected-action views
- outcome class

The query model may vary, but audit records must be reviewable in practical ways.

---

17. Relationship to Run State Store

The Run State Store preserves canonical current execution truth.

The Audit Logger preserves historical evidence that lifecycle actions occurred.

In simple terms:

- Run State Store = what the run currently is
- Audit Logger = what actions happened to the run over time

These roles are related but not interchangeable.

---

18. Relationship to Policy / Approval / Override / AuthZ / Quota Roles

Governance roles make or support governed decisions.

The Audit Logger preserves evidence that those decisions occurred.

Examples:

- policy denied an action
- approval was requested and resolved
- override was activated
- authz denied a protected action
- quota throttled a request

The Audit Logger records those governance events without replacing the governance roles.

---

19. Relationship to Pin / Release / Registry / Notification Roles

Other evidence roles manage specific trust-bearing objects.

The Audit Logger preserves action evidence about those objects and operations.

Examples:

- a pin was created or verification failed
- a release was published or revoked
- lineage was updated
- a notification event was routed or failed

The Audit Logger is the cross-cutting action-evidence layer.

---

20. AI Assistance Boundary

The Audit Logger may be AI-assisted in bounded ways.

AI may help with:

- summarizing long audit trails
- clustering repeated action patterns
- flagging suspicious sequences for operator review
- producing readable timelines from audit history
- identifying likely anomaly clusters

AI may not:

- add fake audit entries
- delete or rewrite audit truth
- certify audit integrity by itself
- reinterpret missing evidence as confirmed history
- overrule failed audit verification

The Audit Logger remains the authoritative audit-evidence role.

---

21. Failure Handling

The Audit Logger must fail safely.

21.1 Malformed audit append request

Reject append and preserve explicit reason.

21.2 Broken prior-link/integrity chain

Surface verification failure explicitly.

21.3 Missing required audit fields

Reject entry rather than invent missing truth.

21.4 Query inconsistency

Return explicit failure or partial-state warning rather than silent misleading output.

21.5 AI summarization failure

Ignore AI assistance and preserve raw audit truth access.

---

22. Required Guarantees

The Audit Logger must guarantee:

- important governed actions can be recorded explicitly
- audit entries preserve actor/action/target/time/outcome linkage
- audit order is inspectable
- tamper-evidence or equivalent integrity protection is explicit
- audit history is not silently rewritten
- audit behavior remains safe without AI

---

23. Success Criteria

The Audit Logger is correctly established when:

- audit entries are first-class governed objects
- critical execution, governance, and evidence actions are audit-covered
- actors and outcomes are inspectable
- audit ordering is explicit
- tamper-evidence or equivalent integrity verification exists

- operators can investigate protected actions through audit history
- AI remains advisory and non-authoritative for audit truth

---

24. Summary Statement

The Audit Logger is the action-evidence role within the Evidence Control Plane.

It preserves ordered, durable, and inspectable historical evidence of important system actions across execution, governance, and evidence surfaces, with explicit integrity protections where required.

It may use AI for summarization and anomaly-review support, but it does not delegate audit authority to AI.

ECP-02 — Pin Manager

ECP-02 — Pin Manager

1. Purpose

The Pin Manager is the role responsible for **anchoring artifact integrity to explicit verification references** inside the Evidence Control Plane.

Its purpose is to ensure that artifacts which matter to the system can be tied to stable integrity-bearing records so the system can later determine whether those artifacts still match expected content.

The Pin Manager governs **artifact integrity anchors**.

It is the role that turns “this artifact should remain this exact thing” into an explicit, verifiable system record.

---

2. Objective

The objective of the Pin Manager is to ensure that:

- important artifacts can be pinned explicitly
- pin records are durable and inspectable
- integrity verification can be performed deterministically
- drift or mismatch is surfaced clearly
- protected unpinning is governed
- downstream roles can rely on pin truth when needed
- AI may assist explanation and drift analysis, but may not become integrity authority

---

3. Problem It Solves

Without a dedicated pinning role, artifact integrity tends to become fragmented across:

- ad hoc hashes in files
- release notes
- build outputs
- local assumptions
- operator memory
- partial packaging metadata

That fragmentation causes:

- unclear artifact identity stability
- undetected drift
- weak release integrity confidence
- poor operator trust in whether an artifact is still what it was expected to be

The Pin Manager solves this by making integrity anchoring explicit.

---

4. Scope

The Pin Manager governs:

- pin creation
- pin identity
- integrity reference storage
- pin lookup
- pin verification

- drift/mismatch detection
- unpin requests
- protected unpin governance linkage
- verification result preservation
- pin visibility and query surfaces

The Pin Manager does not govern:

- general artifact registry truth
- release approval truth
- policy allow/deny logic
- canonical run persistence
- audit storage mechanics
- environment capability truth

Those are handled by adjacent roles.

---

5. Core Principle

The Pin Manager is the **integrity-anchor authority**, not the artifact creator and not the full artifact registry.

It preserves the expected integrity reference for an artifact and evaluates whether current material matches that reference.

This means:

- it anchors expected artifact state
- it verifies against explicit references

- it surfaces mismatch or drift
- it does not invent missing artifacts
- it does not replace artifact lineage
- it does not itself approve unpinning outside governed paths

It is the formal integrity-reference layer for artifact trust.

---

6. Primary Responsibilities

6.1 Create pin records

- create stable pin IDs
- bind pin to an artifact identity or artifact reference
- preserve integrity reference material
- preserve scope and class of pin

6.2 Maintain pin state

- preserve active pin records
- preserve superseded/unpinned/revoked state where supported
- expose pin lookup surfaces

6.3 Verify artifacts against pins

- compare current artifact state to pinned reference
- return match/mismatch/unverifiable result
- preserve verification metadata

6.4 Detect and surface drift

- identify when artifact state no longer matches expected pin
- preserve drift result for downstream inspection
- expose severity or class if defined

6.5 Govern unpin behavior

- accept governed unpin requests
- enforce protected unpin requirements where applicable
- preserve unpin reason and linkage

---

7. Authority

The Pin Manager has authority to:

- create and manage pin records
- determine whether a given artifact matches its pinned reference
- return verification outcomes
- mark pin state as active/unpinned/superseded according to explicit rules
- reject malformed or invalid pin operations

The Pin Manager does not have authority to:

- fabricate artifact content
- redefine artifact identity by itself
- authorize protected unpinning outside governed rules

- rewrite audit truth
- replace release governance
- silently ignore mismatch

It governs integrity anchors, not all artifact truth.

---

8. Inputs

The Pin Manager consumes:

8.1 Artifact inputs

- artifact ID or artifact reference
- artifact path or locator
- artifact content hash material or equivalent integrity source
- artifact version metadata where relevant

8.2 Pin operation inputs

- pin request
- verify request
- unpin request
- supersede request if supported

8.3 Governance inputs

- authorization result for protected unpin operations
- approval result where required

- policy result where pin operations are governed
- release linkage where relevant

8.4 Registry/provenance inputs

- artifact registry reference
- related release or run linkage
- provenance refs where relevant

---

9. Outputs

The Pin Manager produces:

9.1 Pin records

- active pin
- superseded pin
- unpinned pin record
- pin verification record

9.2 Verification decisions

- match
- mismatch
- unverifiable
- artifact missing
- pin missing

- pin invalid

9.3 Drift results

- drift detected
- drift severity/class if supported
- affected artifact reference
- verification timestamp

---

10. Primary Interfaces

The Pin Manager should expose, at minimum:

10.1 `pinArtifact`

Create a pin for an artifact.

10.2 `getPin`

Return a pin record by ID or artifact reference.

10.3 `listPins`

Return pin records by scope, state, or artifact linkage.

10.4 `verifyPin`

Verify one artifact against its pin.

10.5 `verifyAllPins`

Verify a set of pins or all relevant pins in scope.

10.6 `unpinArtifact`

Deactivate or remove pin according to explicit governance rules.

10.7 `explainVerificationResult`

Return readable explanation of verification outcome.

---

11. Pin Record Model

The Pin Manager must require an explicit pin record model.

At minimum, a pin record should include:

- pin_id
- artifact_id or artifact reference
- integrity reference value
- integrity algorithm/class
- scope
- created_at
- created_by
- state
- verification history or latest verification refs
- related run/release refs where applicable
- unpin/supersession reason where applicable

Pins must be first-class governed objects, not hidden side data.

---

12. Pin Scope Rules

The Pin Manager must define what a pin applies to.

Pin scope may be:

- single artifact instance
- artifact version
- release-bound artifact
- run-produced artifact
- protected artifact class

The role must define:

- whether multiple active pins are allowed
- whether newer pins supersede older ones
- how pin reuse across scopes is handled
- what constitutes scope mismatch during verification

Pin scope must not be ambiguous.

---

13. Verification Rules

The Pin Manager must define how verification works.

At minimum it must answer:

- what is compared
- what constitutes a match
- what constitutes mismatch
- what happens when artifact material cannot be read
- what happens when pin record is missing or malformed
- how verification timestamps/results are preserved

Verification must be deterministic.

---

14. Drift Rules

The Pin Manager must define how drift is represented.

Drift may include:

- content hash mismatch
- artifact missing
- artifact replaced
- pin reference invalid
- verification impossible due to missing source material

The role must define:

- whether drift severity is classified
- when drift triggers downstream action
- whether drift automatically invalidates release readiness or other system states

The Pin Manager surfaces integrity outcome; downstream responses may belong elsewhere.

---

15. Unpin Rules

Unpinning must be governed explicitly.

The Pin Manager must define:

- who may unpin
- which pins are protected

- whether approval is required
- whether unpin preserves historical record
- whether unpin deletes or merely deactivates the pin
- whether unpinned artifacts may be repinned later
- how unpin reason is recorded

Unpin must not become silent integrity erasure.

---

16. Supersession Rules

If the system supports replacing one valid pin with another, the Pin Manager must define:

- when supersession is allowed
- whether prior pin remains historical
- what links old and new pin
- whether approval is needed for protected artifacts
- whether supersession implies re-verification

If supersession is not supported, that should be stated explicitly.

---

17. Relationship to Artifact Registry / Lineage Governor

The Artifact Registry / Lineage Governor governs artifact identity and provenance.

The Pin Manager governs integrity references tied to that artifact identity.

In simple terms:

- Artifact Registry = what artifact this is and how it relates

- Pin Manager = what integrity reference it should match

These roles are linked but distinct.

---

18. Relationship to Release Manager

The Release Manager may depend on pin verification to determine release readiness or release integrity posture.

In simple terms:

- Pin Manager = artifact matches expected integrity reference?
- Release Manager = use that result in governed release lifecycle

The Pin Manager does not itself decide release publishability unless explicitly delegated a narrow gate input role.

---

19. Relationship to Policy / Approval / AuthZ Roles

Pinning and especially unpinning may require governance checks.

Examples:

- protected artifacts may require privileged authorization to unpin
- some pin operations may require approval
- policy may forbid unpinning certain artifact classes

The Pin Manager consumes those governance outputs but does not replace them.

---

20. Relationship to Audit Logger

Pin operations are evidence-relevant and must be audit-linked.

The Pin Manager must ensure audit coverage for:

- pin creation
- verification
- mismatch/drift detection
- unpin
- supersession
- protected pin operation denial

The audit trail should preserve:

- actor or system source
- action
- artifact reference
- pin reference
- verification result
- reason metadata
- AI participation metadata where applicable

---

21. AI Assistance Boundary

The Pin Manager may be AI-assisted in bounded ways.

AI may help with:

- explaining likely causes of mismatch
- summarizing drift impact

- clustering repeated integrity failures
- producing readable verification summaries

AI may not:

- declare mismatch as match
- fabricate integrity references
- authorize protected unpinning by itself
- ignore deterministic verification failure
- redefine artifact integrity truth

The Pin Manager remains the authoritative integrity-anchor role.

---

22. Failure Handling

The Pin Manager must fail safely.

22.1 Missing artifact

Return explicit unverifiable or missing-artifact result.

22.2 Missing pin

Return explicit missing-pin result.

22.3 Malformed integrity reference

Reject invalid pin or mark verification unresolved according to explicit rule.

22.4 Protected unpin without required governance

Reject unpin request and preserve reason.

22.5 AI explanation failure

Ignore AI assistance and preserve deterministic verification truth.

---

23. Required Guarantees

The Pin Manager must guarantee:

- artifact integrity anchors are explicit
- verification outcomes are deterministic
- mismatch/drift is surfaced explicitly
- protected unpinning is governable
- historical integrity state is not silently erased
- pin behavior remains safe without AI

---

24. Success Criteria

The Pin Manager is correctly established when:

- pins are first-class governed objects
- important artifacts can be pinned and verified
- drift is visible and inspectable
- protected unpinning cannot occur without proper governance
- release and artifact roles can consume integrity outcomes cleanly
- AI remains advisory and non-authoritative for integrity truth

---

25. Summary Statement

The Pin Manager is the artifact-integrity-anchor role within the Evidence Control Plane.

It creates, maintains, verifies, and governs pins as explicit trust-bearing records that allow the system to determine whether important artifacts still match expected integrity state.

It may use AI for explanation and drift-analysis support, but it does not delegate integrity authority to AI.

ECP-03 — Release Manager

ECP-03 — Release Manager

1. Purpose

The Release Manager is the role responsible for **governing release records and release lifecycle evidence** inside the Evidence Control Plane.

Its purpose is to ensure that releases are represented as explicit, trust-bearing system objects with clear lifecycle state, evidence linkage, integrity context, and publish/revoke history.

The Release Manager governs **release evidence authority**.

It is the role that turns “this set of outputs is now a governed release” into an explicit, inspectable system record.

---

2. Objective

The objective of the Release Manager is to ensure that:

- releases are first-class governed objects
- release lifecycle state is explicit
- release evidence is preserved
- release signatures and approvals can be linked where required
- publish and revoke actions are inspectable
- downstream operators can determine release status and readiness from explicit records
- AI may assist summarization and readiness explanation, but may not become release authority

---

3. Problem It Solves

Without a dedicated release role, release truth tends to become fragmented across:

- ad hoc packaging outputs
- local folders
- CLI logs
- informal “final” markers
- partial notes about what shipped
- disconnected approval or signature records

That fragmentation causes:

- unclear release identity
- weak release traceability
- poor linkage between artifacts and release status
- weak trust in what is actually published, revoked, or pending

The Release Manager solves this by making release evidence explicit.

---

4. Scope

The Release Manager governs:

- release object creation
- release identity
- release lifecycle state
- release artifact-set linkage
- release evidence linkage

- release signature linkage
- publish lifecycle
- revoke lifecycle
- release visibility/query surfaces
- release status inspection

The Release Manager does not govern:

- whether policy allows publish in abstract terms
- whether approvals are required in abstract terms
- artifact pin truth
- canonical run persistence
- audit storage mechanics
- artifact lineage truth itself

Those are handled by adjacent roles.

---

5. Core Principle

The Release Manager is the **release-record authority**, not the full governance engine and not the artifact-integrity engine.

It preserves the official lifecycle and evidence record for a release.

This means:

- it creates and manages release objects
- it records publish/revoke state
- it links release to relevant artifacts and evidence

- it does not invent policy clearance
- it does not invent approval truth
- it does not replace pin verification
- it does not replace artifact lineage governance

It is the formal release-evidence layer for governed outputs.

---

6. Primary Responsibilities

6.1 Create release records

- create stable release IDs
- define release scope
- bind release to artifact set or release payload
- preserve release metadata

6.2 Maintain release lifecycle state

- preserve draft/pending/approved/published/revoked or equivalent states
- preserve state transitions
- preserve publish/revoke timestamps

6.3 Preserve release evidence linkage

- link artifacts included in release
- link signatures where applicable
- link approvals where applicable

- link related run/release provenance refs

6.4 Support publish and revoke operations

- mark release published when all required conditions are satisfied
- mark release revoked when valid revoke path is used
- preserve reason metadata for lifecycle changes

6.5 Support release inspection

- expose release status
- expose release contents
- expose release evidence completeness/readiness fields where required
- support release lookup and listing

---

7. Authority

The Release Manager has authority to:

- create release objects
- maintain release lifecycle state
- preserve release evidence references
- record publish and revoke lifecycle outcomes
- expose official release status

The Release Manager does not have authority to:

- approve publish without required governance signals

- fabricate signatures
- fabricate artifact integrity
- redefine artifact identity
- rewrite audit truth
- silently treat incomplete release evidence as valid publish state

It governs release records, not all conditions that make a release acceptable.

---

8. Inputs

The Release Manager consumes:

8.1 Release inputs

- release creation request
- target artifact set
- release metadata
- publish request
- revoke request

8.2 Evidence inputs

- artifact references
- artifact registry refs
- integrity/pin verification refs
- approval refs
- override refs if applicable

- signature refs
- policy result refs where relevant

8.3 Governance inputs

- authorization result
- approval satisfaction
- policy clearance
- protected-action classification

8.4 Provenance inputs

- originating run refs
- packaging refs
- lineage refs where relevant
- release replacement/supersession refs if supported

---

9. Outputs

The Release Manager produces:

9.1 Release records

- draft release
- pending release
- published release
- revoked release

- superseded release if supported

9.2 Release lifecycle results

- publish success/denial
- revoke success/denial
- readiness incomplete
- approval missing
- integrity unresolved

9.3 Release inspection outputs

- release status
- included artifact refs
- evidence completeness/readiness view
- release timeline

---

10. Primary Interfaces

The Release Manager should expose, at minimum:

10.1 `createRelease`

Create a new governed release object.

10.2 `getRelease`

Return a release by ID.

10.3 `listReleases`

Return releases by scope, state, or target object.

10.4 `signRelease`

Attach or record release signature state where supported.

10.5 `publishRelease`

Mark a release as published when required conditions are satisfied.

10.6 `revokeRelease`

Revoke a published or pending release according to explicit rules.

10.7 `getReleaseStatus`

Return structured lifecycle status for a release.

10.8 `explainReleaseState`

Return readable summary of release readiness or lifecycle state.

---

11. Release Object Model

The Release Manager must require an explicit release object model.

At minimum, a release record should include:

- release_id
- release scope
- created_by
- created_at
- lifecycle state
- artifact set refs

- originating run refs where relevant
- signature refs where applicable
- approval refs where applicable
- publish metadata
- revoke metadata
- reason/summary metadata
- evidence completeness or readiness fields where supported

Releases must be first-class governed objects, not just folders or tags.

---

12. Release State Rules

The Release Manager must define explicit lifecycle states.

Examples may include:

- draft
- pending
- approved
- publishable
- published
- revoked
- superseded

The exact set may vary, but states must be explicit and machine-usable.

The role must also define:

- which transitions are legal
- which states are terminal
- how publish/revoke transitions are represented
- how supersession behaves if supported

---

13. Publish Rules

The Release Manager must define what it means to publish.

At minimum it must answer:

- what lifecycle state a release must be in before publish
- whether approval is required
- whether signature is required
- whether integrity verification is required
- what happens when evidence is incomplete
- whether publish is atomic or staged
- how publish time and actor are recorded

A release must not become published through informal assumption.

---

14. Revoke Rules

The Release Manager must define revocation behavior.

It must answer:

- who may revoke

- which release states are revocable
- whether revocation requires approval
- how revocation affects downstream visibility
- whether revoked releases remain historically visible
- how revocation reason is recorded

Revocation must be explicit and inspectable.

---

15. Signature Rules

If the system supports release signing, the Release Manager must define:

- what a signature record is
- who may sign
- whether multiple signatures are required
- how signature validity is checked or linked
- whether signature expiry or invalidation is possible
- whether publish depends on signature completion

If signing is not supported, that should be stated explicitly.

The Release Manager stores or links signature truth; it does not fabricate it.

---

16. Evidence Completeness Rules

The Release Manager should define release readiness/evidence completeness expectations.

This may include:

- required artifact set completeness
- required pin or integrity results
- required approval linkage
- required signature linkage
- required provenance linkage
- required release metadata completeness

The exact release gate may depend on governance policy, but the Release Manager must preserve evidence state clearly.

---

17. Relationship to Pin Manager

The Pin Manager governs artifact integrity anchors.

The Release Manager may depend on pin verification outcomes to determine release readiness or integrity posture.

In simple terms:

- Pin Manager = do included artifacts match expected integrity references?
- Release Manager = preserve that as part of the release evidence picture

The Release Manager must not invent integrity truth on its own.

---

18. Relationship to Artifact Registry / Lineage Governor

The Artifact Registry / Lineage Governor defines artifact identity and provenance.

The Release Manager binds a release to a governed set of artifacts and related provenance references.

In simple terms:

- Artifact Registry = what these artifacts are
- Release Manager = this release includes these artifacts

The Release Manager depends on registry truth rather than replacing it.

---

19. Relationship to Policy / Approval / AuthZ / Override Roles

Publishing or revoking a release may require:

- policy clearance
- approval
- authorization
- override in exceptional cases

The Release Manager consumes those governance outputs but does not replace them.

In simple terms:

- governance roles decide whether publish/revoke is allowed
- Release Manager records the governed release lifecycle result

---

20. Relationship to Audit Logger

Release actions are highly evidence-sensitive and must be audit-linked.

The Release Manager must ensure audit coverage for:

- release creation
- signature attach/update

- publish
- revoke
- readiness denial/block
- supersession if supported

The audit trail should preserve:

- actor or system source
- action
- target release
- included artifact refs or scope
- resulting release state
- reason metadata
- AI participation metadata where applicable

---

21. AI Assistance Boundary

The Release Manager may be AI-assisted in bounded ways.

AI may help with:

- summarizing release readiness
- drafting release review packets
- highlighting likely missing evidence
- clustering repeated revoke causes
- producing operator-friendly release timelines

AI may not:

- publish releases by itself
- revoke releases by itself
- fabricate signatures
- treat incomplete evidence as complete
- override deterministic release-state rules

The Release Manager remains the authoritative release-record role.

---

22. Failure Handling

The Release Manager must fail safely.

22.1 Missing artifact set

Do not create or publish ambiguous release content without explicit rule support.

22.2 Missing required governance signals

Do not publish/revoke until required signals exist.

22.3 Missing integrity/signature/evidence linkage

Preserve not-ready or blocked state rather than silently treating release as valid.

22.4 Invalid revoke request

Reject revoke request and preserve reason.

22.5 AI summarization failure

Ignore AI assistance and preserve deterministic release truth.

---

23. Required Guarantees

The Release Manager must guarantee:

- releases are explicit first-class objects
- release lifecycle state is inspectable
- publish/revoke transitions are explicit
- release evidence linkage is preserved
- incomplete release evidence does not silently become publish truth
- release behavior remains safe without AI

---

24. Success Criteria

The Release Manager is correctly established when:

- releases are created and tracked as governed objects
- publish and revoke state are explicit
- included artifacts and evidence refs are inspectable
- downstream roles can consume release truth cleanly
- release actions are auditable
- AI remains advisory and non-authoritative for release truth

---

25. Summary Statement

The Release Manager is the release-evidence role within the Evidence Control Plane.

It creates, tracks, and governs release objects as explicit trust-bearing records, preserving lifecycle state, artifact linkage, and publish/revoke evidence through deterministic rules.

It may use AI for readiness and summarization support, but it does not delegate release authority to AI.

ECP-04 — Artifact Registry / Lineage Governor

ECP-04 — Artifact Registry / Lineage Governor

1. Purpose

The Artifact Registry / Lineage Governor is the role responsible for **governing artifact identity, artifact metadata, provenance, and lineage relationships** inside the Evidence Control Plane.

Its purpose is to ensure that artifacts are not treated as anonymous files or loose outputs. Instead, they exist as explicit, queryable, trust-bearing system objects with known origin, known relationships, and known placement in the broader build and release graph.

The Artifact Registry / Lineage Governor governs **artifact truth and provenance structure**.

It is the role that turns “this file came from somewhere and is used somewhere” into explicit system knowledge.

---

2. Objective

The objective of the Artifact Registry / Lineage Governor is to ensure that:

- artifacts are first-class governed objects
- artifact identity is explicit
- producer relationships are explicit
- consumer relationships are explicit
- lineage can be followed across runs, stages, packaging, and release surfaces
- provenance gaps are surfaced clearly
- downstream roles can rely on artifact references instead of loose path assumptions
- AI may assist explanation and impact analysis, but may not become lineage authority

---

3. Problem It Solves

Without a dedicated artifact registry and lineage role, artifact knowledge tends to become fragmented across:

- folder structures

- filenames
- build scripts
- stage outputs
- packaging manifests
- release bundles
- operator memory

That fragmentation causes:

- unclear artifact identity
- weak provenance
- broken dependency visibility
- poor impact analysis
- weak trust in what output came from what step
- weak ability to explain how a release was assembled

The Artifact Registry / Lineage Governor solves this by making artifact identity and provenance explicit.

---

4. Scope

The Artifact Registry / Lineage Governor governs:

- artifact registration
- artifact identity
- artifact metadata
- artifact classification
- producer-stage linkage
- run linkage
- release linkage
- consumer and dependency linkage
- lineage edge creation and inspection
- provenance query surfaces
- artifact resolution by stable reference

The Artifact Registry / Lineage Governor does not govern:

- artifact integrity reference truth
- publish/revoke lifecycle truth
- general policy allow/deny logic
- canonical run persistence
- audit storage mechanics
- environment capability truth

Those are handled by adjacent roles.

---

5. Core Principle

The Artifact Registry / Lineage Governor is the **artifact identity and provenance authority**, not the artifact creator and not the integrity anchor.

It answers questions such as:

- what artifact is this?
- where did it come from?
- what produced it?
- what depends on it?
- what release includes it?
- what run or stage generated it?

This means:

- it preserves artifact identity
- it preserves lineage structure
- it preserves provenance links
- it does not itself hash artifacts as the source of integrity truth
- it does not itself publish releases
- it does not replace audit history

It is the formal artifact-knowledge layer for the system.

---

6. Primary Responsibilities

6.1 Register artifacts

- create stable artifact IDs
- preserve artifact type/class
- preserve locator/reference metadata
- preserve creation context

6.2 Maintain artifact identity

- ensure artifacts can be resolved by stable identity
- prevent uncontrolled ambiguity between artifacts

- support artifact version or instance distinction where required

6.3 Preserve provenance

- record producing run
- record producing stage
- record packaging or transformation source
- record upstream source refs where relevant

6.4 Preserve lineage relationships

- record parent/child relationships
- record producer/consumer relationships
- record dependency relationships
- record inclusion relationships for packaging and release contexts

6.5 Support artifact inspection

- resolve artifact by ID
- query artifacts by scope/type/state
- inspect artifact lineage graph
- expose provenance gaps or unresolved edges clearly

---

7. Authority

The Artifact Registry / Lineage Governor has authority to:

- create and manage artifact registry records
- define artifact identity structure
- define lineage edge structure
- resolve artifact references
- state what provenance and lineage data is currently known

The Artifact Registry / Lineage Governor does not have authority to:

- fabricate missing provenance as truth
- declare integrity match or mismatch by itself
- publish releases by itself
- rewrite audit truth
- mutate canonical run truth directly
- silently infer authoritative lineage from guesswork

It governs artifact identity and provenance, not every aspect of artifact trust.

---

8. Inputs

The Artifact Registry / Lineage Governor consumes:

8.1 Artifact creation inputs

- artifact creation event
- artifact locator/path/ref
- artifact class/type
- artifact metadata
- producing run/stage refs

8.2 Provenance inputs

- parent artifact refs
- upstream dependency refs
- transformation refs
- packaging refs
- release inclusion refs

8.3 Context inputs

- workspace/project scope
- run ID
- stage ID
- environment/package context where relevant
- artifact state metadata

8.4 Adjacent evidence inputs

- pin refs where applicable
- release refs where applicable
- audit refs where applicable

---

9. Outputs

The Artifact Registry / Lineage Governor produces:

9.1 Artifact records

- artifact registry entry
- artifact metadata record
- artifact type/class record
- artifact state summary where supported

9.2 Lineage records

- producer edge
- dependency edge
- inclusion edge
- transformation edge
- provenance graph view

9.3 Resolution outputs

- artifact found
- artifact unresolved
- lineage resolved
- lineage partial
- provenance missing
- dependency impact view

---

10. Primary Interfaces

The Artifact Registry / Lineage Governor should expose, at minimum:

10.1 `registerArtifact`

Create a new artifact record.

10.2 `getArtifact`

Return artifact record by ID or stable reference.

10.3 `listArtifacts`

Return artifacts by scope, type, run, stage, or release linkage.

10.4 `linkLineage`

Create or preserve lineage relationship between artifacts or artifact-bearing objects.

10.5 `getLineage`

Return lineage graph or lineage view for a target artifact.

10.6 `queryArtifacts`

Return artifacts matching query constraints.

10.7 `explainArtifactProvenance`

Return readable explanation of provenance/lineage state.

---

11. Artifact Record Model

The Artifact Registry / Lineage Governor must require an explicit artifact record model.

At minimum, an artifact record should include:

- artifact_id
- artifact type/class
- locator/reference
- scope
- producing run ref where relevant
- producing stage ref where relevant
- creation timestamp
- status/state where relevant
- related release refs where applicable
- related pin refs where applicable
- metadata summary
- provenance completeness status where supported

Artifacts must be first-class governed objects, not only filesystem paths.

---

12. Artifact Identity Rules

The Artifact Registry / Lineage Governor must define how artifact identity works.

It must answer:

- what makes an artifact unique

- whether identity is per-instance, per-version, or per-logical artifact
- how identical content in different scopes is handled
- how renamed or moved artifacts are tracked
- whether locator changes affect identity

Artifact identity must not rely solely on mutable path naming.

---

13. Classification Rules

Artifacts should be classified explicitly.

Examples may include:

- stage output artifact
- canonical document artifact
- build kit artifact
- verification report artifact
- packaged release artifact
- derived analysis artifact
- runtime manifest artifact

The exact taxonomy may vary, but classification must be structured enough to support routing, lineage, and inspection.

---

14. Provenance Rules

The Artifact Registry / Lineage Governor must define how provenance is recorded.

At minimum it must answer:

- what produced the artifact
- which run/stage or packaging action produced it
- whether source artifacts contributed to it
- whether provenance is complete, partial, or unknown
- how provenance gaps are represented

Missing provenance must be explicit rather than silently guessed.

---

15. Lineage Rules

The Artifact Registry / Lineage Governor must define explicit lineage edge types.

Examples may include:

- produced_by
- derived_from
- includes
- depends_on
- supersedes
- packaged_into
- released_as

The exact set may vary, but relationships must be structured and machine-usable.

The role must also define:

- whether lineage can be many-to-many
- whether cycles are allowed
- how unresolved refs are handled
- how lineage updates are versioned or tracked

---

16. Resolution Rules

The Artifact Registry / Lineage Governor must define how artifacts and lineage are resolved.

It must answer:

- how to resolve an artifact by ID
- how to resolve artifacts by run/stage/release scope
- how to return partial lineage
- how to represent unresolved references
- what happens when a lineage edge points to missing artifact truth

Resolution must be inspectable and deterministic.

---

17. Relationship to Pin Manager

The Pin Manager governs integrity anchors for artifacts.

The Artifact Registry / Lineage Governor governs artifact identity and provenance.

In simple terms:

- Artifact Registry / Lineage Governor = what artifact this is and how it relates
- Pin Manager = whether it matches expected integrity reference

The registry must not replace integrity truth.

---

18. Relationship to Release Manager

The Release Manager governs release objects and their lifecycle.

The Artifact Registry / Lineage Governor governs the artifact set that a release references.

In simple terms:

- Artifact Registry / Lineage Governor = what these artifacts are and where they came from
- Release Manager = this release includes these artifacts and is in this lifecycle state

These roles are linked but distinct.

---

19. Relationship to Audit Logger

Artifact registration and lineage changes are evidence-bearing actions and must be audit-linked.

The Artifact Registry / Lineage Governor must ensure audit coverage for:

- artifact registration
- lineage link creation
- lineage mutation
- provenance correction annotation where allowed
- unresolved/missing lineage events where materially important
- privileged artifact metadata updates

The audit trail should preserve:

- actor or system source
- action
- target artifact

- related lineage refs
- reason metadata
- AI participation metadata where applicable

---

20. Relationship to Execution and Governance Planes

The Execution Control Plane produces many artifacts and therefore provides producer context.

The Governance Control Plane may constrain who may mutate artifact registry data or when release inclusion is allowed.

But neither replaces the registry's role as artifact/provenance truth.

In simple terms:

- ICP = produces artifacts
- GCP = governs whether certain artifact-related actions are allowed
- ECP Artifact Registry = preserves artifact identity and lineage truth

---

21. AI Assistance Boundary

The Artifact Registry / Lineage Governor may be AI-assisted in bounded ways.

AI may help with:

- explaining lineage graphs
- summarizing dependency impact
- identifying likely provenance gaps for operator review
- clustering similar artifact derivation patterns
- producing readable artifact histories

AI may not:

- invent authoritative lineage
- mark unknown provenance as known truth
- rewrite artifact identity by itself
- collapse conflicting provenance into a fake clean answer
- overrule explicit registry truth

The Artifact Registry / Lineage Governor remains the authoritative artifact/provenance role.

---

22. Failure Handling

The Artifact Registry / Lineage Governor must fail safely.

22.1 Missing producer context

Register artifact with explicit incomplete provenance state if allowed, or reject according to explicit rule.

22.2 Unresolved lineage reference

Preserve unresolved reference explicitly rather than fabricating linkage.

22.3 Duplicate identity conflict

Reject or reconcile according to explicit identity rules.

22.4 Invalid lineage mutation

Reject mutation and preserve reason.

22.5 AI explanation failure

Ignore AI assistance and preserve deterministic registry truth.

---

23. Required Guarantees

The Artifact Registry / Lineage Governor must guarantee:

- artifacts are first-class explicit system objects
- artifact identity is stable and queryable
- provenance is explicit where known
- lineage relationships are structured and inspectable
- unresolved provenance is not silently fabricated
- registry behavior remains safe without AI

---

24. Success Criteria

The Artifact Registry / Lineage Governor is correctly established when:

- artifacts can be registered and resolved through stable identity
- producer/consumer and dependency relationships are explicit
- release and run surfaces can reference artifacts cleanly
- provenance gaps are visible
- downstream evidence roles can consume artifact truth consistently
- AI remains advisory and non-authoritative for artifact/provenance truth

---

25. Summary Statement

The Artifact Registry / Lineage Governor is the artifact-identity-and-provenance role within the Evidence Control Plane.

It creates and governs explicit artifact records and lineage relationships so the system can resolve what artifacts exist, where they came from, how they relate, and what downstream objects depend on them.

It may use AI for explanation and impact-analysis support, but it does not delegate artifact or lineage authority to AI.

ECP-05 — Notification Router

ECP-05 — Notification Router

1. Purpose

The Notification Router is the role responsible for **routing important control-plane events into explicit notification flows and preserving delivery evidence where required** inside the Evidence Control Plane.

Its purpose is to ensure that significant system events do not remain implicit or buried in raw state changes. Instead, they are turned into visible, governed event deliveries that can be inspected, retried, and linked back to the action or condition that triggered them.

The Notification Router governs **event-delivery evidence**.

It is the role that turns “this happened and someone/system should know” into an explicit routed notification record.

---

2. Objective

The objective of the Notification Router is to ensure that:

- important events become explicit notification objects
- notification routing follows defined rules
- delivery state is visible and inspectable
- retries and failures are governed explicitly
- notification evidence can be linked to underlying events
- downstream operators can distinguish emitted, delivered, failed, and retried notifications
- AI may assist prioritization and summarization, but may not become delivery truth

---

3. Problem It Solves

Without a dedicated notification role, event delivery tends to become fragmented across:

- scattered UI alerts
- controller printouts
- ad hoc emails or logs
- partial in-app messages
- hidden workflow assumptions
- operator memory about “someone should know about this”

That fragmentation causes:

- unclear whether important events were surfaced
- inconsistent alerting behavior
- weak retry handling
- poor evidence of whether delivery occurred
- weak operator trust in event visibility

The Notification Router solves this by making event-delivery flow explicit.

---

4. Scope

The Notification Router governs:

- notification-worthy event intake
- notification object creation
- notification classification
- channel/route selection

- delivery attempt tracking
- retry behavior
- delivery status tracking
- escalation linkage where supported
- notification query and inspection surfaces
- delivery evidence preservation

The Notification Router does not govern:

- whether an event should exist in the first place at the source-system level
- policy truth
- approval truth
- canonical run persistence
- artifact integrity truth
- audit storage mechanics

Those are handled by adjacent roles.

---

5. Core Principle

The Notification Router is the **event-delivery authority**, not the source of the underlying event and not the general policy engine.

It preserves how important events are surfaced and whether delivery succeeded, failed, was retried, or remained pending.

This means:

- it routes event notifications

- it tracks delivery lifecycle
- it preserves delivery evidence
- it does not invent event truth
- it does not decide policy approval by itself
- it does not replace the audit log

It is the formal notification-evidence layer for surfaced system events.

---

6. Primary Responsibilities

6.1 Accept notification-worthy events

- receive structured event input
- classify event type and severity
- bind notification to source event/object

6.2 Create notification records

- create stable notification IDs
- preserve event metadata
- preserve routing metadata
- preserve intended recipients/targets where applicable

6.3 Select notification routes

- choose delivery channel(s)
- choose route priority

- apply severity- or scope-based routing rules
- reject invalid route requests

6.4 Track delivery lifecycle

- preserve pending/sent/delivered/failed/retried state
- preserve timestamps and attempt counts
- preserve failure reasons where available

6.5 Retry and escalate when appropriate

- retry failed delivery according to explicit rules
- mark exhausted retries
- link escalation flows where applicable

6.6 Support notification inspection

- list notifications
- filter by status/event/severity/object
- expose source-event linkage
- expose delivery evidence

---

7. Authority

The Notification Router has authority to:

- create notification objects
- route notifications to configured channels

- record delivery attempts and outcomes
- retry notifications according to explicit rules
- expose notification and delivery state

The Notification Router does not have authority to:

- fabricate underlying system events
- redefine whether an event is allowed under policy
- claim delivery success without recorded delivery outcome
- rewrite audit truth
- replace approval or escalation authority
- silently suppress mandatory notifications outside explicit rules

It governs event routing and delivery evidence, not all event semantics.

---

8. Inputs

The Notification Router consumes:

8.1 Source-event inputs

- run failed event
- run gated event
- approval requested event
- approval expiring event
- override activated/revoked event
- policy denied event

- quota exhausted/throttled event
- release published/revoked event
- pin drift event
- lineage/provenance issue event
- privileged admin event where notification is required

8.2 Routing inputs

- notification class
- severity
- scope
- target audience or route group
- delivery channel rules
- retry rules

8.3 Delivery inputs

- channel delivery result
- delivery acknowledgement if available
- failure reason
- retry timing

8.4 Governance inputs

- whether event is mandatory, optional, or suppressed by rule
- authorization constraints on sensitive notification surfaces where relevant

9. Outputs

The Notification Router produces:

9.1 Notification objects

- pending notification
- delivered notification
- failed notification
- retried notification
- exhausted notification
- cancelled/suppressed notification where explicitly allowed

9.2 Delivery records

- delivery attempt record
- delivery success record
- delivery failure record
- retry record
- escalation linkage record where supported

9.3 Inspection outputs

- notification status
- notification timeline
- source-event linkage
- severity/routing summary

---

10. Primary Interfaces

The Notification Router should expose, at minimum:

10.1 `emitNotification`

Create and route a notification from a source event.

10.2 `getNotification`

Return notification record by ID.

10.3 `listNotifications`

Return notifications by scope, status, severity, or source event type.

10.4 `routeNotification`

Apply routing rules and initiate delivery.

10.5 `recordDeliveryResult`

Record outcome of a delivery attempt.

10.6 `retryDelivery`

Retry notification delivery according to explicit rules.

10.7 `getDeliveryStatus`

Return structured delivery state for a notification.

10.8 `explainNotificationState`

Return readable explanation of routing/delivery state.

---

11. Notification Object Model

The Notification Router must require an explicit notification object model.

At minimum, a notification record should include:

- notification_id
- source_event_type
- source_event_ref
- target object refs
- severity
- route/channel class
- intended recipients or target surface
- created_at
- current delivery state
- attempt count
- last attempt time
- failure reason where applicable
- retry metadata
- escalation metadata where applicable

Notifications must be first-class governed objects, not transient side effects only.

---

12. Event Classification Rules

The Notification Router must define how events are classified.

Classification may depend on:

- event type

- severity
- scope
- protected-action sensitivity
- operator audience
- system audience
- urgency
- required acknowledgement class if supported

Event classification must be structured and deterministic.

---

13. Routing Rules

The Notification Router must define how routes are selected.

It must answer:

- which events route to which channels or surfaces
- whether multiple channels may be used
- whether severity changes routing
- whether scope changes routing
- whether some routes are mandatory
- whether certain notifications are suppressible
- how fallback routes work

Routing must not rely on hidden convention.

---

14. Delivery State Rules

The Notification Router must define explicit delivery states.

Examples may include:

- pending
- routed
- sending
- delivered
- failed
- retry_scheduled
- exhausted
- cancelled
- suppressed

The exact set may vary, but states must be explicit and machine-usable.

---

15. Retry Rules

The Notification Router must define retry behavior.

At minimum it must answer:

- which failures are retryable
- how many retries are allowed
- retry timing/backoff rules
- when retry exhaustion occurs

- what state is used after exhaustion
- whether escalation happens after retry exhaustion

Retry must be governed, not ad hoc.

---

16. Escalation Rules

If escalation is supported, the Notification Router must define:

- which notification classes escalate
- what triggers escalation
- who or what receives escalation
- whether escalation creates new notification objects or linked child objects
- how escalation is tracked

If escalation is not supported for a class, that should be explicit.

---

17. Mandatory vs Optional Notification Rules

The Notification Router must define whether notifications are:

- mandatory
- optional
- suppressible under rule
- informational only
- acknowledgement-sensitive if supported

This matters because some events, such as protected-action failures or approval requirements, may need stronger routing guarantees than routine informational events.

---

18. Relationship to Audit Logger

The Audit Logger preserves historical evidence of actions.

The Notification Router preserves delivery evidence for surfaced events.

In simple terms:

- Audit Logger = the action or decision happened
- Notification Router = the notification about it was routed/delivered/failed

Both may reference the same source event, but they serve different purposes.

---

19. Relationship to Governance Roles

Governance roles often create notification-worthy events.

Examples:

- approval request created
- approval expiring
- override activated/revoked
- policy denial
- authz denial for protected action
- quota exhaustion

The Notification Router does not decide those governance truths. It routes the resulting notification objects and preserves delivery state.

---

20. Relationship to Execution Roles

Execution roles also create notification-worthy events.

Examples:

- run failed
- run gated
- run completed
- dispatch blocked
- environment unavailable

The Notification Router consumes those events and routes them according to notification rules.

---

21. Relationship to Release / Pin / Registry Roles

Evidence roles also create notification-worthy events.

Examples:

- release published/revoked
- pin drift detected
- lineage/provenance gap detected
- artifact registration anomaly

The Notification Router turns those evidence-bearing events into routed notifications with preserved delivery evidence.

---

22. AI Assistance Boundary

The Notification Router may be AI-assisted in bounded ways.

AI may help with:

- summarizing noisy notification streams
- prioritizing notification review order for operators
- clustering repeated failures
- suggesting likely escalation paths for deterministic validation
- generating readable notification summaries

AI may not:

- mark failed delivery as delivered
- suppress mandatory notifications by itself
- invent source events
- overrule deterministic routing rules
- fabricate acknowledgement or delivery success

The Notification Router remains the authoritative notification-delivery role.

---

23. Failure Handling

The Notification Router must fail safely.

23.1 Missing route definition

Fail explicitly or preserve unroutable state rather than silently dropping the event.

23.2 Delivery failure

Record failed delivery and apply retry or exhaustion rules explicitly.

23.3 Retry exhaustion

Mark exhausted state and escalate if rules require it.

23.4 Invalid source event reference

Reject notification creation or preserve invalid-source state according to explicit rules.

23.5 AI summarization/prioritization failure

Ignore AI assistance and preserve deterministic delivery truth.

---

24. Required Guarantees

The Notification Router must guarantee:

- notification-worthy events can become explicit notification objects
- routing rules are deterministic
- delivery state is inspectable
- retries are governed explicitly
- failed or unroutable notifications are not silently treated as delivered
- notification behavior remains safe without AI

---

25. Success Criteria

The Notification Router is correctly established when:

- important events are routable through explicit notification objects
- delivery state is visible and queryable
- retry and exhaustion behavior are explicit
- downstream operators can inspect whether a notification was delivered or failed
- notification delivery evidence is preserved distinctly from audit history

- AI remains advisory and non-authoritative for delivery truth

---

26. Summary Statement

The Notification Router is the event-delivery-evidence role within the Evidence Control Plane.

It creates, routes, tracks, retries, and exposes notifications as explicit governed objects so important system events can be surfaced with inspectable delivery evidence.

It may use AI for summarization and prioritization support, but it does not delegate notification or delivery authority to AI.

AIG Artifact Pack

AIG-00 — Purpose + Boundaries

1. Purpose

The AI Governance layer (AIG) defines the **rules, limits, contracts, and audit expectations for AI assistance inside Axion**.

Its purpose is to ensure that AI can help the control planes without becoming the source of authority.

AIG exists to govern:

- where AI may participate
- what AI may do
- what AI may not do
- what context AI may receive
- what outputs AI must return
- how AI output is validated
- how AI participation is recorded

AIG governs **AI operating boundaries**.

It is the layer that turns “AI can help here” into a formal, safe, inspectable system rule set.

---

2. Objective

The objective of AIG is to ensure that:

- AI assistance is bounded explicitly
- AI does not replace deterministic control-plane authority

- AI participation is structured, predictable, and reviewable
- AI outputs are machine-usable where needed
- AI outputs cannot silently become system truth without validation
- protected actions remain governed by deterministic and human-controlled paths where required
- AI failures degrade safely
- AI usage is auditable

---

3. Problem It Solves

Without an AI governance layer, AI use tends to become fragmented across:

- ad hoc prompts
- hidden assistant calls
- inconsistent tool usage
- unclear operator expectations
- undocumented automation behavior
- unbounded model outputs

That fragmentation causes:

- authority confusion
- unclear AI responsibility boundaries
- unsafe action automation
- weak auditability of AI influence

- inconsistent prompt/context handling
- poor operator trust in AI-assisted decisions

AIG solves this by making AI participation explicit.

---

4. What AIG Governs

AIG governs:

- AI decision scope
- AI authority limits
- AI input boundaries
- AI output contracts
- AI action gating
- AI validation expectations
- AI audit and replay requirements
- model routing and fallback rules
- confidence handling
- AI failure behavior

---

5. What AIG Does Not Govern

AIG does **not** govern:

- execution-state legality
- policy truth

- actor authorization truth
- approval truth
- artifact integrity truth
- release truth
- canonical run truth
- audit truth itself

Those remain owned by ICP, GCP, and ECP.

AIG only governs **how AI may assist** those planes.

---

6. Core Principle

AIG is based on this rule:

AI may assist authority.
AI may not replace authority.

That means:

- AI can recommend
- AI can classify
- AI can summarize
- AI can explain
- AI can draft structured proposals
- AI can support operators

But:

- AI does not become policy truth

- AI does not become approval authority
- AI does not become permission authority
- AI does not become audit truth
- AI does not become transition legality
- AI does not become integrity truth

---

7. Governing Model

AIG follows this model:

bounded request → bounded context → bounded model task → structured output → deterministic validation → governed use or rejection

This model ensures that AI output is treated as:

- advisory
- propose-and-validate
- or narrowly auto-eligible only where explicitly allowed

It prevents AI from being treated as raw authority.

---

8. Primary Responsibilities

AIG is responsible for defining:

8.1 AI scope rules

- where AI may be called
- which roles may use AI

- which task classes AI may support

8.2 AI authority limits

- what AI may never do
- what AI may only do with validation
- what AI may do only in low-risk paths

8.3 AI contract rules

- what context AI may receive
- what format AI must return
- what metadata must accompany AI output

8.4 AI action-gating rules

- when AI output is advisory only
- when AI output may be validated into action
- when human review is mandatory

8.5 AI audit rules

- what must be logged
- how AI participation is replayed or reviewed
- how AI-influenced actions remain inspectable

8.6 AI failure rules

- how to behave when AI is unavailable

- how to behave when AI confidence is low
- how to behave when AI output is malformed or conflicting

---

9. AIG Artifact Set

The AI Governance layer is composed of these artifacts:

AIG-01 AI Decision Scope

Defines where AI may operate.

AIG-02 AI Authority Limits

Defines what AI may and may not do.

AIG-03 AI Input Contract

Defines what context AI receives.

AIG-04 AI Output Contract

Defines what AI must return.

AIG-05 AI Action Gating

Defines how AI output may or may not become action.

AIG-06 AI Audit + Replay Rules

Defines how AI participation is recorded and reviewed.

AIG-07 Model Routing + Fallback Policy

Defines which model is used when, and how fallback behaves.

---

10. Inputs

AIG consumes the following conceptual input classes:

10.1 Control-plane role context

- ICP role usage needs
- GCP role usage needs
- ECP role usage needs

10.2 AI task classes

- summarization
- classification
- recommendation
- anomaly review
- operator assistance
- structured proposal generation

10.3 Risk context

- protected-action class
- low-risk vs high-risk usage
- evidence sensitivity
- governance sensitivity
- operator review requirements

10.4 Model capability context

- model strengths
- output reliability

- reasoning depth
- structured output suitability
- fallback model availability

---

11. Outputs

AIG produces:

11.1 AI governance rules

- allowed task surfaces
- prohibited task surfaces
- authority boundaries
- gating classes

11.2 AI contracts

- request envelope rules
- response envelope rules
- context restrictions
- output validation requirements

11.3 AI operational signals

- advisory-only classification
- human-review-required classification
- validation-required classification

- fallback-required classification

11.4 AI audit requirements

- trace metadata requirements
- replay requirements
- model selection visibility requirements

---

12. Relationship to ICP

AIG governs how AI may assist the Execution Control Plane.

Examples:

- failure classification
- retry suggestions
- environment recommendation
- queue-pressure explanation

AIG does not allow AI to replace:

- Run Controller
- State Machine Governor
- Queue / Scheduler Manager
- Adapter / Environment Manager

---

13. Relationship to GCP

AIG governs how AI may assist the Governance Control Plane.

Examples:

- policy explanation
- approval packet summarization
- override risk summarization
- authz anomaly review
- quota pressure forecasting

AIG does not allow AI to replace:

- Policy Engine
- Approval Router
- Override Manager
- AuthZ Manager
- Quota Manager

---

14. Relationship to ECP

AIG governs how AI may assist the Evidence Control Plane.

Examples:

- audit summarization
- drift explanation
- release readiness summary
- lineage explanation

- notification stream summarization

AIG does not allow AI to replace:

- Audit Logger
- Pin Manager
- Release Manager
- Artifact Registry / Lineage Governor
- Notification Router

---

15. Authority Boundaries

AIG has authority to define:

- whether AI may be used for a given task class
- whether AI is advisory or validate-first
- whether human review is mandatory
- what models may be used
- how AI participation is logged

AIG does **not** have authority to:

- override the core role authority of ICP, GCP, or ECP
- redefine policy truth
- authorize protected actions
- bypass deterministic validation
- allow undocumented AI authority creep

---

16. Protected AI Boundaries

AIG must explicitly protect against AI being treated as authority in areas such as:

- approval decisions for protected actions
- permission grants
- override activation
- illegal state transitions
- integrity verification overrides
- release publish/revoke authority
- audit rewrite or correction authority
- hidden silent suppression of mandatory notifications

These are non-negotiable AI boundary classes unless the architecture is intentionally changed later.

---

17. Operating Modes

AIG should support these AI operating modes:

17.1 AI disabled

Control planes operate with no AI assistance.

17.2 Advisory-only mode

AI provides suggestions, summaries, and explanations only.

17.3 Validate-first mode

AI returns structured proposed outputs that must be validated before use.

17.4 Restricted low-risk automation mode

A small whitelisted set of low-risk AI-assisted actions may auto-apply after deterministic checks.

AIG must define which tasks belong in which mode.

---

18. Failure Handling Expectations

AIG must require safe degradation.

18.1 AI unavailable

Control planes continue without AI assistance.

18.2 AI malformed output

Reject AI output and log failure.

18.3 AI low confidence

Treat as advisory or escalate to human review.

18.4 AI conflicts with deterministic rules

Deterministic rules win.

18.5 AI model unavailable

Fallback model may be used only if routing policy allows it.

---

19. Required Guarantees

AIG must guarantee:

- AI participation is bounded explicitly
- AI does not silently become authority
- AI output is validated before governed use where required

- AI failures do not break control-plane safety
- AI participation is inspectable
- model choice and fallback behavior are governable
- operators can distinguish AI suggestion from system truth

---

20. Success Criteria

AIG is correctly established when:

- every AI-assisted surface has an explicit boundary
- every AI-assisted output path has a contract
- protected actions remain governed without AI authority creep
- AI usage is auditable
- AI can be disabled without breaking the core system
- model routing and fallback behavior are explicit
- operators can tell what came from AI vs deterministic system logic

---

21. Summary Statement

The AI Governance layer defines the rules that allow Axion to use AI safely inside the control planes.

It governs where AI may participate, what AI may return, how AI output is validated, and how AI influence is audited, while preserving ICP, GCP, and ECP as the actual authoritative system layers.

AIG-01 — AI Decision Scope

AIG-01 — AI Decision Scope

1. Purpose

The AI Decision Scope artifact defines **where AI is allowed to participate inside Axion and what classes of tasks AI is permitted to support**.

Its purpose is to prevent AI usage from spreading through the system as an undefined convenience layer.

AIG-01 governs **AI participation surfaces**.

It is the artifact that answers:

- where may AI be used?
- for what kinds of tasks?
- under what risk class?
- in which control-plane roles?
- in which operating mode?

---

2. Objective

The objective of AI Decision Scope is to ensure that:

- AI is only used in explicitly approved task surfaces
- AI support is mapped role-by-role
- high-risk surfaces are treated differently from low-risk surfaces
- operators and builders know exactly where AI may be called
- AI is not inserted into protected workflows informally

---

3. Problem It Solves

Without explicit AI decision scope, AI usage tends to drift into:

- hidden helper calls
- random summarization layers
- undocumented approval suggestions
- silent routing logic
- fragile reasoning shortcuts
- unreviewed automation paths

That creates:

- authority confusion
- inconsistent trust boundaries
- unclear audit coverage
- unpredictable AI influence on system behavior

AI Decision Scope solves this by making AI participation explicit.

---

4. Scope

This artifact governs:

- approved AI task classes
- disallowed AI task classes
- role-by-role AI usage surfaces

- risk classification for AI-supported tasks
- mode classification for each task surface
- AI participation boundaries for ICP, GCP, and ECP roles

This artifact does **not** govern:

- exact AI input schema
- exact AI output schema
- exact audit payload structure
- exact model routing logic

Those are governed by later AIG artifacts.

---

5. Core Principle

AI may only be used where the system explicitly says it may be used.

No implicit AI participation is allowed.

That means:

- no hidden AI calls in protected workflows
- no undocumented AI decision points
- no silent AI transformations of governed system truth

If a task surface is not explicitly listed as AI-allowed, it is AI-disallowed by default.

---

6. Decision Scope Model

Every AI-eligible task surface must be defined by:

- **plane**
- **role**
- **task class**
- **risk class**
- **AI mode**
- **required validation path**
- **human review requirement**
- **prohibited authority behaviors**

This creates a consistent scoped map of AI participation.

---

7. AI Task Classes

The following task classes are allowed to exist as scoped AI categories.

7.1 Summarization

Used to compress or explain structured system information.

Examples:

- audit summary
- release readiness summary
- policy denial explanation
- approval packet summary

7.2 Classification

Used to categorize a known input into structured classes.

Examples:

- failure type classification
- anomaly category classification
- drift severity suggestion
- notification priority suggestion

7.3 Recommendation

Used to suggest one or more next actions without executing them.

Examples:

- retry path recommendation
- environment recommendation
- approver path recommendation
- quota-aware defer recommendation

7.4 Structured Proposal Generation

Used to return a machine-usable proposed action payload for downstream validation.

Examples:

- proposed retry request
- proposed queue defer action
- proposed approval summary packet
- proposed release readiness checklist

7.5 Anomaly Review Support

Used to highlight suspicious or unusual patterns for operator review.

Examples:

- suspicious permission activity
- audit anomaly clusters
- repeated drift patterns
- unusual queue starvation signals

7.6 Operator Assistance

Used for human-facing explanation and workflow acceleration.

Examples:

- readable denial explanation
- lifecycle state explanation
- provenance explanation
- notification stream explanation

---

8. Disallowed AI Task Classes

The following are disallowed by default unless the overall architecture is explicitly redesigned.

8.1 Final Policy Authority

AI may not be the final source of allow/deny policy truth.

8.2 Final Approval Authority

AI may not approve protected actions as an authoritative approver.

8.3 Final Permission Authority

AI may not grant permissions or declare privileged access as final truth.

8.4 Final State Transition Authority

AI may not decide legal run or stage transitions.

8.5 Final Integrity Authority

AI may not decide whether integrity mismatch should be treated as valid.

8.6 Final Release Authority

AI may not publish or revoke releases by itself.

8.7 Final Audit Authority

AI may not add, rewrite, remove, or certify audit truth by itself.

8.8 Hidden Governance Substitution

AI may not be inserted as an undocumented replacement for deterministic role logic.

---

9. AI Scope by Plane

9.1 Execution Control Plane Scope

AI may assist the ICP in bounded ways.

Allowed ICP task classes

- failure summarization
- failure classification
- retry recommendation
- environment recommendation
- queue pressure explanation
- blocked-dispatch explanation
- operator-facing lifecycle explanation

Disallowed ICP task classes

- direct run-state mutation
- direct stage advancement authority
- direct legality override
- direct dispatch authority
- direct environment-truth override

9.2 Governance Control Plane Scope

AI may assist the GCP in bounded ways.

Allowed GCP task classes

- policy explanation
- denial summarization
- approval packet drafting
- override risk summarization
- authz anomaly review
- quota pressure explanation
- recommendation support for reviewers/operators

Disallowed GCP task classes

- policy pass/fail authority
- approval authority
- override activation authority

- permission grant authority
- quota override authority

9.3 Evidence Control Plane Scope

AI may assist the ECP in bounded ways.

Allowed ECP task classes

- audit summarization
- drift explanation
- release readiness summarization
- provenance explanation
- lineage impact explanation
- notification stream summarization
- anomaly clustering for evidence review

Disallowed ECP task classes

- audit entry truth mutation
- integrity-truth override
- release publish/revoke authority
- authoritative lineage fabrication
- delivery-success fabrication

---

10. AI Scope by Role

10.1 ICP roles

ICP-01 Run Controller

Allowed:

- failure classification
- operator summary drafting
- retry suggestion

Not allowed:

- lifecycle authority
- terminal-state authority
- transition bypass

ICP-02 Run State Store

Allowed:

- state inconsistency summarization
- storage anomaly explanation

Not allowed:

- canonical write authority
- truth repair without governed validation

ICP-03 State Machine Governor

Allowed:

- readable rejection explanation
- valid-next-step explanation

Not allowed:

- transition-law definition
- legality override

ICP-04 Queue / Scheduler Manager

Allowed:

- queue pressure forecasting
- retry timing recommendation
- starvation analysis

Not allowed:

- dispatch authority
- privileged reprioritization authority

ICP-05 Adapter / Environment Manager

Allowed:

- target recommendation among valid candidates
- capability mismatch explanation

Not allowed:

- target truth invention
- readiness override

10.2 GCP roles

GCP-01 Policy Engine

Allowed:

- readable policy explanation
- conflict summarization
- likely-applicable-policy suggestion for deterministic validation

Not allowed:

- policy truth authority
- policy override by AI

GCP-02 Approval Router

Allowed:

- approval packet summarization
- likely approver recommendation for deterministic validation

Not allowed:

- approval decision authority
- approver-authority invention

GCP-03 Override Manager

Allowed:

- override risk summarization

- missing evidence highlighting

Not allowed:

- override activation authority
- scope expansion authority

GCP-04 Permissions / AuthZ Manager

Allowed:

- readable denial explanation
- anomaly clustering for permission use

Not allowed:

- permission grant authority
- scope override authority

GCP-05 Quota Manager

Allowed:

- quota pressure forecasting
- exhaustion explanation
- defer-window recommendation

Not allowed:

- quota increase authority
- hard-limit bypass authority

10.3 ECP roles

ECP-01 Audit Logger

Allowed:

- audit summarization
- suspicious-sequence highlighting

Not allowed:

- audit mutation authority
- audit certification authority

ECP-02 Pin Manager

Allowed:

- drift explanation
- mismatch clustering

Not allowed:

- integrity override
- protected unpin authority

ECP-03 Release Manager

Allowed:

- release readiness summarization
- missing-evidence highlighting

Not allowed:

- publish authority
- revoke authority

ECP-04 Artifact Registry / Lineage Governor

Allowed:

- provenance explanation
- lineage impact summarization

Not allowed:

- authoritative lineage invention
- artifact-identity rewrite

ECP-05 Notification Router

Allowed:

- notification stream summarization
- prioritization suggestion for operator view

Not allowed:

- delivery truth override
- silent suppression of mandatory notifications

---

11. Risk Classes for AI Use

Each AI task surface must have a risk class.

11.1 Low-risk

AI output is explanatory or assistive and does not directly influence governed action.

Examples:

- readable summaries
- explanations
- operator-facing descriptions

11.2 Medium-risk

AI output may shape human decisions or be transformed into proposed structured action, but must be validated before use.

Examples:

- retry recommendation
- approver recommendation
- defer recommendation

11.3 High-risk

AI output touches protected-action context, governance-sensitive interpretation, or evidence-sensitive recommendations.

Examples:

- override risk review
- release readiness synthesis
- policy conflict synthesis for protected actions

High-risk AI surfaces must never skip deterministic validation and may require mandatory human review.

---

12. AI Operating Modes by Scope

Each AI task surface must be assigned an operating mode.

12.1 Advisory-only

AI provides information, explanation, or suggestions only.

12.2 Validate-first

AI produces structured output that must pass deterministic validation before any governed use.

12.3 Human-review-required

AI output may support decision-making, but a human must explicitly accept or act on it.

12.4 Restricted low-risk automation

Only explicitly whitelisted low-risk tasks may auto-apply after deterministic checks.

Default mode is:
advisory-only unless explicitly upgraded.

---

13. Default AI Scope Rules

Unless explicitly overridden by a later artifact:

- AI use is **disallowed by default**
- allowed AI use is **advisory-only by default**
- protected-action-related AI outputs are **human-review-required by default**
- AI may not directly mutate governed truth
- AI may not act as final authority in any control-plane role

These defaults exist to prevent silent scope expansion.

---

14. Human Review Rules

Human review is mandatory when AI output affects:

- protected actions
- approval context
- override recommendation context
- release publish/revoke readiness
- permission-sensitive interpretation
- evidence-sensitive ambiguity
- integrity or provenance ambiguity

Human review may be optional only for low-risk explanatory surfaces.

---

15. Validation Rules

AI output may only be used when the correct downstream deterministic validation path exists.

Examples:

- retry recommendation must still pass lifecycle and state checks
- approver recommendation must still pass approver resolution rules
- release-readiness summary must not replace actual release evidence checks
- drift explanation must not replace pin verification
- permission explanation must not replace authz result

AI Decision Scope only grants participation. It never grants direct authority.

---

16. Prohibited Scope Expansion Rules

The system must reject:

- AI inserted into new roles without artifact update
- AI moved from advisory to validate-first without governance update
- AI upgraded into protected-action workflows without explicit approval
- AI outputs being treated as truth because they are convenient
- AI calls added to hidden internal helpers outside scoped task surfaces

Any new AI surface requires explicit AIG update.

---

17. Audit Expectations

Every scoped AI surface must be auditable enough to answer:

- which role invoked AI
- for what task class
- in what mode
- with what risk class
- whether the output was used, ignored, or rejected
- whether human review was required

Full trace structure is defined later, but scope-level auditability is mandatory.

---

18. Failure Handling Expectations

If an AI surface fails:

- the surface may be skipped

- downstream deterministic roles remain authoritative
- protected workflows remain blocked unless all normal requirements are met
- the system may fall back to non-AI operation
- no AI failure may silently convert into implicit permission to proceed

AI scope does not create dependency on AI existence.

---

19. Required Guarantees

AI Decision Scope must guarantee:

- every AI-assisted task surface is explicit
- disallowed surfaces are explicit
- protected workflows are not implicitly AI-enabled
- risk and mode are defined per task surface
- control planes remain authoritative even where AI participates

---

20. Success Criteria

AIG-01 is correctly established when:

- each control-plane role has explicit AI-allowed and AI-disallowed task surfaces
- task classes are standardized
- risk classes exist
- operating modes exist
- default disallow/advisory rules are clear

- future AI expansion requires deliberate governance updates

---

21. Summary Statement

AI Decision Scope defines where AI may operate inside Axion, what task classes it may support, and what it is forbidden from doing in each control-plane role.

It ensures that AI participation remains explicit, bounded, risk-classified, and subordinate to deterministic system authority.

AIG-02 — AI Authority Limits

AIG-02 — AI Authority Limits

1. Purpose

The AI Authority Limits artifact defines **the hard boundaries on what AI is not allowed to decide, approve, mutate, or certify inside Axion**.

Its purpose is to prevent AI assistance from turning into AI authority.

AIG-02 governs **non-delegable system authority boundaries**.

It is the artifact that answers:

- what may AI never do?
- what may AI do only after deterministic validation?
- what may AI do only with human review?
- what authority must always remain with the control planes and authorized humans?

---

2. Objective

The objective of AI Authority Limits is to ensure that:

- AI never becomes the hidden governor of the system
- protected actions remain under deterministic and human-controlled authority
- core truth-bearing system records cannot be overwritten by AI
- AI cannot silently bypass policy, approval, authz, integrity, or audit requirements
- operators can trust that AI assistance has hard ceilings

---

3. Problem It Solves

Without explicit authority limits, AI systems tend to drift from:

- explanation
- recommendation
- summarization

into:

- de facto approval
- silent routing control
- hidden policy interpretation authority
- implied truth rewriting
- operational bypass behavior

That creates:

- authority confusion
- unsafe automation
- weak auditability
- trust erosion
- governance failure

AI Authority Limits solves this by defining explicit “cannot cross” boundaries.

---

4. Scope

This artifact governs:

- non-delegable authorities
- protected-action AI restrictions
- truth-mutation restrictions
- validation-before-use requirements
- human-review-required classes
- prohibited direct-action classes
- prohibited override/bypass behaviors

This artifact does **not** define:

- the full AI request schema
- the full AI response schema
- exact model routing rules
- exact audit payload format

Those belong to later AIG artifacts.

---

5. Core Principle

AI assistance is subordinate to system authority.

If there is any conflict between:

- AI output
- deterministic control-plane rules
- required human approval
- required evidence truth

then:

- deterministic rules win
- required human authority wins
- evidence truth wins
- AI output loses

This is the central non-negotiable rule.

---

6. Authority Classes

The system must distinguish between these authority classes.

6.1 Deterministic system authority

Authority held by ICP, GCP, and ECP roles through explicit rules and contracts.

6.2 Human authority

Authority held by authorized human actors such as approvers, operators, reviewers, or admins within governed rules.

6.3 AI assistance

Non-authoritative support used for recommendation, explanation, classification, and structured proposal generation.

AI may assist classes 1 and 2.
AI may not replace classes 1 and 2.

---

7. Non-Delegable Authorities

The following authorities are non-delegable to AI unless the entire system architecture is intentionally redesigned.

7.1 Policy authority

AI may not be the final source of allow/deny policy truth.

7.2 Approval authority

AI may not act as final approver for protected actions.

7.3 Authorization authority

AI may not grant permissions, roles, or privileged access.

7.4 State-transition legality authority

AI may not determine legal run or stage transitions as final truth.

7.5 Execution mutation authority

AI may not directly mutate canonical run or stage state.

7.6 Integrity authority

AI may not decide that a mismatch should be treated as valid.

7.7 Release publish/revoke authority

AI may not publish or revoke governed releases by itself.

7.8 Audit authority

AI may not create fake audit history, delete audit history, or certify audit truth by itself.

7.9 Provenance authority

AI may not invent authoritative lineage or provenance where evidence is missing.

7.10 Mandatory notification suppression authority

AI may not suppress required notifications by itself.

These are hard system boundaries.

---

8. Prohibited Direct AI Actions

AI must be prohibited from directly performing these actions as an authoritative actor:

- approve protected action requests
- activate overrides
- grant privileged permissions
- bypass quota hard limits
- bypass policy denial
- bypass authz denial
- bypass evidence requirements
- rewrite canonical run records
- rewrite audit entries
- mark failed integrity checks as passed
- mark undelivered notifications as delivered
- force illegal lifecycle transitions
- publish or revoke releases
- unpin protected artifacts without governed authorization
- fabricate missing lineage as confirmed truth

If an AI output appears to do one of these, the system must reject it.

---

9. Restricted AI Actions

Some actions are not fully prohibited, but they are restricted.

These may only occur if all explicit validation rules are satisfied.

9.1 Structured proposal generation

AI may propose structured actions, but the system must validate them.

9.2 Low-risk operational suggestions

AI may suggest defer windows, retry paths, target candidates, and explanations.

9.3 Risk summaries for human review

AI may summarize protected-action context for humans, but not decide it.

9.4 Candidate ranking

AI may rank already-valid options, but only among options the deterministic system has already declared valid.

Restricted AI actions are never raw authority.

---

10. Validation-Before-Use Rule

Any AI output that may influence a governed action must pass all relevant deterministic checks before use.

Examples:

- retry recommendation → must pass Run Controller + State Machine + policy/authz if needed
- environment recommendation → must pass Adapter/Environment validation
- approval route suggestion → must pass Approval Router + AuthZ checks
- release readiness summary → must not replace actual release evidence checks
- drift explanation → must not replace Pin verification
- override risk summary → must not activate override

AI output may influence workflow, but never skip validation.

---

11. Human-Review-Required Classes

Human review is mandatory for AI outputs that influence:

- protected action execution
- approval decisions
- override activation or scope interpretation
- permission-sensitive decisions
- release publish/revoke readiness in a decisive way
- integrity ambiguity with operational consequences
- provenance ambiguity with governance or release consequences
- any action with legal, security, or irreversible operational consequence

These classes may use AI for support, but the final decision must not be made by AI alone.

---

12. AI Authority by Risk Class

12.1 Low-risk

AI may provide explanations or summaries.
No authority transfer.
Human review optional.

12.2 Medium-risk

AI may provide structured proposals or recommendations.
Deterministic validation mandatory.
Human review often required depending on surface.

12.3 High-risk

AI may assist with summary and analysis only.
Deterministic validation mandatory.
Human review mandatory.
AI may not be action-authoritative.

The default for unclear cases is **treat as higher risk**.

---

13. Plane-Level Authority Limits

13.1 ICP authority limits

AI may not:

- advance a run by authority alone
- complete/fail/cancel a run by authority alone
- legalize an illegal transition
- dispatch work by authority alone
- route work to unsupported environments
- modify canonical execution truth directly

AI may:

- explain failure state
- recommend retry path
- rank valid environment options
- summarize blocked dispatch reasons

13.2 GCP authority limits

AI may not:

- issue final policy pass/fail
- approve protected actions
- authorize actors
- activate overrides
- grant quota exceptions by itself
- waive separation-of-duties rules

AI may:

- summarize policy conflicts
- summarize approval packets
- summarize override risk
- explain authz denials
- forecast quota pressure

13.3 ECP authority limits

AI may not:

- write or rewrite audit truth
- declare integrity mismatch acceptable
- publish or revoke releases
- fabricate lineage
- mark missing evidence as complete

- mark failed notification delivery as successful

AI may:

- summarize audit history
- explain drift findings
- summarize release readiness
- explain provenance gaps
- summarize notification stream health

---

14. Candidate-Set Rule

When AI is allowed to recommend among options, the option set must already be constrained by deterministic logic.

Examples:

- environment recommendation must choose among valid candidate targets
- queue reprioritization suggestion must operate within allowed priority rules
- approver recommendation must choose among valid approver candidates
- release summary may highlight evidence, but not redefine what counts as required evidence

AI may rank valid options.
AI may not invent valid options.

---

15. No Silent Escalation Rule

AI authority level may not silently escalate over time.

That means:

- advisory surfaces may not become validate-first without artifact update
- validate-first surfaces may not become auto-action surfaces without explicit approval
- low-risk automation may not expand to protected actions without architecture change
- hidden internal helpers may not introduce new AI action paths

Any authority expansion requires deliberate governance update.

---

16. No Hidden Bypass Rule

AI may not be used as a convenience path to bypass:

- policy blockers
- approval blockers
- authz blockers
- quota blockers
- integrity blockers
- evidence blockers
- missing-context blockers

If a deterministic role says “blocked,” AI may explain that, but AI may not convert blocked into allowed.

---

17. No Truth Fabrication Rule

AI may not convert uncertainty into authoritative truth.

Examples of prohibited fabrication:

- missing audit event treated as if it happened
- missing lineage edge treated as proven
- missing approval treated as implied approval
- missing permission treated as implied permission
- missing evidence treated as likely sufficient
- missing artifact integrity reference treated as acceptable

AI may discuss uncertainty, but may not erase it.

---

18. Low-Risk Automation Limits

If the system supports restricted low-risk AI automation, it must remain bounded.

Permitted low-risk examples may include:

- formatting an operator-facing summary
- grouping notification noise for display
- generating a proposed retry request payload
- drafting a release review summary

Even then:

- deterministic validation remains required if the output affects action
- no protected action may be auto-authorized by AI
- no evidence truth may be auto-modified by AI

Low-risk automation is not general AI action authority.

---

19. Operator Override Rule

Human operators do not gain permission to “make AI authoritative” outside system rules just by trusting it.

That means an operator may not:

- treat AI output as approval without approval object/state
- treat AI output as policy truth without policy evaluation
- treat AI output as authz truth without authz result
- treat AI output as evidence truth without evidence verification

This prevents AI from becoming authority through social habit instead of architecture.

---

20. Conflict Resolution Rule

When AI output conflicts with system truth:

- canonical system state wins over AI description
- deterministic policy result wins over AI interpretation
- authz result wins over AI suggestion
- integrity verification result wins over AI explanation
- audit record wins over AI reconstruction
- required human approval wins over AI recommendation

This rule must be universal and explicit.

---

21. Failure Handling Rules

If AI produces:

- malformed output
- low-confidence output
- contradictory output
- risky output outside its allowed scope
- output implying prohibited authority

then the system must:

- reject or ignore that output
- preserve deterministic safety
- log or trace the failed AI participation where required
- require normal validation and human review paths

AI failure must never widen authority.

---

22. Audit Expectations

Authority-limit enforcement must be inspectable.

For meaningful AI-assisted paths, the system should be able to answer:

- what AI was allowed to do on that surface
- what AI was forbidden to do
- whether AI output remained within scope
- whether validation and/or human review occurred
- whether AI output was rejected because it crossed an authority boundary

This ensures boundary violations are visible, not hidden.

---

23. Required Guarantees

AI Authority Limits must guarantee:

- non-delegable authorities remain non-delegable
- protected actions cannot become AI-authoritative by drift
- deterministic validation remains mandatory where required
- AI cannot fabricate truth-bearing system records
- option ranking never becomes option invention
- AI failures cannot widen system authority

---

24. Success Criteria

AIG-02 is correctly established when:

- hard non-delegable authority classes are explicit
- prohibited direct AI actions are explicit
- restricted AI actions are explicit
- human-review-required classes are explicit
- conflict resolution rules are explicit
- no role can silently treat AI as final authority

---

25. Summary Statement

AI Authority Limits defines the hard ceiling on what AI may never become inside Axion.

It preserves control-plane and human authority by making policy, approval, authorization, execution legality, integrity truth, release authority, audit truth, and provenance truth non-delegable to AI, while allowing bounded assistance under deterministic validation and human oversight.

AIG-03 — AI Input Contract

AIG-03 — AI Input Contract

1. Purpose

The AI Input Contract defines **what context may be sent to AI, in what structure, under what restrictions, and for what task classes** inside Axion.

Its purpose is to ensure that AI does not receive arbitrary, excessive, ambiguous, or unsafe context.

AIG-03 governs **AI input boundaries**.

It is the artifact that answers:

- what may be sent to AI?
- what must be sent to AI?
- what must never be sent to AI?
- how must AI input be structured?
- how is input scoped to the task being performed?

---

2. Objective

The objective of the AI Input Contract is to ensure that:

- AI receives only the minimum context needed for its scoped task
- input structure is predictable and machine-governed
- sensitive or authority-bearing data is not sent casually or unnecessarily
- AI tasks remain bounded by explicit task envelopes
- downstream auditing and replay can reconstruct what class of context was provided

- input ambiguity is reduced so AI output becomes more stable and reviewable

---

3. Problem It Solves

Without an explicit AI input contract, AI input tends to become:

- oversized
- inconsistent
- ad hoc
- poorly scoped
- overly privileged
- difficult to replay or inspect

That causes:

- unstable outputs
- hidden context leakage
- inconsistent recommendations
- poor operator trust
- weak AI auditability
- accidental exposure of unnecessary sensitive data

The AI Input Contract solves this by making AI input structured and bounded.

---

4. Scope

This artifact governs:

- AI request envelope structure
- allowed input classes
- prohibited input classes
- minimum required fields
- optional fields
- scope and sensitivity tags
- context minimization rules
- task-specific input shaping
- input validity requirements

This artifact does **not** define:

- AI output schema
- final validation logic for AI-produced outputs
- model routing policy
- full audit storage structure

Those are handled by later AIG artifacts.

---

5. Core Principle

AI receives only the context needed for the permitted task.

Not more. Not less than required. Not loosely assembled.

This means:

- no raw uncontrolled context dumps

- no hidden privileged context injection
- no “just send the whole object tree”
- no ambiguous task framing

All AI calls must use a governed request envelope.

---

6. Input Contract Model

Every AI request must be represented as a structured envelope with these conceptual sections:

- request identity
- task classification
- target plane and role
- risk class
- allowed AI mode
- input context payload
- constraints and forbidden actions
- traceability metadata

This creates a consistent boundary across all AI usage.

---

7. Required Top-Level Input Fields

Every AI request should include, at minimum:

7.1 `request_id`

Stable ID for the AI request.

7.2 `task_type`

The scoped AI task class.

Examples:

- summarization
- classification
- recommendation
- structured_proposal
- anomaly_review
- operator_assistance

7.3 `target_plane`

Which control plane is invoking AI.

Values:

- ICP
- GCP
- ECP

7.4 `target_role`

Which specific role is invoking AI.

Examples:

- ICP-01
- GCP-02
- ECP-03

7.5 `risk_class`

Risk classification for the task surface.

Values:

- low
- medium
- high

7.6 `allowed_mode`

How AI output may be used.

Values:

- advisory_only
- validate_first
- human_review_required
- restricted_low_risk_automation

7.7 `context_payload`

The actual structured task input.

7.8 `constraints`

Explicit boundaries on what the AI may and may not do.

7.9 `trace_metadata`

Metadata needed for auditing, replay, or inspection.

---

8. Allowed Input Classes

AI input may include only scoped, structured data classes relevant to the task.

8.1 Task framing data

- task objective
- expected output type
- role and plane
- risk class
- mode
- constraints

8.2 Structured state summaries

- run summary
- stage summary
- policy result summary
- approval summary
- quota summary
- release summary
- artifact summary
- lineage summary
- notification summary

8.3 Selected evidence references

- relevant audit excerpts or summaries
- relevant artifact refs

- relevant release refs
- relevant approval refs
- relevant override refs
- relevant policy IDs/results

8.4 Candidate option sets

Where AI is ranking or recommending among valid options, the valid candidate set may be included.

8.5 Human-facing explanation context

Where the task is explanation or summary, AI may receive limited readable context needed to explain the situation.

---

9. Prohibited Input Classes

The following inputs are prohibited unless explicitly justified by a governed task contract and policy.

9.1 Unbounded raw state dumps

Do not send the entire system state unless the task contract explicitly permits a bounded subset.

9.2 Hidden authority substitutions

Do not send input framed in a way that encourages AI to act as final approver, final policy source, or final permission source.

9.3 Irrelevant sensitive data

Do not send identity, secrets, credentials, or unrelated privileged content not needed for the scoped task.

9.4 Ambiguous unrestricted prompts

Do not send vague instructions like:

- “decide what we should do”
- “act as the controller”
- “approve this if it looks okay”
  without explicit bounded contract structure

9.5 Fake candidate sets

Do not send AI a mix of valid and invalid options without labeling or deterministic prevalidation if the task is recommendation among allowed options.

9.6 Raw truth-mutation authority hints

Do not send prompts implying that AI may rewrite canonical truth, approvals, policies, or evidence records.

---

10. Context Minimization Rules

Every AI request must minimize context.

10.1 Minimum necessary rule

Only include fields needed for the scoped task.

10.2 Relevance rule

All included context must be relevant to the requested output.

10.3 No convenience-overreach rule

Do not include extra context “just in case” if it increases ambiguity or sensitivity unnecessarily.

10.4 Explicit omission rule

If important context is intentionally omitted, the task contract should allow for incomplete-context handling rather than pretending the context is complete.

---

11. Structured Context Sections

The `context_payload` should be organized into explicit sections rather than raw free-form dumps.

Recommended structure:

11.1 `object_context`

The main object being analyzed.

Examples:

- run summary
- approval summary
- release summary
- artifact summary

11.2 `decision_context`

Any policy, state, or governance context directly affecting the task.

11.3 `candidate_context`

If AI is selecting or ranking among valid options.

11.4 `evidence_context`

Relevant supporting refs or summaries.

11.5 `operator_context`

Optional human-facing framing for explanation tasks.

The exact sections may vary by task, but the input must remain structured.

---

12. Task-Specific Input Rules

12.1 Summarization input

May include:

- bounded summaries
- relevant object state
- selected evidence refs
- intended audience

Must not include:

- unnecessary unrelated history
- authority-bearing prompts

12.2 Classification input

May include:

- structured subject data
- bounded feature context
- class schema or class labels if required

Must not include:

- hidden target outcome bias framed as authority instruction

12.3 Recommendation input

May include:

- bounded decision context
- valid candidate set

- risk class
- prohibited actions list

Must not include:

- unrestricted authority framing
- invalid options mixed as if they are valid

12.4 Structured proposal input

May include:

- target output schema
- bounded context
- deterministic constraints
- required fields

Must not include:

- permission to self-validate or self-authorize

12.5 Anomaly review input

May include:

- bounded event sequences
- grouped history windows
- anomaly criteria
- relevant evidence summaries

Must not include:

- demands for final guilt/fault/permission authority

---

13. Candidate-Set Requirements

When AI is being asked to recommend among options:

- the candidate set must already be deterministically filtered where required
- invalid candidates must not be presented as equally eligible unless clearly marked
- candidate metadata must include relevant selection criteria
- AI must be told that it is ranking among valid candidates, not inventing authority

Examples:

- valid execution targets
- valid approvers
- valid retry windows
- valid notification priority groupings

---

14. Constraint Block Requirements

Every AI request must include a constraint block.

At minimum, constraints should include:

- prohibited authority actions
- required output mode
- required uncertainty behavior

- requirement not to invent missing facts
- requirement to stay within candidate set where applicable
- requirement to return structured output if required

This prevents the task frame from relying on implied behavior.

---

15. Sensitivity and Scope Tags

Each AI request should include metadata describing sensitivity.

Recommended tags include:

- sensitivity_class
- contains_governance_context
- contains_evidence_context
- contains_protected_action_context
- contains_actor_identity_context
- contains_release_context
- contains_integrity_context

These tags help with routing, auditing, and fallback behavior.

---

16. Incomplete Context Rules

Sometimes AI will receive incomplete context by design.

The AI Input Contract must require that such cases be explicit.

16.1 Incomplete-context declaration

If context is partial, that must be tagged.

16.2 No fake completeness rule

The request must not imply completeness when context is partial.

16.3 Missing-data expectation

The task should instruct the AI to preserve uncertainty when relevant context is absent.

This reduces false confidence and authority drift.

---

17. Identity and Actor Context Rules

Actor-related information may be included only when relevant to the task.

Examples of allowed actor-context uses:

- approval routing summary
- authz explanation
- audit summarization
- separation-of-duties explanation

Rules:

- include only role/scope details needed for the task
- do not include unrelated identity data
- do not imply that actor identity grants AI authority to decide access truth

Actor context is for analysis support, not permission substitution.

---

18. Evidence Context Rules

When evidence context is included:

- evidence must be referenced or summarized clearly
- evidence relevance must be tied to the task
- evidence gaps should remain explicit
- AI must not be asked to treat evidence summaries as permission to invent missing evidence

Examples:

- release evidence summary
- selected audit entries
- pin verification result
- lineage partial graph

---

19. Request Validity Rules

An AI request is invalid if it:

- lacks task classification
- lacks role/plane association
- lacks allowed mode
- lacks constraints
- contains prohibited unrestricted authority framing
- contains obviously irrelevant or excessive sensitive context
- requests final authority behavior outside scope

- mixes incompatible tasks without explicit structuring

Invalid AI requests should be rejected before model invocation.

---

20. Relationship to AIG-01 and AIG-02

- **AIG-01** says where AI may operate
- **AIG-02** says what AI may never become
- **AIG-03** says what AI may be given as input when operating in allowed scope

This artifact translates allowed participation into a bounded context contract.

---

21. Failure Handling Expectations

If input preparation fails:

- do not invoke AI with malformed or unbounded context
- reject the AI request
- preserve deterministic fallback behavior
- log or trace invalid-request class where required

If context is incomplete:

- mark it incomplete
- proceed only if the task contract allows incomplete-context handling
- otherwise reject or escalate

AI input failure must not widen scope or authority.

---

22. Audit Expectations

For meaningful AI-assisted tasks, the system should be able to answer:

- what task type was requested
- which role requested it
- what risk and mode applied
- what input sections were included
- whether context was complete or partial
- what constraints were applied

This does not require full raw prompt storage in every case, but the input class and structure must be inspectable.

---

23. Required Guarantees

The AI Input Contract must guarantee:

- AI input is structured
- AI input is bounded
- AI input is task-specific
- AI input excludes prohibited authority framing
- AI input minimizes unnecessary sensitive context
- AI input does not pretend partial context is complete
- invalid AI requests can be rejected before invocation

---

24. Success Criteria

AIG-03 is correctly established when:

- every AI call can be described by a governed request envelope
- allowed input classes are explicit
- prohibited input classes are explicit
- context minimization rules are explicit
- candidate-set and constraint rules are explicit
- incomplete-context handling is explicit
- AI is never invoked through uncontrolled raw context dumps

---

25. Summary Statement

The AI Input Contract defines the bounded, structured context that Axion may send to AI for approved task surfaces.

It ensures that AI receives only task-relevant, scoped, constrained input through explicit request envelopes, preventing uncontrolled context sprawl, authority confusion, and unsafe hidden AI behavior.

AIG-04 — AI Output Contract

AIG-04 — AI Output Contract

1. Purpose

The AI Output Contract defines **what AI is allowed to return, in what structure, with what required metadata, and with what limits on interpretive authority** inside Axion.

Its purpose is to ensure that AI output is:

- predictable
- bounded
- machine-usable where needed
- clearly non-authoritative unless validated
- auditable and reviewable

AIG-04 governs **AI output structure and allowed output semantics**.

It is the artifact that answers:

- what may AI return?
- what must AI return?
- what must AI never return as authoritative truth?
- how should uncertainty be expressed?
- how should recommendations be packaged for downstream validation?

---

2. Objective

The objective of the AI Output Contract is to ensure that:

- AI output is structured enough to be handled safely
- AI output does not blur into final system authority
- downstream roles can distinguish explanation from recommendation from structured proposal
- uncertainty and incompleteness remain explicit
- prohibited authority claims are blocked at the contract level
- AI output can be validated, accepted, rejected, or ignored deterministically

---

3. Problem It Solves

Without an explicit AI output contract, AI output tends to become:

- overly free-form
- hard to validate
- hard to audit
- hard to distinguish from system truth
- easy to misuse as de facto authority

That causes:

- action confusion
- weak safety boundaries
- poor replayability
- inconsistent operator trust
- hidden authority creep

The AI Output Contract solves this by making output form and meaning explicit.

---

4. Scope

This artifact governs:

- output envelope structure
- allowed output types
- required metadata fields
- confidence and uncertainty fields
- evidence-reference behavior
- recommendation and proposal packaging
- prohibited output behaviors
- incomplete-output handling
- output validity requirements

This artifact does **not** define:

- whether output may become action
- which model is chosen
- complete audit persistence rules

Those are handled by later AIG artifacts.

---

5. Core Principle

AI output is not system truth by default.

AI output must always be recognizable as one of:

- explanation
- summary
- classification
- recommendation
- structured proposal
- anomaly flag

It must never silently impersonate:

- final approval
- final policy truth
- final authz truth
- final release truth
- final audit truth
- final integrity truth

---

6. Output Contract Model

Every AI response should be represented as a structured envelope with these conceptual sections:

- response identity
- request linkage
- task classification

- output type
- primary output payload
- confidence/uncertainty fields
- evidence/reference fields
- actionability classification
- human-review flag
- validation hints
- limitation notes

This creates a consistent shape across all AI usage.

---

7. Required Top-Level Output Fields

Every AI response should include, at minimum:

7.1 `response_id`

Stable ID for the AI response.

7.2 `request_id`

Link back to the governing AI request.

7.3 `task_type`

The task class being answered.

7.4 `output_type`

What kind of result is being returned.

Examples:

- summary
- explanation
- classification
- recommendation
- structured_proposal
- anomaly_flag

7.5 `result_payload`

The main structured output.

7.6 `confidence`

A bounded confidence indicator or confidence band.

7.7 `uncertainty_notes`

Explicit statement of ambiguity, missing context, or partial confidence where relevant.

7.8 `evidence_refs`

References to the supplied context elements relied upon, where applicable.

7.9 `requires_human_review`

Boolean flag indicating whether human review is required before meaningful use.

7.10 `actionability_class`

Classification of how the output may be used.

Values:

- informational_only
- advisory_only

- validate_before_use
- human_review_required

7.11 `validation_hints`

Optional structured hints for downstream deterministic validation.

7.12 `limitations`

Explicit statement of what the output does not determine.

---

8. Allowed Output Types

The contract should support these output classes.

8.1 Summary

Condensed explanation of provided structured context.

Examples:

- audit summary
- release readiness summary
- queue blockage summary

8.2 Explanation

Readable explanation of an already-determined system result.

Examples:

- why a transition failed
- why authz denied action
- why release is not ready

8.3 Classification

Structured categorization of a bounded input.

Examples:

- failure type
- anomaly class
- drift severity suggestion

8.4 Recommendation

Suggested next action or ranked candidate choice.

Examples:

- retry recommendation
- defer recommendation
- candidate target recommendation

8.5 Structured Proposal

Machine-usable candidate object for downstream validation.

Examples:

- proposed retry payload
- proposed approval summary packet
- proposed release review checklist

8.6 Anomaly Flag

Structured signal that something deserves attention.

Examples:

- suspicious audit pattern
- unusual permission activity
- repeated notification failure cluster

---

9. Prohibited Output Types

AI responses must not present themselves as any of the following.

9.1 Final Authority Declarations

Examples:

- “approved”
- “authorized”
- “policy passed”
- “integrity verified”
- “release published”
  when AI is not the authority for those results.

9.2 Fake Deterministic Results

AI must not emit outputs that pretend deterministic validation already occurred when it did not.

9.3 Fabricated Evidence Truth

AI must not claim missing approvals, missing audit events, missing lineage, or missing evidence as confirmed truth.

9.4 Hidden Action Commands

AI must not return disguised direct commands intended to bypass gating.

9.5 Scope Expansion Outputs

AI must not invent new valid candidates, permissions, overrides, or approvals beyond what input scope allowed.

---

10. Result Payload Rules

The `result_payload` must match the task type and output type.

Examples:

For summary/explanation

Should include:

- concise structured narrative
- key factors
- relevant refs
- unresolved points if any

For classification

Should include:

- class label
- optional secondary labels
- rationale summary
- uncertainty marker if needed

For recommendation

Should include:

- recommended option

- ranked alternatives if applicable
- basis for ranking
- blocked options if applicable
- no authority claim

For structured proposal

Should include:

- schema-conformant candidate payload
- omitted fields list if incomplete
- proposal rationale
- explicit note that validation is required

For anomaly flag

Should include:

- anomaly category
- severity suggestion
- affected object refs
- reason summary
- review recommendation

---

11. Confidence Rules

Every output must represent confidence carefully.

The contract should support confidence in a bounded way such as:

- low
- medium
- high

or a numeric band if you later prefer.

Rules:

- confidence must not imply authority
- confidence must not hide uncertainty
- high confidence does not remove validation requirements
- low confidence should usually strengthen human review requirements

Confidence is descriptive, not permission-bearing.

---

12. Uncertainty Rules

AI must preserve uncertainty explicitly when:

- context is incomplete
- evidence is partial
- multiple plausible interpretations exist
- candidate quality is close or ambiguous
- classification is weakly supported

Uncertainty must not be hidden inside vague language.

The response should include:

- what is uncertain

- why it is uncertain
- whether that uncertainty blocks action use

---

13. Evidence Reference Rules

Where relevant, AI output should reference the supplied context elements it relied on.

Examples:

- input section IDs
- object refs
- evidence summary refs
- candidate IDs
- artifact or release refs
- audit excerpt refs

Rules:

- AI may only reference supplied or derived-from-supplied context
- AI may not fabricate evidence refs
- evidence refs do not make the output authoritative by themselves

This supports replay and review.

---

14. Actionability Rules

Every AI output must declare how it may be used.

14.1 `informational_only`

Pure explanation or summary. No downstream action should be derived directly.

14.2 `advisory_only`

A recommendation or suggestion intended for operator consideration.

14.3 `validate_before_use`

Structured proposal that may be transformed into action only after deterministic validation.

14.4 `human_review_required`

Output that must not meaningfully influence protected workflow completion without explicit human review.

This field is mandatory for keeping AI output from drifting into authority.

---

15. Limitation Statement Rules

Every meaningful AI response should include limitations describing what it does **not** determine.

Examples:

- “This does not approve the action.”
- “This does not replace policy evaluation.”
- “This does not verify integrity.”
- “This does not authorize the actor.”
- “This does not publish the release.”

This can be structured rather than verbose, but it must be present conceptually.

---

16. Incomplete Output Rules

Sometimes AI cannot fully answer within the contract.

The AI Output Contract must support incomplete output states.

Examples:

- insufficient_context
- ambiguous_candidate_set
- evidence_gap_detected
- no_recommendation_possible
- explanation_limited

Rules:

- incomplete output must be explicit
- missing sections must not be fabricated
- downstream roles must treat incomplete output conservatively

---

17. Task-Specific Output Rules

17.1 Summarization output

Must:

- summarize only supplied bounded context
- preserve uncertainty if context is partial
- avoid authority claims

17.2 Classification output

Must:

- provide class label
- indicate uncertainty where relevant
- avoid pretending class label is a governance decision

17.3 Recommendation output

Must:

- identify recommended option
- show basis for recommendation
- remain clearly non-authoritative

17.4 Structured proposal output

Must:

- match the declared schema
- clearly require validation
- identify omitted or uncertain fields

17.5 Anomaly review output

Must:

- indicate anomaly type
- indicate severity suggestion
- indicate that operator or deterministic review may still be required

---

18. Candidate-Set Output Rules

When AI is ranking candidates:

- only provided valid candidates may appear in the recommendation set
- output must preserve candidate IDs or stable refs
- output must not invent extra candidates
- output should explicitly state if no safe recommendation can be made among the provided candidates

This keeps recommendation bounded to deterministic prevalidated options.

---

19. Prohibited Language and Semantics

Even if the exact text varies, AI output must not semantically behave as:

- final approval
- final authorization
- final publish/revoke act
- final integrity certification
- final audit certification
- final policy clearance

The contract should reject outputs whose semantics cross those lines, even if phrased indirectly.

---

20. Output Validity Rules

An AI output is invalid if it:

- lacks request linkage
- lacks output type
- lacks result payload
- lacks actionability class
- makes prohibited authority claims
- fabricates evidence refs
- invents candidates outside allowed scope
- hides uncertainty in a protected or medium/high-risk task
- outputs malformed structured proposal payload where schema is required

Invalid outputs should be rejected before use.

---

21. Relationship to AIG-03

- **AIG-03** governs what AI may receive
- **AIG-04** governs what AI may return

Together they form the bounded request/response contract for AI assistance.

---

22. Failure Handling Expectations

If AI output is malformed, incomplete in an unusable way, or crosses authority boundaries:

- reject the output
- do not treat it as valid advisory result
- do not convert it into action

- preserve deterministic system safety
- trace or log the invalid-output class where required

If output is valid but low-confidence:

- preserve it as advisory or human-review-required according to gating rules

---

23. Audit Expectations

For meaningful AI-assisted paths, the system should be able to answer:

- what output type AI returned
- what actionability class it carried
- whether uncertainty was declared
- whether the output was incomplete
- whether the output was later validated, ignored, or rejected

This does not require exposing raw hidden reasoning, but the output class and semantics must be inspectable.

---

24. Required Guarantees

The AI Output Contract must guarantee:

- AI outputs are structured
- AI outputs do not silently impersonate authority
- uncertainty remains explicit
- evidence references remain bounded to supplied context

- candidate recommendations do not invent new valid options
- malformed or authority-crossing outputs can be rejected deterministically

---

25. Success Criteria

AIG-04 is correctly established when:

- all AI outputs fit known output classes
- actionability is explicit
- confidence and uncertainty are explicit
- structured proposals are distinguishable from explanations
- prohibited authority claims are blocked
- downstream systems can validate, reject, or safely ignore outputs

---

26. Summary Statement

The AI Output Contract defines the bounded, structured forms AI may return inside Axion.

It ensures that summaries, explanations, classifications, recommendations, anomaly flags, and structured proposals remain explicit, reviewable, non-authoritative by default, and safe for downstream validation and operator use.

AIG-05 — AI Action Gating

AIG-05 — AI Action Gating

1. Purpose

The AI Action Gating artifact defines **how AI output is prevented from becoming system action unless it passes the correct control path** inside Axion.

Its purpose is to ensure that AI output remains bounded at the moment it becomes operationally relevant.

AIG-05 governs **the path from AI output to governed use**.

It is the artifact that answers:

- when is AI output only informational?
- when may AI output influence action?
- when is deterministic validation mandatory?
- when is human review mandatory?
- when may low-risk automation be allowed?
- what blocks AI output from becoming action?

---

2. Objective

The objective of AI Action Gating is to ensure that:

- AI output never becomes action by accident
- protected actions cannot be completed from raw AI output
- every AI-assisted action path passes the required deterministic checks
- human review is inserted where risk requires it

- low-risk automation remains explicitly bounded
- rejected AI output does not leak into operational truth

---

3. Problem It Solves

Without explicit action gating, AI output tends to drift from:

- recommendation
- explanation
- proposal

into:

- implied command
- silent controller behavior
- de facto decision authority
- hidden automation

That causes:

- authority creep
- unsafe action execution
- weak auditability
- operator confusion about what actually decided the outcome

AI Action Gating solves this by forcing every AI-assisted action path through explicit gates.

---

4. Scope

This artifact governs:

- actionability classes
- output-to-action transition rules
- deterministic validation requirements
- human review requirements
- protected-action gating
- low-risk automation gating
- rejection handling for invalid AI outputs
- safe fallback when AI-assisted action cannot proceed

This artifact does **not** define:

- the exact input envelope
- the exact output envelope
- the detailed model routing policy

Those are handled by adjacent AIG artifacts.

---

5. Core Principle

AI output is inert until it passes its required gate.

That means:

- an explanation stays an explanation
- a recommendation stays a recommendation

- a proposal stays a proposal

unless the system explicitly:

- validates it
- reviews it if required
- authorizes it if required
- checks policy if required
- checks evidence if required
- records the governed outcome

No AI output may directly self-promote into action.

---

6. Gating Model

Every AI output must be placed into one of these gating classes:

6.1 Informational

Output may be shown, summarized, or inspected, but not used to drive action.

6.2 Advisory

Output may influence operator thinking, but system action must still be initiated through normal non-AI authority paths.

6.3 Validate-first

Output may be transformed into candidate action input, but only after deterministic validation.

6.4 Human-review-required

Output may support a protected or meaningful action, but a human must explicitly review before it can proceed.

6.5 Restricted low-risk automation

A small explicitly whitelisted class of low-risk outputs may be auto-applied after deterministic validation.

These classes are mandatory for every AI-assisted surface.

---

7. Default Gating Rule

Unless explicitly upgraded by governance:

- AI output is **advisory-only by default**
- protected-action-related AI output is **human-review-required by default**
- no AI output is **auto-actionable by default**

This prevents silent action creep.

---

8. Gating by Output Type

8.1 Summary outputs

Default gate:

- informational_only

These may be displayed or logged for operator understanding, but should not directly cause state mutation.

8.2 Explanation outputs

Default gate:

- informational_only or advisory_only

These explain system truth. They do not replace it.

8.3 Classification outputs

Default gate:

- advisory_only

If a classification may influence action, downstream validation must still occur.

8.4 Recommendation outputs

Default gate:

- advisory_only or validate_first

Recommendations may shape action, but may not become action directly.

8.5 Structured proposal outputs

Default gate:

- validate_first

These are the clearest AI-to-action bridge, so explicit deterministic validation is mandatory.

8.6 Anomaly flag outputs

Default gate:

- advisory_only or human_review_required

An anomaly flag may trigger attention, not automatic irreversible action.

---

9. Protected-Action Gating Rule

Any AI output that could influence a protected action must be gated as:

- **human_review_required**
- plus all required deterministic validation

Protected actions may include:

- approval completion
- override activation
- release publish/revoke
- privileged execution mutation
- protected unpin
- restricted environment use
- privileged queue reprioritization
- permission-sensitive actions

AI may support these workflows, but may not close them autonomously.

---

10. Deterministic Validation Rule

For any AI output in `validate_first` or stronger classes, the system must run the relevant deterministic validators before operational use.

Examples:

ICP examples

- retry proposal → Run Controller + State Machine + policy/authz if relevant
- environment proposal → Adapter / Environment validation
- queue timing proposal → Queue / Scheduler checks + quota if relevant

GCP examples

- approver recommendation → Approval Router + AuthZ validation
- override support payload → Override eligibility + approval + authz
- quota defer suggestion → Quota evaluation + queue coordination

ECP examples

- release review proposal → Release readiness checks + policy/approval + evidence completeness
- drift interpretation → Pin verification remains source of truth
- lineage explanation → registry truth remains source of truth

Validation is never optional once output may influence action.

---

11. Human Review Rule

Human review is required whenever AI output affects:

- protected actions
- irreversible actions
- high-risk governance interpretations
- evidence ambiguity with operational consequence
- security-sensitive or permission-sensitive outcomes
- exception/override paths
- release publish/revoke readiness in a decisive way

Human review must be explicit, not implied.

This means:

- there must be a visible review step
- a human actor must consciously accept/reject/ignore the AI output where required
- review outcome must be traceable where governance requires it

---

12. Candidate-to-Action Rule

When AI returns a candidate action or structured proposal:

- the proposal must remain a **candidate**
- it must not be written directly into canonical state
- it must not bypass required validation
- it must not bypass approval/authz/policy/evidence checks
- it must not be treated as equivalent to a human-approved action

Candidate action is not authorized action.

---

13. Low-Risk Automation Rule

Restricted low-risk automation may exist only if all of the following are true:

- the task surface is explicitly whitelisted
- the output class is low-risk
- deterministic validation fully exists
- no protected action is involved
- no truth-bearing record is being authoritatively rewritten

- failure can degrade safely
- audit/trace expectations are satisfied if required

Examples of possible low-risk automation:

- formatting an operator-facing summary
- grouping notification noise for display
- generating a candidate review packet
- suggesting non-binding queue display prioritization

Low-risk automation is narrow by design.

---

14. Disallowed Auto-Action Classes

The following classes must never auto-apply directly from AI output under this architecture:

- approval granting
- policy allow/deny finalization
- authorization granting
- override activation
- release publish/revoke
- integrity verification pass/fail substitution
- audit truth mutation
- lineage truth mutation
- mandatory notification suppression
- illegal or ambiguous state transition execution

If one of these appears auto-actionable, the system design is wrong or the gate is broken.

---

15. Rejection Rules

AI output must be rejected from action use when:

- it is malformed
- it crosses authority limits
- it lacks required fields
- it invents options outside allowed candidate set
- it hides uncertainty in a medium/high-risk context
- deterministic validation fails
- required human review is missing
- it implies prohibited direct authority

Rejected AI output must not partially mutate state.

---

16. Partial Acceptance Rules

In some cases, only part of an AI output may be usable.

Examples:

- readable summary may be shown, but proposed action payload is rejected
- candidate ranking may be kept, but top choice is blocked by later validation
- anomaly flag may be preserved, but suggested remediation is not used

The gating model should allow:

- full acceptance
- partial acceptance
- rejection
- informational-only retention

This prevents all-or-nothing misuse.

---

17. Confidence and Gating Relationship

Confidence does not grant authority.

Rules:

- high confidence does not remove validation
- medium/high-risk outputs with low confidence should strengthen human review requirements
- low confidence may downgrade usable actionability
- confidence may influence operator handling, not governance authority

Confidence informs review. It does not replace the gate.

---

18. Uncertainty and Gating Relationship

If AI output includes significant uncertainty:

- it should remain informational or advisory unless downstream rules explicitly allow more
- protected-action workflows should require human review
- ambiguous structured proposals may be rejected or partially accepted

- missing context should block direct use where deterministic requirements need complete input

Uncertainty must narrow actionability, not expand it.

---

19. Plane-Specific Gating Guidance

19.1 ICP gating

Examples:

- failure explanation → informational_only
- retry suggestion → advisory_only or validate_first
- proposed retry payload → validate_first
- dispatch/action in protected context → human_review_required + validation

19.2 GCP gating

Examples:

- policy explanation → informational_only
- approver suggestion → validate_first
- override risk summary → human_review_required
- permission anomaly flag → advisory_only or human_review_required depending on risk

19.3 ECP gating

Examples:

- audit summary → informational_only

- release readiness summary → advisory_only or human_review_required
- drift explanation → informational_only
- lineage impact summary → advisory_only
- any output implying evidence truth mutation → prohibited

---

20. Operator Interaction Rule

Where human review is required, the UI or operator workflow must clearly distinguish:

- deterministic system truth
- AI output
- validation result
- final governed action/result

An operator must never need to guess:

- whether something is a suggestion
- whether it has been validated
- whether it has become action
- whether it still requires approval or review

This is essential to safe gating.

---

21. Failure Handling Expectations

If AI-assisted action gating fails at any point:

- do not apply the action
- preserve deterministic system safety
- downgrade to advisory or reject outright as appropriate
- require human review if the surface is sensitive
- retain normal non-AI workflows as fallback

Examples:

- invalid proposal schema → reject
- validation failure → reject action use
- missing reviewer → block protected workflow
- downstream state changed since proposal → revalidate or reject stale proposal

---

22. Audit Expectations

For meaningful AI-assisted actions, the system should be able to answer:

- what AI output type was produced
- what gating class applied
- whether validation occurred
- whether human review occurred
- whether output was accepted, partially accepted, or rejected
- whether output influenced final action

The exact trace format comes later, but the gating path must be inspectable.

---

23. Required Guarantees

AI Action Gating must guarantee:

- AI output cannot become action accidentally
- protected actions cannot be AI-completed without required human/deterministic gates
- structured proposals remain candidate inputs until validated
- rejected or invalid AI outputs do not mutate authoritative state
- low-risk automation remains explicitly bounded
- AI failure cannot widen action authority

---

24. Success Criteria

AIG-05 is correctly established when:

- every AI output class maps to a gating class
- deterministic validation is mandatory where needed
- human review is mandatory where needed
- prohibited auto-action classes are explicit
- rejection rules are explicit
- operators can distinguish suggestion, validation, and final action clearly

---

25. Summary Statement

AI Action Gating defines how Axion prevents AI output from becoming operational authority without passing the correct deterministic and human-controlled control path.

It ensures that AI remains advisory or validate-first by default, that protected actions remain gated, and that only tightly whitelisted low-risk automations can proceed after explicit validation.

AIG-06 — AI Audit + Replay Rules

AIG-06 — AI Audit + Replay Rules

1. Purpose

The AI Audit + Replay Rules artifact defines **how AI participation is recorded, inspected, and reconstructed inside Axion**.

Its purpose is to ensure that whenever AI influences an operator workflow, recommendation flow, or candidate action path, the system can later answer:

- where was AI used?
- for what task?
- with what bounded context class?
- what did AI return?
- what gate applied?
- was the output accepted, rejected, ignored, or partially used?
- what final deterministic or human-governed result followed?

AIG-06 governs **AI traceability and replayability**.

It is the artifact that turns “AI helped here” into explicit, reviewable system evidence.

---

2. Objective

The objective of AI Audit + Replay Rules is to ensure that:

- AI usage is inspectable
- AI influence on governed actions is reviewable
- replay of AI-assisted decision paths is possible at the right level

- hidden AI influence is prevented
- operators can distinguish AI suggestion from deterministic result
- audit scope is sufficient without requiring unsafe or excessive raw retention
- AI-assisted workflows remain accountable

---

3. Problem It Solves

Without explicit AI audit and replay rules, AI usage tends to become:

- opaque
- difficult to investigate
- easy to misremember
- hard to separate from human or deterministic system decisions
- difficult to debug after incidents

That causes:

- weak accountability
- poor incident analysis
- poor operator trust
- unclear AI influence boundaries
- hidden authority drift

AI Audit + Replay Rules solves this by making AI participation traceable.

---

4. Scope

This artifact governs:

- AI invocation trace requirements
- AI request/response linkage requirements
- AI usage outcome recording
- replay metadata requirements
- acceptance/rejection/partial-use recording
- human review linkage where required
- model selection visibility requirements
- retention-class expectations at the metadata level

This artifact does **not** define:

- the full raw prompt text retention policy in all environments
- the exact UI implementation
- the exact model routing algorithm

Those may be implementation-specific, but the audit/replay obligations remain.

---

5. Core Principle

If AI meaningfully participated, the system must be able to prove how it participated.

That means:

- no hidden AI influence on governed actions
- no “the system just decided” when AI materially shaped the candidate path
- no silent promotion of AI output into accepted action

AI auditability does not require exposing private model internals, but it does require preserving the operationally relevant trace.

---

6. AI Trace Model

Every meaningful AI participation should be traceable through a linked chain of records:

- AI request trace
- AI response trace
- gating/validation trace
- human review trace where required
- final outcome trace

This creates a reconstructable AI influence path.

---

7. Required Trace Questions

The system should be able to answer, for a meaningful AI-assisted event:

1. **Which role invoked AI?**
2. **For what task type?**
3. **At what risk class and actionability mode?**
4. **What bounded context class was provided?**
5. **Which model or fallback path was used?**
6. **What output type did AI return?**
7. **What uncertainty/confidence state applied?**
8. **What gate applied to the output?**

9. **Was the output accepted, rejected, ignored, or partially used?**
10. **What final deterministic or human-governed result followed?**

If these cannot be answered, the trace is insufficient.

---

8. Required Trace Objects

The AI audit/replay model should include, at minimum, these conceptual objects.

8.1 AI Invocation Record

Represents that AI was called.

8.2 AI Response Record

Represents what output class AI returned.

8.3 AI Gating Record

Represents what happened between output and potential use.

8.4 AI Review Record

Represents human review when required.

8.5 AI Outcome Record

Represents how the AI output ultimately affected or did not affect system action.

These may be stored separately or linked through one trace model, but all concepts must be representable.

---

9. Minimum AI Invocation Metadata

Each AI invocation should preserve, at minimum:

- invocation_id

- request_id
- target_plane
- target_role
- task_type
- risk_class
- allowed_mode
- context_class summary
- context completeness marker
- constraint block reference
- model selection reference
- timestamp

This is enough to prove the nature of the invocation even if raw input is not always fully retained.

---

10. Minimum AI Response Metadata

Each AI response should preserve, at minimum:

- response_id
- invocation_id or request_id linkage
- output_type
- actionability_class
- confidence band
- uncertainty marker

- evidence/reference summary
- completion/incomplete-output marker
- limitation summary
- timestamp

This allows later inspection of the response semantics without overstating authority.

---

11. Minimum Gating Metadata

For any AI output that could influence action, the system should preserve:

- gating_class
- validation required yes/no
- validation result
- validation failure reason if any
- human review required yes/no
- human review performed yes/no
- accepted / rejected / partially accepted / ignored
- timestamp(s)

This is what proves the AI output did or did not cross into operational use.

---

12. Minimum Human Review Metadata

Where human review is required, the system should preserve:

- reviewer identity

- review requirement class
- review outcome
- review timestamp
- reviewed output reference
- acceptance/rejection rationale where required by governance

AI-assisted protected workflows are not fully reviewable unless this linkage exists.

---

13. Minimum Outcome Metadata

For a meaningful AI-assisted event, the system should preserve:

- final action or no-action result
- deterministic role(s) that made final decision
- governed object affected
- whether AI output influenced the result
- influence type:
    - informational only
    - recommendation considered
    - structured proposal validated
    - partial guidance used
    - rejected entirely
- timestamp

This is what separates AI participation from actual authority.

---

14. Replay Model

Replay does **not** mean recreating hidden chain-of-thought.

Replay means reconstructing the operational path of AI participation using inspectable metadata and bounded content references.

Replay should allow an authorized reviewer to reconstruct:

- what the task was
- what class of context was used
- what output type came back
- what validations occurred
- what human review occurred
- what final outcome followed

Replay is about governance visibility, not model internals.

---

15. Replay Levels

The system may support different replay depths depending on sensitivity.

15.1 Metadata replay

Reconstructs invocation, task type, gate, and final result.

15.2 Structured content replay

Reconstructs bounded request/response envelopes or summaries.

15.3 Evidence-linked replay

Reconstructs the request/response using refs to source evidence objects, candidates, or summaries.

The exact implementation depth may vary, but metadata replay is mandatory for meaningful AI-assisted actions.

---

16. Acceptance-State Rules

The system must explicitly record one of the following acceptance states for meaningful AI outputs:

- ignored
- shown informationally
- advisory considered
- validated and used
- partially used
- rejected before validation
- rejected after validation
- blocked pending review
- blocked by deterministic rules

No meaningful AI output should disappear without a recorded outcome class.

---

17. Partial-Use Rules

Sometimes only a portion of an AI output is used.

The system should be able to represent cases such as:

- summary shown, proposal rejected
- ranking considered, top suggestion not used

- explanation retained, action candidate blocked
- anomaly flag accepted, remediation suggestion rejected

Partial-use traceability is important because it prevents all-or-nothing false narratives.

---

18. Model Selection Visibility Rules

The AI trace should preserve enough to answer:

- which model family was selected
- whether fallback was used
- whether routing selected a reasoning-optimized or coding-optimized path
- whether the invocation deviated from the default route

This does not require surfacing secret infrastructure details, but model-path visibility is required for debugging and trust.

---

19. Incomplete and Failed Invocation Rules

The AI audit model must also represent when AI participation failed.

Examples:

- invalid request rejected before invocation
- model unavailable
- malformed output
- output crossed authority boundary
- validation failed

- reviewer rejected AI-assisted path

These failure paths must remain visible rather than disappearing.

---

20. Retention-Class Rules

AIG-06 should define retention expectations at the class level, even if exact durations are implementation-specific.

At minimum, distinguish between:

- low-risk informational traces
- medium-risk recommendation traces
- high-risk protected-workflow traces

High-risk protected-workflow traces should have the strongest retention and replay expectations.

The exact storage duration can be set elsewhere, but the classification must exist.

---

21. Relationship to Audit Logger

The Audit Logger preserves system action evidence broadly.

AIG-06 defines the AI-specific trace requirements that should link into or alongside that broader evidence model.

In simple terms:

- Audit Logger = general action evidence
- AIG-06 = AI participation evidence requirements

These should integrate, not duplicate blindly.

---

22. Relationship to AIG-03 / AIG-04 / AIG-05

- **AIG-03** defines what AI received
- **AIG-04** defines what AI returned
- **AIG-05** defines how that output was gated
- **AIG-06** defines how all of that is recorded and replayed

This artifact is the accountability layer across those contracts.

---

23. Prohibited Audit Gaps

The following gaps are unacceptable for meaningful AI-assisted workflows:

- AI used but no trace of which role invoked it
- AI output influenced action but no gating record
- protected workflow used AI but no review record where review was required
- AI recommendation used but no acceptance-state record
- fallback model used with no visibility
- AI-assisted action occurred but no way to reconstruct the influence path

These gaps create hidden authority risk.

---

24. Failure Handling Expectations

If AI trace capture fails for a meaningful AI-assisted path:

- do not silently continue as though trace exists
- downgrade to non-AI flow where required, or

- block use of the AI-assisted path where governance sensitivity requires traceability
- preserve explicit failure reason

High-risk protected workflows should not rely on untraceable AI participation.

---

25. Required Guarantees

AI Audit + Replay Rules must guarantee:

- meaningful AI usage is traceable
- AI-assisted action paths are reconstructable
- acceptance/rejection state is explicit
- human review linkage is preserved where required
- fallback model use is visible
- AI participation cannot hide behind generic system language

---

26. Success Criteria

AIG-06 is correctly established when:

- every meaningful AI invocation produces a traceable record
- every meaningful AI response has an outcome class
- gating and review linkage are inspectable
- replay can reconstruct AI influence without relying on hidden model internals
- protected workflows do not use AI invisibly
- failure paths are traceable too

27. Summary Statement

AI Audit + Replay Rules defines how Axion records and reconstructs AI participation so AI-assisted workflows remain inspectable, accountable, and clearly subordinate to deterministic and human-governed authority.

It ensures that meaningful AI usage leaves an evidence trail linking invocation, response, gating, review, and final outcome.

AIG-07 — Model Routing + Fallback Policy

AIG-07 — Model Routing + Fallback Policy

1. Purpose

The Model Routing + Fallback Policy defines **how Axion chooses which AI model to use for an approved task surface, and what happens when the preferred model is unavailable, unsuitable, or fails**.

Its purpose is to ensure that model selection is:

- explicit
- task-aware
- risk-aware
- replaceable
- inspectable
- safe under failure

AIG-07 governs **AI model selection and degradation behavior**.

It is the artifact that answers:

- which model should handle which task class?
- when should one model be preferred over another?
- when is fallback allowed?
- when is fallback forbidden?
- how does the system behave when no acceptable model path exists?

---

2. Objective

The objective of Model Routing + Fallback Policy is to ensure that:

- AI model choice is not arbitrary
- different task classes can use models suited to their strengths
- fallback behavior does not widen authority or reduce safety
- protected and higher-risk flows do not silently degrade into lower-trust behavior
- model changes do not require redesigning the control planes
- operators can inspect what model path was used

---

3. Problem It Solves

Without an explicit routing and fallback policy, model use tends to become:

- ad hoc
- hardcoded in scattered places
- inconsistent by role
- brittle during outages
- opaque to operators
- unsafe when “any available model” gets used as a substitute

That causes:

- unstable output quality
- weak trust in AI-assisted workflows
- unclear failure behavior
- silent capability mismatches

- hidden differences in how the same task gets handled over time

Model Routing + Fallback Policy solves this by making model choice and degradation explicit.

---

4. Scope

This artifact governs:

- model family selection by task class
- model family selection by role
- reasoning-depth selection rules
- structured-output suitability rules
- fallback eligibility rules
- fallback ordering
- no-fallback cases
- unavailable-model handling
- degraded-mode behavior
- routing traceability expectations

This artifact does **not** define:

- the internal architecture of the models
- exact vendor/runtime implementation
- the full AI request or response schema
- action gating rules

Those are handled by adjacent artifacts.

---

5. Core Principle

Model routing may change which AI assists the system.
It may not change who holds authority.

That means:

- switching models does not alter the control-plane authority structure
- fallback cannot bypass validation
- fallback cannot upgrade AI authority
- fallback cannot silently lower required human review
- no model choice may turn advisory output into authoritative output

Routing affects assistance quality and suitability, not governance boundaries.

---

6. Routing Model

Every AI invocation should be routed using explicit inputs such as:

- task class
- target plane
- target role
- risk class
- required mode
- required output structure
- reasoning depth needed
- candidate-set complexity

- sensitivity class
- fallback eligibility

This creates a deterministic or policy-governed routing path instead of arbitrary model choice.

---

7. Model Roles in Axion

Based on the direction already established, the default Axion model roles are:

7.1 Primary general control-plane assistant

Qwen
Best fit for:

- code-aware assistance
- structured output generation
- repo-aware operator support
- bounded workflow assistance
- general control-plane support tasks

7.2 Reasoning specialist

DeepSeek
Best fit for:

- heavier reasoning
- complex exception analysis
- policy conflict explanation
- override-risk synthesis
- more involved postmortem-style reasoning tasks

7.3 Optional implementation/coding specialist

Devstral
Best fit for:

- engineering-side implementation assistance
- coding workflows
- issue-solving support
- patch or dev-oriented help surfaces

These roles are routing defaults, not authority boundaries.

---

8. Routing Dimensions

Model routing should consider at least these dimensions.

8.1 Task class

Examples:

- summarization
- classification
- recommendation
- structured proposal
- anomaly review
- operator assistance

8.2 Output structure requirement

Some tasks need:

- stable structured payloads

- constrained response format
- candidate ranking
- readable explanation only

8.3 Reasoning depth

Some tasks are shallow:

- summarization
- explanation
- simple categorization

Others are deeper:

- override-risk synthesis
- policy conflict summarization across multiple conditions
- complex release readiness synthesis

8.4 Sensitivity / risk

High-risk contexts may require:

- stronger routing discipline
- no opportunistic fallback
- mandatory human review regardless of model quality

8.5 Context class

Some tasks are:

- code/repo aware

- governance heavy
- evidence heavy
- anomaly heavy
- operationally explanatory

Routing should reflect that.

---

9. Default Routing Policy

Unless a more specific route is declared, default routing should be:

9.1 Qwen default route

Use Qwen for:

- most operator assistance tasks
- most structured proposal tasks
- code-aware explanation tasks
- repo-linked summaries
- ordinary bounded control-plane assistance

9.2 DeepSeek specialized route

Use DeepSeek for:

- deeper reasoning tasks
- multi-factor override risk synthesis
- complex policy conflict explanation
- heavier analytical recommendation tasks

9.3 Devstral optional route

Use Devstral for:

- implementation-oriented support
- development and code-change assistance
- engineering workflow help where coding specialization matters

If a task does not need the specialist path, stay with the default route.

---

10. Role-Oriented Routing Guidance

10.1 ICP routing guidance

Use Qwen by default for:

- failure explanation
- retry suggestion
- blocked-dispatch summary
- environment mismatch explanation
- operator-facing execution summaries

Use DeepSeek when:

- the execution issue requires deeper multi-factor reasoning
- there is complex ambiguity across failure causes
- a high-context recovery recommendation is needed

Use Devstral when:

- the task is implementation-support-heavy rather than operationally authoritative

---

10.2 GCP routing guidance

Use Qwen by default for:

- approval packet drafting
- readable policy explanation
- readable denial explanation
- routine quota explanation
- ordinary governance summaries

Use DeepSeek when:

- policy conflicts are complex
- override-risk analysis is complex
- there are multiple interacting governance conditions
- a deeper structured reasoning pass is warranted

Use Devstral only if:

- the task is effectively engineering support around governance implementation rather than live governance assistance

---

10.3 ECP routing guidance

Use Qwen by default for:

- audit summaries
- release readiness summaries
- lineage explanation
- notification stream summaries
- operator-facing evidence explanations

Use DeepSeek when:

- anomaly review is more complex
- provenance ambiguity requires deeper synthesis
- release-readiness reasoning spans many evidence classes

Use Devstral only if:

- the task is about implementation/debugging of evidence systems rather than operational evidence assistance

---

11. Output-Structure Routing Rules

When structured output reliability matters, prefer the model route best suited to stable constrained outputs.

Routing should prefer:

- the model with better structured payload stability for `structured_proposal`
- the model with stronger explanation readability for `summary` / `explanation`
- the model with stronger reasoning depth for multi-factor `recommendation` or `anomaly_review`

If structured output stability is insufficient for the task, fallback must not silently turn a validate-first task into a loosely formatted advisory blob.

---

12. Fallback Eligibility Rules

Fallback is allowed only when all of these are true:

- the task surface allows fallback
- the fallback model is approved for that task class
- the fallback does not widen AI authority
- the fallback does not reduce required gating or human review
- the fallback can satisfy the output contract at an acceptable level

If these are not true, the task should fail closed or degrade to non-AI operation.

---

13. Fallback Ordering

The system should define explicit fallback order by task class.

A reasonable default pattern for Axion is:

13.1 General bounded assistance

Qwen → DeepSeek → no-AI fallback

13.2 Deep reasoning tasks

DeepSeek → Qwen → human/non-AI fallback

13.3 Coding/implementation support tasks

Devstral → Qwen → no-AI or human/manual fallback

This order may vary by deployment, but fallback order must be explicit.

---

14. No-Fallback Cases

Some tasks should not silently fall back.

No-fallback or fail-closed behavior should apply when:

- the task requires a specialist capability the fallback cannot satisfy
- the task is high-risk and fallback has materially weaker suitability
- the task depends on a structured output guarantee the fallback cannot meet
- the task is in a protected workflow where degraded explanation quality would increase operator risk
- the deployment policy explicitly forbids fallback for that task class

In these cases, the system should:

- reject the AI path
- degrade to non-AI operation
- require human/manual handling as appropriate

---

15. Degraded-Mode Rules

If preferred routing is unavailable, the system may enter degraded AI mode only under explicit rules.

Degraded mode may include:

- explanation-only fallback instead of proposal generation
- advisory-only fallback instead of validate-first
- no-AI fallback with human/manual workflow

Degraded mode must not:

- silently preserve stronger actionability than the degraded model can safely support
- weaken human review requirements
- weaken deterministic validation requirements

Degradation may reduce capability. It may not reduce governance.

---

16. Capability-Mismatch Rules

The routing layer must reject model paths that do not match task needs.

Examples:

- a model poor at structured output should not be used for validate-first proposal generation without explicit approval
- a weak reasoning path should not silently replace a deeper reasoning route in high-risk governance synthesis
- a coding-specialist path should not be used as default for evidence-governance interpretation unless explicitly justified

Routing is not just about availability. It is about task-model fit.

---

17. Protected-Workflow Routing Rules

Protected workflows require stricter routing discipline.

For workflows touching:

- approvals
- overrides
- release publish/revoke readiness

- protected execution recovery
- integrity ambiguity with operational consequence

the system should require:

- approved model family only
- explicit fallback eligibility
- no relaxation of human review
- no relaxation of deterministic validation
- traceable model path selection

Protected workflows should never use “whatever model answered” logic.

---

18. Manual / No-AI Fallback Rules

When no acceptable AI model path exists, the system must be able to degrade to:

- deterministic-only operation
- human/manual review
- ordinary non-AI system behavior

This is essential.

The absence of a model must not:

- force unsafe AI substitution
- block all safe system operation when AI is optional
- create hidden pressure to widen fallback authority

No-AI fallback is a valid and necessary route.

---

19. Routing Transparency Rules

For meaningful AI-assisted tasks, the system should be able to answer:

- what model family was selected
- whether the route was default or specialized
- whether fallback occurred
- whether degraded mode occurred
- whether no-AI fallback occurred

Operators do not need every infrastructure detail, but routing path visibility is necessary for trust and debugging.

---

20. Routing Change Control Rules

Model routing changes should be governed.

Examples of meaningful routing changes:

- changing default model family for a role
- enabling fallback where none existed
- allowing a weaker model on a previously protected path
- upgrading a model into validate-first structured proposal tasks

These changes should require explicit governance update, not silent deployment drift.

Routing policy is part of the safety surface.

---

21. Failure Handling Expectations

If routing fails:

- do not invoke a non-approved model
- do not silently downgrade protected workflows into unsafe assistance
- use explicit fallback if allowed
- otherwise degrade to no-AI/manual path
- preserve traceability of routing failure where required

If a selected model returns unusable output:

- apply output validation rules
- fallback only if the task allows it
- otherwise reject and degrade safely

Routing failure must never widen authority.

---

22. Audit Expectations

For meaningful AI-assisted tasks, the system should be able to answer:

- which model family was selected
- whether fallback was used
- whether routing was default or specialized
- whether degraded mode occurred
- whether routing failure caused no-AI/manual handling

This should integrate with AIG-06 traceability expectations.

---

23. Required Guarantees

Model Routing + Fallback Policy must guarantee:

- model choice is explicit
- fallback behavior is explicit
- fallback does not widen AI authority
- degraded mode does not weaken validation or human review requirements
- no-AI fallback remains possible
- protected workflows have stricter routing discipline than ordinary assistance tasks

---

24. Success Criteria

AIG-07 is correctly established when:

- default model routes are explicit
- specialist routes are explicit
- fallback eligibility is explicit
- no-fallback cases are explicit
- degraded-mode behavior is explicit
- routing changes require deliberate governance updates
- operators can inspect routing path decisions for meaningful AI-assisted workflows

---

25. Summary Statement

Model Routing + Fallback Policy defines how Axion selects the appropriate AI model for each approved task surface and how it degrades safely when the preferred route is unavailable or unsuitable.

It ensures that model choice remains task-aware, risk-aware, replaceable, and subordinate to the same deterministic and human-governed authority boundaries as every other part of the AI assistance layer.